

RAPPORT DE PROJET DE FIN D'ETUDES

POUR L'OBTENTION DU DIPLOME

LICENCE SCIENCES ET TECHNIQUES

SPECIALITE : GENIE INFORMATIQUE

Central Projet

Réalisé par :

KHADIJA BOUALLAOUI

Sous la direction de :

M. NABIL et LAAFCHOUBI, professeur à la FST de Settat

M. HOFFMAN et NGOUMA, Société ISIUM Développement Web

Soutenu le : 12 juin 2025

JURY

M. FOUAD et TOUFIK, professeur à la FST de Settat

M. AMINE et BENMAKHLOUF, professeur à la FST de Settat

M. NABIL et LAAFCHOUBI, professeur à la FST de Settat

Année Universitaire : 2024-2025

DEDICAS

Je dédie ce projet à :

À mes chers parents,

C'est avec une immense reconnaissance et une profonde émotion que je vous dédie ce travail.

Votre patience infinie, votre soutien inconditionnel, et vos encouragements constants ont été ma plus grande force tout au long de ce parcours.

Vous avez toujours cru en moi, même dans les moments de doute, et vos sacrifices silencieux mais immenses m'ont permis d'avancer, de rêver, et de concrétiser ce projet.

C'est grâce à votre amour indéfectible, votre sagesse et votre bienveillance que j'ai pu franchir cette étape importante de ma vie.

Je vous dois tout. Merci pour tout ce que vous êtes.

À mon cher petit frère,

Ta présence, ton affection sincère et ton soutien discret mais si précieux m'ont accompagné chaque jour.

Ta tendresse, tes encouragements et ton regard admiratif m'ont souvent donné le courage de persévérer.

À toi, qui es bien plus qu'un frère, je souhaite que ce travail t'inspire autant que ton amour m'a inspiré.

Reçois ici l'expression de ma profonde gratitude et de mon affection.

Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer mes sincères remerciements à tous ceux qui ont contribué à l'élaboration de ce travail.

Je remercie tout particulièrement **M. Ait Kaddour Said**, directeur de l'entreprise **ISIUM Développement Web**, ainsi que l'ensemble du personnel pour leur accueil, leur encadrement et leur soutien tout au long de cette expérience.

Je tiens également à exprimer ma profonde gratitude à mes encadrants, **M. Hoffman Ngouma** et **M. Nabil Laafchoubi**, pour leur guidance précieuse, leurs conseils avisés et leur disponibilité, qui m'ont permis de mener à bien ce projet dans les meilleures conditions. Enfin, je remercie chaleureusement tous les membres de l'équipe technique pour leur esprit de collaboration, leur soutien et leurs précieuses suggestions, qui ont été essentiels à la réalisation de ce projet.

À tous ceux qui, de près ou de loin, ont contribué à la réussite de ce travail, je vous adresse mes plus sincères remerciements et ma profonde reconnaissance.

Résumé

L'objectif de mon projet de fin d'études, réalisé au sein de l'entreprise ISIUM

Développement Web, est de concevoir et développer une application web dédiée à la gestion de projets. Cette application permet de centraliser la gestion de différents types de projets (web, desktop, mobile, etc.) tout en assurant un suivi efficace des éléments essentiels tels que les hébergements, les noms de domaine, et les licences.

Le système intègre un module de notifications automatiques, notamment pour alerter les utilisateurs avant l'échéance des hébergements et services associés, ainsi qu'une interface intuitive et sécurisée grâce à un mécanisme d'authentification par identifiants.

Pour atteindre ces objectifs, j'ai utilisé le Framework **Symfony 7** pour le développement du **backend**, tandis que le **frontend** a été réalisé en **HTML**, **CSS** et **Bootstrap** pour une interface responsive. Des technologies comme **JavaScript**, **jQuery**, **AJAX** et **JSON** ont été utilisées pour dynamiser l'application. La phase de conception a été structurée à l'aide de diagrammes **UML** pour une modélisation claire et rigoureuse du système.

Ce projet vise à fournir à l'entreprise une solution robuste, sécurisée et évolutive pour mieux organiser, superviser et suivre ses projets informatiques

Abstract

The objective of my end-of-studies project, carried out within the company ISIUM Développement Web, is to design and develop a web application dedicated to project management. This application centralizes the management of various types of projects (web, desktop, mobile, etc.) while ensuring effective tracking of key elements such as hosting, domain names, and software licenses.

The system includes an automatic notification module, particularly to alert users before the expiration of hosting services and related items, as well as an intuitive and secure interface through an authentication mechanism.

To achieve these objectives, I used the **Symfony 7** framework for backend development, while the frontend was built with **HTML**, **CSS**, and **Bootstrap** to ensure a responsive interface. Technologies such as **JavaScript**, **jQuery**, **AJAX**, and **JSON** were used to enhance interactivity. The design phase was supported by **UML diagrams** to provide a clear and structured system model.

This project aims to provide the company with a robust, secure, and scalable solution to better organize, manage, and monitor its IT projects.

Table de figures

Figure 1. Organigramme d'isum.	Page 19
Figure2. Diagramme de Gantt.	Page 21
Figure 3. Diagramme de contexte statique.	Page 26
Figure 4. Cas d'utilisation admin.	Page 28
Figure 5. Cas d'utilisation utilisateur.	Page 29
Figure 6. Diagramme séquence Connexion.	Page 30
Figure 7. Diagramme de séquence Inscription.	Page 31
Figure 8. Diagramme de séquence Ajout projet.	Page 31
Figure 9. Diagramme de séquence modification projet.	Page 32
Figure 10. Diagramme de séquence suppression projet.	Page 33
Figure 11. Diagramme de séquence rechercher projet.	Page 33
Figure 12. Diagramme de séquence plus détail projet.	Page 34
Figure 13. Diagramme de séquence ping url du projet.	Page 35
Figure 14. Diagramme de séquence modification hébergement.	Page 36
Figure 15. Diagramme de séquence suppression hébergement.	Pag 36
Figure 16. Diagramme de séquence rechercher hébergement.	Page 37
Figure 17. Diagramme de séquence plus détail hébergement.	Page 38
Figure 18. Diagramme de séquence Ajout technologie	Page 38
Figure 19. Diagramme de séquence modification technologie.	Page 39
Figure 20. Diagramme de séquence suppression technologie.	Page 41
Figure 21. Diagramme de séquence recherche technologie	Page 41

Figure 22. Diagramme de séquence plus détail technologie.	Page 41
Figure 23. Diagramme de séquence Ajout développeur.	Page 42
Figure 24. Diagramme de séquence modification développeur.	Page 42
Figure 25. Diagramme de séquence suppression développeur.	Page 43
Figure 26. Diagramme de séquence recherche développeur.	Page 43
Figure 27. Diagramme de séquence recherche développeur	Page 44
Figure 28. Diagramme de séquence notification et alerte.	Page 45
Figure 29. Diagramme de séquence modification projet user	Page 46
Figure 30. Diagramme d'activité inscription.	Page 47
Figure 31. Diagramme d'activité connexion.	Page 47
Figure 32. Diagramme d'activité Ajout projet.	Page 49
Figure 33. Diagramme d'activité modification projet.	Page 50
Figure 34. Diagramme d'activité rechercher projet.	Page 52
Figure 35. Diagramme d'activité ajout technologie.	Page 53
Figure 36. Diagramme d'activité modification technologie	Page 53
Figure 37. Diagramme d'activité suppression technologie.	Page 54
Figure 38. Diagramme d'activité recherche technologie.	Page 54
Figure 39. Diagramme d'activité Ajout hébergement.	Page 55
Figure 40. Diagramme d'activité modification hébergement	Page 55
Figure 41. Diagramme d'activité suppression hébergement.	Page 56
Figure 42. Diagramme d'activité recherche hébergement	Page 56
Figure 43. Diagramme d'activité ajout développeur.	Page 58
Figure 44. Diagramme d'activité modification développeur.	Page 58
Figure 45. Diagramme d'activité suppression développeur.	Page 59
Figure 46. Diagramme d'activité recherche développeur	Page 59

Figure 47. Diagramme d'activité alerte et notification	Page 60
Figure 48. Diagramme de classe.	Page 61
Figure 49. Schéma des patrons utilisées.	Page 74
Figure 50 Structure du projet complète 1	Page 78
Figure 51 Structure du projet /src	Page 79
Figure 52 Structure du projet /template	Page 79
Figure 53. Schéma de la base de données.	Page 84
Figure 54. Page d'accueil.	Page 85
Figure 55. Page Inscription.	Page 85
Figure 56. Page connexion.	Page 86
Figure 57. Page de réinitialisation du mot de passe.	Page 86
Figure 58. Page Dashboard admin.	Page 87
Figure 59. Page principale projet.	Page 87
Figure 60. Pop-up ajout projet.	Page 88
Figure 61. Pop-up modification proje	Page 88
Figure 62. Pop-up suppression projet.	Page 89
Figure 63. Page de recherche pleine	Page 89
Figure 64. Page de recherche vide.	Page 90
Figure 65. Pop-up plus détail du projet.	Page 90
Figure 66. Page erreur du teste url du projet.	Page 91
Figure 67. Page succès du teste url du projet.	Page 91
Figure 68. Page principale hébergement	Page 92
Figure 69. Pop-up ajout hébergement.	Page 92
Figure 70. Pop-up modification hébergement.	Page 93
Figure 71. Pop-up suppression hébergement.	Page 93
Figure 72. Page recherche hébergement pleine.	Page 94
Figure 73. Page recherche hébergement vide.	Page 94
Figure 74. Pop-up plus détail hébergement.	Page 95
Figure 75. Page principale technologie.	Page 95
Figure 76. Pop-up ajout technologie.	Page 96
Figure 77. Pop-up modification technologie.	Page 96
Figure 78. Pop-up suppression technologie.	Page 97

Figure 79. Page de recherche technologie pleine.	Page 98
Figure 80. Page de recherche technologie vide.	Page 98
Figure 81. Pop-up plus détail technologie.	Page 99
Figure 82. Page principale développeur.	Page 99
Figure 83. Pop-up ajout développeur.	Page 100
Figure 84. Pop-up modifier développeur.	Page 100
Figure 85. Pop-up suppression développeur.	Page 101
Figure 86. Page de recherche développeur pleine.	Page 101
Figure 87. Page de recherche développeur vide.	Page 102
Figure 88. Pop-up plus détails développeur.	Page 102
Figure 89. Page Mon compte	Page 103
Figure 90. Page Alertes et notifications pleine.	Page 103
Figure 91. Page Alertes et notifications vide.	Page 104
Figure 92. Page Historique.	Page 104
Figure 93. Page Dashboard user.	Page 105
Figure 94. Pop-up plus détail carte projet.	Page 105
Figure 95 Page projet assignées pleine	Page 106
Figure 96. Page projet assignées vide.	Page 106
Figure 97. Pop-up modification projet assignées	Page 107
Figure 98. Pop-up plus détail projet assignées.	Page 107
Figure 99. Résultat test NotificationService.	Page 111
Figure 100. Résultat test UrlService	Page 111
Figure 101. Résultat test UrlCheckerService	Page 111
Figure 102. Résultat test HistoriqueService	Page 112
Figure 103. Résultat test Entity Developpeur.	Page 113
Figure 104. Résultat test Entity Projet..	Page 113
Figure 105. Résultat test Entity Hebergement..	Page 113
Figure 106. Résultat test Entity Technologie.	Page 114
Figure 107. Résultat test Entity Notification.	Page 114

Figure 108. Résultat test Entity Historique	Page 114
Figure 109 Résultat test Entity Use	Page 114
Figure 110. Résultat test HistoriqueEventSubscriber	Page 115
Figure 111. Résultat test NotificationSubscriber.	Page 115
Figure 112.Diagramme de Gantt réel.	Page 117

Table de tableaux

Tableaux 1 : Patrons Comportemental	Page 71
Tableaux 2 : Patrons de création	Page 72
Tableaux 3 : Patrons architectural	Page 73
Tableaux 4 : Patrons structurel	Page 73

SOMMAIRE

DEDICAS.....	2
Remerciements	3
Résumé.....	4
Abstract	5
Table de figures.....	6
Table de tableaux.....	11
Chapitre 1 : Contexte générale du projet	22
I. Présentation de l'organisme d'accueil.....	22
II. Organigramme.....	22
III. Métiers/Activité de l'entreprise /Secteur d'activité.....	23
IV. Contexte du projet	24
1. Critique de l'existant.....	24
2. Objectifs du projet.....	24
3. Solution proposée	24
4. Conduite du projet/Méthodologie de travail.....	25
5. Planification du projet/Planning prévisionnel du déroulement du projet.....	25
6. Diagramme de Gantt	25
V. Conclusion.....	25
Chapitre 2 : Analyse et conception	28
I. Besoins fonctionnels.....	28
1. Identification et description des acteurs	28
2. Diagramme de contexte statique	29
3. Cas d'utilisation/fonctionnalités attendues	30
a. Diagramme de cas d'utilisation admin :	30
b. Diagramme de cas d'utilisation utilisateur :	31
4. Diagramme de séquences :.....	32
a. Connexion :	32
b. Admin :.....	33

i.	Inscription :	33
ii.	Gestion projets :	34
	(1) Ajout projet :	34
	(2) Modifier projet :	35
	(3) Supprimer projet :	35
	(4) Recherche projet :	36
	(5) Voir plus :	37
	(6) Ping url :	37
iii.	Gestion hébergements :	38
	(1) Ajout hébergement :	38
	(2) Modifier hébergement :	38
	(3) Supprimer hébergement :	39
	(4) Rechercher un hébergement :	39
	(5) Voir plus :	40
iv.	Gestion technologies :	40
	(1) Ajout technologie :	40
	(2) Modifier technologie :	41
	(3) Supprimer technologie :	42
	(4) Rechercher technologie :	42
	(5) Voir plus :	43
v.	Gestion développeurs :	43
	(1) Ajouter développeur :	43
	(2) Modifier développeur :	44
	(3) Supprimer développeur :	44
	(4) Rechercher développeur :	44
	(5) Voir plus :	45
vi.	Recevoir Notification & Alertes :	45
c.	Utilisateur :	46
	i. Modifiée projet :	46
5.	Diagramme d'activités :	47
	a. Inscription :	47

b.	Connexion :	48
c.	Gestion projets :	49
(1)	Ajout projet :	49
(2)	Modifier projet :	50
(3)	Supprimer projet :	51
(4)	Rechercher un projet :	52
d.	Gestion technologies :	53
(1)	Ajout technologie :	53
(2)	Modifier technologie :	54
(3)	Supprimer technologie :	55
(4)	Rechercher une technologie :	55
e.	Gestion hébergements :	56
(1)	Ajout hébergement :	56
(2)	Modifier un hébergement :	56
(3)	Supprimer un hébergement :	57
(4)	Rechercher un hébergement :	57
f.	Gestion développeurs :	57
(1)	Ajout développeur :	57
(2)	Modifier développeur :	58
(3)	Supprimer développeur :	58
(4)	Rechercher un développeur :	59
g.	Alerte et Notifications :	59
6.	Diagramme de classe :	60
II.	Besoins non fonctionnels /techniques	61
1.	La performance	61
2.	La sécurité	62
3.	La portabilité	62
4.	La maintenance	62
Chapitre 3 : Etude technique		65
I.	Architecture logicielle du système	65
1.	Couche présentation (Frontend)	65

2. Couche métier (Backend)	65
3. Couche d'accès aux données (Data)	65
II. Présentation de technologies utilisées	66
III. Patrons de conception utilisées	68
1. Définition :	68
2. Types de patrons de conception :	68
a. Patrons architecturaux :	68
b. Patrons de création :	69
c. Patrons structurels :	69
d. Patrons comportementaux :	70
3. Les patrons de conception utilisées :	71
a. Patrons Comportemental :	71
b. Patrons de création :	71
c. Patrons architectural :	72
d. Patrons structurel :	72
4. Schémas globaux des patrons utilisées :	73
IV. Le framework adopté	73
V. Les outils de développement utilisés	74
VI. Choix du SGBDR	74
Chapitre 4 : Réalisation/Mise en œuvre de la solution	77
I. Structure du projet	77
II. Schéma de la base de données	78
1. Table User :	79
2. Table Développeur :	79
3. Table Projet :	80
4. Table Hébergement :	80
5. Table Technologie :	81
6. Table Notification :	81
7. Table Historique :	82
III. Présentation des interfaces de l'application/Captures écrans	84
1. Page d'accueil :	84

2.	Page Registre :	84
3.	Page login :	85
4.	Page réinitialiser password :	85
5.	Interface Admin :	86
a.	Dashboard :	86
b.	Gestion Projets :	86
1)	Page principale :	86
2)	Pop-up ajout :	87
3)	Pop-up modification :	87
4)	Pop-up suppression :	88
5)	Pop-up voir plus de détails :	89
6)	Page d'erreur Url :	90
7)	Page de succès Url :	90
c.	Gestion Hébergements :	91
1.	Page principale :	91
2.	Pop-up ajout :	91
3.	Pop-up modification :	91
4.	Pop-up suppression :	92
5.	Page de rechercher :	92
6.	Pop –up voir plus de détails :	93
d.	Gestion Technologie :	94
1.	Page principale :	94
2.	Pop -up ajout :	94
3.	Pop-up modification :	95
4.	Pop-up suppression :	96
6.	Pop –up voir plus de détails :	97
e.	Gestion Développeur :	97
1.	Page principale :	97
2.	Pop-up ajout :	98
3.	Pop-up modification :	99
4.	Pop-up suppression :	99

5.	Page de rechercher :	100
6.	Pop –up voir plus de détails :	101
f.	Consultation Compte :	101
g.	Alertes & Notification :	102
h.	Suivie des modifications :	103
6.	Interface Développeur :	103
a.	Page Dashboard user :	103
b.	Pop-up voire détail du projet :	104
c.	Page projet assignées :	104
d.	Page rechercher :	105
e.	Pop-up modification projet :	105
f.	Pop-up voir plus détails :	106
IV.	Les tests réalisés	106
1.	But des tests :	106
2.	Types des tests :	107
a.	Test unitaire :	107
b.	Test fonctionnel :	107
c.	Test d'intégration :	107
d.	Test de validation du formulaire :	108
e.	Test de sécurité :	108
3.	Test réalisé :	109
a.	Test unitaire :	109
i)	Service :	109
ii)	Entité :	110
iii)	EventSubscriber :	112
V.	La qualité du code	112
VI.	Diagramme de Gantt réel	113
VII.	Conclusion générale et perspective	113
1.	Conclusion générale :	113

2. Perspective :	114
VIII. Bibliographie/Webographie.....	115
IX. Annexes	116
Annexe 1 : Captures écrans script des tâches crônes :.....	116
Annexe 3 : Accès code source :	116



INTRODUCTION GENERALE

Introduction Générale :

En tant qu'étudiante en licence en Génie Informatique, nous étions tenus d'effectuer, durant le second semestre de cette année, un stage de fin d'études (PFE) en entreprise. L'objectif principal de ce stage est de compléter notre formation académique par une expérience professionnelle, aujourd'hui indispensable pour favoriser une bonne insertion dans le marché du travail après l'obtention du diplôme.

Ce stage a également été pour moi l'occasion de me familiariser avec le fonctionnement d'une entreprise, tout en mettant en pratique mes connaissances théoriques pour répondre à une problématique concrète.

Mon objectif était de concevoir une application web permettant à l'entreprise de centraliser toutes les informations relatives à ses projets, de suivre leur état d'avancement, et de recevoir des notifications concernant les dates d'échéance des hébergements.

Ce rapport est structuré en quatre chapitres :

- **Chapitre 1** : Présentation de l'organisme d'accueil et du contexte général du projet.
- **Chapitre 2** : Étude d'analyse et de conception des besoins fonctionnels et non fonctionnels.
- **Chapitre 3** : Étude technique comprenant les outils et technologies utilisés pour le développement de l'application.
- **Chapitre 4** : Description de la phase de mise en œuvre de l'application.

Le rapport se termine par une conclusion mettant en évidence la valeur ajoutée de l'application développée ainsi que les perspectives d'évolution envisagées.



Chapitre 1 :

Contexte générale du projet

Chapitre 1 : Contexte générale du projet

I. Présentation de l'organisme d'accueil

Isium est une entreprise dynamique et innovante, spécialisée dans le développement de logiciels sur mesure. Avec des implantations stratégiques en France et au Maroc, Isium bénéficie d'une double expertise qui lui permet de conjuguer proximité client et maîtrise des meilleures pratiques internationales en matière de technologies de l'information.

Fondée sur des valeurs d'innovation, de qualité et d'écoute, Isium se distingue par sa capacité à concevoir des solutions logicielles personnalisées répondant précisément aux besoins spécifiques de chaque client. Qu'il s'agisse d'applications métiers, de plateformes web, ou de systèmes intégrés, Isium apporte une expertise technique solide couplée à une compréhension fine des enjeux métiers de ses partenaires.

L'entreprise s'appuie sur une équipe pluridisciplinaire composée de développeurs, chefs de projet, analystes fonctionnels et experts techniques, qui travaillent en synergie pour garantir la réussite des projets. Cette organisation permet à Isium d'assurer un accompagnement complet, depuis l'étude des besoins jusqu'à la mise en production, en passant par le développement, les tests et la maintenance.

Grâce à une méthodologie agile, Isium favorise la collaboration continue avec ses clients, garantissant ainsi une adaptation rapide aux évolutions et une livraison progressive des fonctionnalités. Cette approche garantit la flexibilité et la qualité, tout en optimisant les délais et les coûts.

En résumé, Isium s'impose comme un partenaire de confiance pour les entreprises souhaitant accélérer leur transformation digitale à travers des solutions logicielles sur mesure, innovantes, évolutives et parfaitement adaptées à leurs objectifs stratégiques.

II. Organigramme

L'entreprise Isium dispose d'une organisation claire et hiérarchisée, permettant une gestion efficace des projets et une bonne coordination entre les différents métiers.

Au sommet de la structure se trouve le **Directeur Général, M. Said AITKADDOUR**, qui supervise la stratégie globale de l'entreprise et veille à son développement.

Sous sa responsabilité directe, le **Directeur Technique, M. Brahim ELAJI**, pilote l'ensemble des activités techniques. Il assure la qualité des solutions développées et coordonne les équipes techniques.

Le **Chef de Projet, M. Hoffman NGOUMA**, est chargé de la planification, du suivi et de la réussite des projets clients. Il fait le lien entre les équipes techniques et les clients afin d'assurer la bonne réalisation des livrables dans les délais impartis.

Les équipes techniques comprennent des **Analystes Programmeurs**, tels que **M. Houcine AIMARAH** et **M. Issam HANNAOUI**, qui sont responsables de l'analyse des besoins, du développement, des tests et de la maintenance des logiciels .

La figure ci-dessous représente l'organigramme de l'entreprise ISIUM :

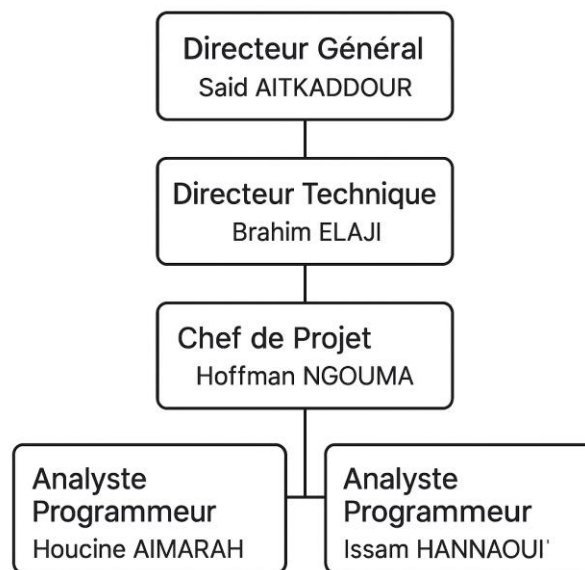


Figure 1. Organigramme d'isum.

III. Métiers/Activité de l'entreprise /Secteur d'activité

Isium évolue dans le secteur du développement informatique, un domaine en constante évolution au cœur de la transformation numérique des entreprises. Elle se spécialise dans la conception, le développement et la maintenance de solutions logicielles personnalisées, en s'adaptant aux besoins spécifiques de chaque client.

L'entreprise intervient sur plusieurs volets :

- **Développement de logiciels sur mesure** : pour des besoins métiers précis, souvent internes à une entreprise (gestion de projet, reporting, automatisation de processus, etc.).
- **Développement d'applications web** : accessibles via navigateur, ces solutions offrent une interface intuitive, une accessibilité multiplateforme et une centralisation des données.

- **Développement d'applications mobiles** : pour répondre aux besoins d'accessibilité en mobilité, Isium propose des solutions compatibles Android/iOS, que ce soit en natif ou via des technologies hybrides ou PWA.

Isium se positionne ainsi comme un acteur innovant dans le domaine des technologies de l'information, apportant une réelle **valeur ajoutée aux entreprises** grâce à des solutions digitales performantes, évolutives et centrées sur l'utilisateur.

IV. Contexte du projet

1. Critique de l'existant

Au sein de l'entreprise ISIUM Développement Web, la gestion des projets se fait de manière manuelle et dispersée. Les informations relatives aux projets (état avancement, développeurs impliqués, accès aux environnements de travail, échéances des hébergements ou licences) sont souvent stockées dans différents fichiers ou outils non centralisés. Cela complique le suivi, la mise à jour des données, et accroît les risques d'oubli, notamment en ce qui concerne les dates critiques comme l'expiration des hébergements.

L'absence d'un outil centralisé de gestion de projet limite la visibilité globale sur les projets en cours et complique la coordination entre les développeurs. De plus, le manque de système de notifications entraîne fréquemment des oublis d'échéances importantes (hébergements, domaines, licences), ce qui peut nuire à la continuité des services de l'entreprise.

2. Objectifs du projet

L'objectif principal du projet est de développer une application web sécurisée qui permet à l'entreprise :

- De centraliser toutes les informations liées aux projets.
- D'attribuer des développeurs à chaque projet.
- D'assurer un suivi en temps réel des échéances (hébergements, domaines...).
- De générer automatiquement des notifications de rappel avant les dates critiques.
- De tracer l'historique des modifications par utilisateur

3. Solution proposée

La solution retenue consiste à concevoir une application web de gestion de projets. Elle offrira :

- Une interface intuitive et responsive pour les utilisateurs.
- Un système d'authentification sécurisé.
- Des modules dédiés : projets, développeurs, hébergements, notifications, historique.
- Un système de notifications automatique à J-15 avant expiration des hébergements

4. Conduite du projet/Méthodologie de travail

Le projet a été mené en suivant une approche itérative inspirée de la méthode agile :

- Découpage en modules fonctionnels.
- Développement incrémental par étapes.
- Tests réguliers après chaque module finalisé.
- Ajustements en fonction des retours de l'entreprise encadrante.

5. Planification du projet/Planning prévisionnel du déroulement du projet

Le projet s'est étalé sur un mois, du 14 avril au 15 mai 2025. Il a été découpé en plusieurs phases : apprentissage, analyse, conception, développement, tests, et finalisation. Chaque phase a été planifiée selon le temps disponible et les priorités fonctionnelles.

6. Diagramme de Gantt

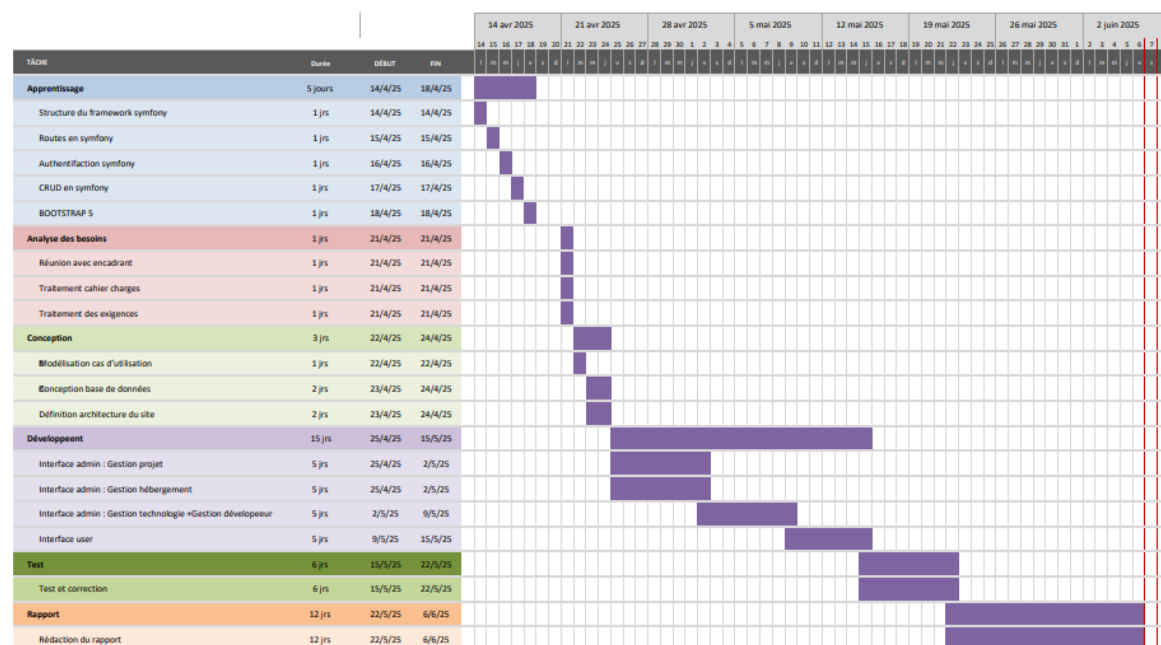


Figure2. Diagramme de Gantt.

V. Conclusion

À l'issue de ce premier chapitre, nous avons pu identifier clairement le contexte dans lequel s'inscrit le projet de développement d'une application de gestion des projets et des tâches au sein de l'entreprise. Cette analyse approfondie de l'environnement organisationnel, des rôles des intervenants, ainsi que des processus métiers existants, a permis de mettre en lumière les besoins spécifiques en matière de centralisation des informations, de suivi des échéances, et de collaboration entre les équipes.

La présentation de l'entreprise et de sa structure hiérarchique nous a permis de mieux comprendre les flux de communication et de responsabilité, ce qui sera essentiel pour

concevoir une solution adaptée à la réalité du terrain. Ce travail préliminaire constitue une base solide pour aborder les phases de conception, de modélisation et de mise en œuvre technique dans les chapitres suivants.



Chapitre 2 :

Analyse et

conception

Chapitre 2 : Analyse et conception

I. Besoins fonctionnels

1. Identification et description des acteurs

Un acteur joue un rôle d'une entité externe (utilisateur humain, dispositif matériel ou autre système) qui interagit directement avec le système étudié. Pour notre application, nous identifions principalement deux types d'acteurs :

❖ Elle permet à l'administrateur de :

⇒ S'authentifier :

- À travers une adresse électronique (e-mail) et un mot de passe.

⇒ Gestion des projets :

- Ajouter des projets.
- Modifier un projet.
- Supprimer un projet.
- Afficher le détail d'un projet.
- Affecter à un projet une ou plusieurs technologie(s).
- Affecter à un projet un ou plusieurs hébergement(s).
- Affecter à un projet un développeur.
- Une recherche d'un projet selon le nom du projet.

⇒ Gestion des technologies :

- Ajouter des technologies.
- Modifier une technologie.
- Supprimer une technologie.
- Ajouter un nouveau développeur.
- Modifier un développeur.
- Supprimer un développeur.
- Afficher le détail d'un développeur.
- Une recherche d'un développeur selon le nom du développeur.

⇒ Gestion des hébergements :

- Ajouter des hébergements.
- Modifier un hébergement.
- Supprimer un hébergement.

- Afficher le détail d'un hébergement.
- Une recherche d'un hébergement selon le fournisseur

⇒ Recevoir Alertes & Notifications :

- Recevoir une alerte sur l'application d'expiration d'un hébergement.
- Une recherche d'une alerte selon le nom du fournisseur.

⇒ Suivre des Historiques :

- Suivre des modifications par utilisateur.

❖ **Elle permet à l'utilisateur de :**

⇒ S'authentifier :

- À travers une adresse électronique (e-mail) et un mot de passe.

⇒ Gestion des projets assignés :

- Consulter les projets assignés.
- Modifier un projet.
- Une recherche d'un projet selon le nom du projet.

⇒ Gestion Statuts des projets :

- Mentionner l'état d'avancement d'un projet.

2. Diagramme de contexte statique

Le diagramme de contexte statique est un outil utilisé en phase d'analyse pour représenter de manière globale et simplifiée le système étudié et ses interactions avec les acteurs externes (utilisateurs, systèmes tiers, etc.). Il permet de visualiser les flux d'informations entre le système et son environnement sans entrer dans les détails internes du fonctionnement.

Le diagramme de contexte statique est une vue d'ensemble du système qui montre :

- Le système comme une seule entité (boîte noire)
- Les acteurs qui interagissent avec lui
- Les échanges de données ou d'informations entre eux

Il constitue le point de départ pour modéliser les besoins fonctionnels et comprendre le périmètre du système.

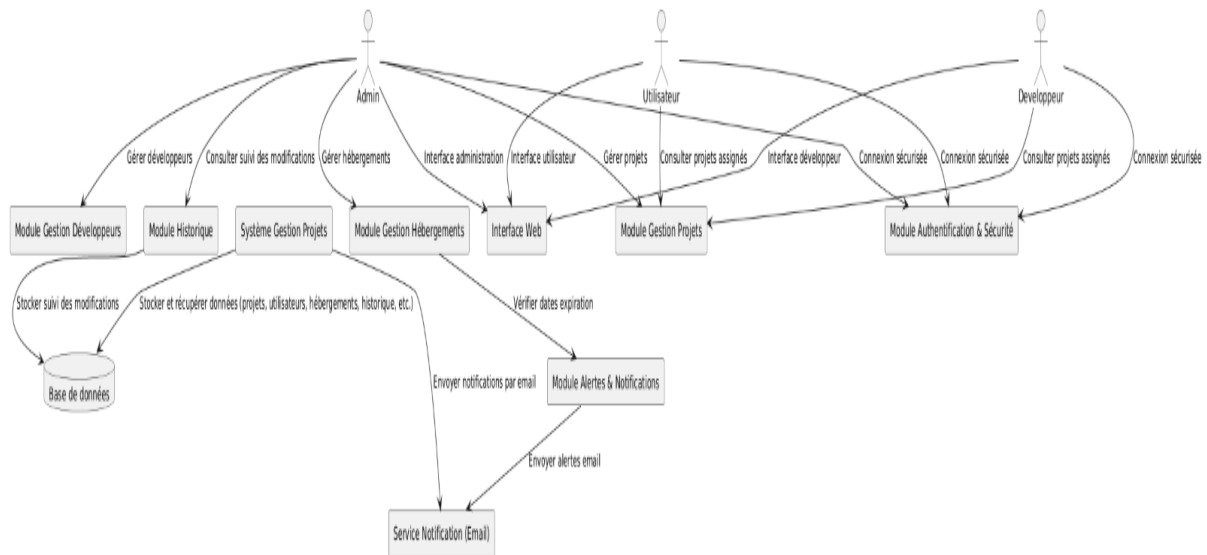


Figure 3. Diagramme de contexte statique.

3. Cas d'utilisation/fonctionnalités attendues

Dans le cadre de la modélisation des exigences fonctionnelles d'un système, le diagramme de cas d'utilisation joue un rôle fondamental. Il permet de représenter de manière claire et synthétique chaque usage que les acteurs font du système. En d'autres termes, chaque interaction entre un utilisateur (ou un système externe) et le système étudié est traduite par un cas d'utilisation.

Un cas d'utilisation (ou *use case*) illustre une fonctionnalité offerte par le système dans le but de répondre à un besoin précis exprimé par un acteur. Cette fonctionnalité, lorsqu'elle est exécutée, doit aboutir à un résultat attendu. Par exemple, dans une application de gestion de projets, un acteur comme un "Administrateur" pourra interagir avec le système à travers des cas d'utilisation tels que "Créer un projet", "Ajouter un utilisateur", ou encore "Consulter le tableau de bord".

Ainsi, le diagramme de cas d'utilisation ne se limite pas à une simple liste de fonctionnalités. Il décrit la manière dont le système doit se comporter vis-à-vis des acteurs externes en déterminant les besoins de ces derniers et les responsabilités précises que le système doit assumer pour y répondre.

a. Diagramme de cas d'utilisation admin :

L'interface Administrateur est destinée aux utilisateurs ayant des privilèges élevés leur permettant de gérer l'ensemble du système. Cet acteur joue un rôle central dans la configuration, la supervision et l'administration des différentes fonctionnalités offertes par l'application. À travers cette interface, l'administrateur peut effectuer des actions critiques telles que la création, la modification ou la suppression de projets, la gestion des utilisateurs,

le suivi de l'avancement des projets ainsi que l'analyse des performances globales du système. Le diagramme de cas d'utilisation correspondant décrit l'ensemble des interactions possibles entre l'administrateur et le système, en mettant en évidence les services et outils à sa disposition pour garantir un fonctionnement fluide et sécurisé de l'application.



Figure 4. Cas d'utilisation admin.

b. Diagramme de cas d'utilisation utilisateur :

L'interface Utilisateur est conçue pour les membres de l'équipe ou les collaborateurs qui interagissent quotidiennement avec le système dans le cadre de leurs tâches. Ces utilisateurs disposent d'un accès limité aux fonctionnalités selon leur rôle. Généralement, ils peuvent consulter les projets qui leur sont assignés, ajouter ou mettre à jour des tâches, suivre l'état d'avancement de leurs activités. Le diagramme de cas d'utilisation de cette interface permet de visualiser les différentes interactions entre l'utilisateur standard et le système, en mettant

en avant les fonctionnalités nécessaires pour accomplir ses objectifs tout en respectant les droits d'accès définis.

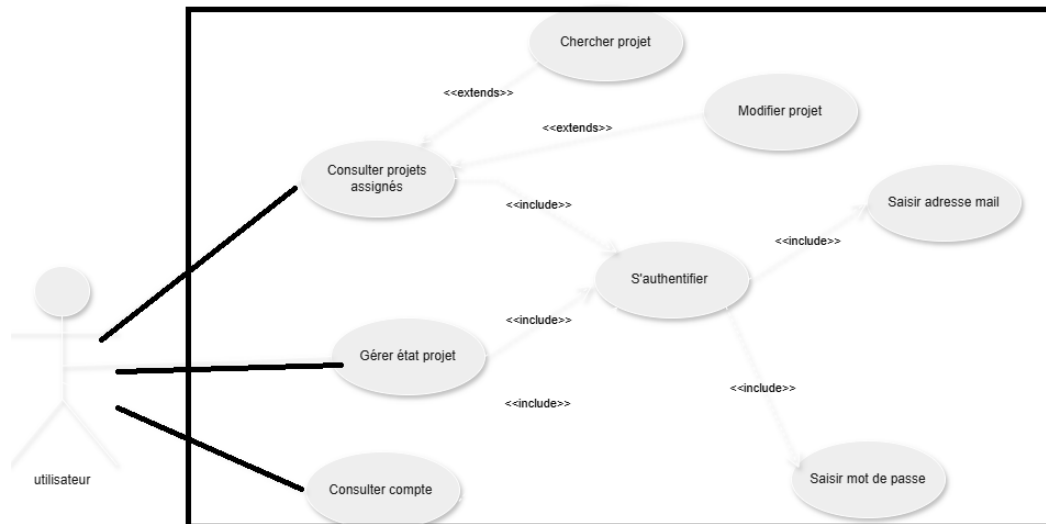


Figure 5. Cas d'utilisation utilisateur.

4. Diagramme de séquences :

Le diagramme de séquence est un diagramme d'interaction défini dans le langage UML. Il permet de modéliser la chronologie des échanges entre les différents objets ou composants d'un système au cours d'un scénario spécifique. Ce type de diagramme est particulièrement utile pour visualiser la dynamique des interactions entre les acteurs et le système.

a. Connexion :

L'interface de connexion permet à toute personne disposant d'un compte de s'authentifier à l'aide de son adresse email et de son mot de passe. Deux types d'utilisateurs peuvent se connecter :

- **Administrateur** : Accède à toutes les fonctionnalités (gestion des projets, développeurs, hébergements, notifications, etc.).
- **Utilisateur (développeur)** : Accède uniquement aux projets qui lui sont assignés (via le Kanban, les détails des projets, etc.).

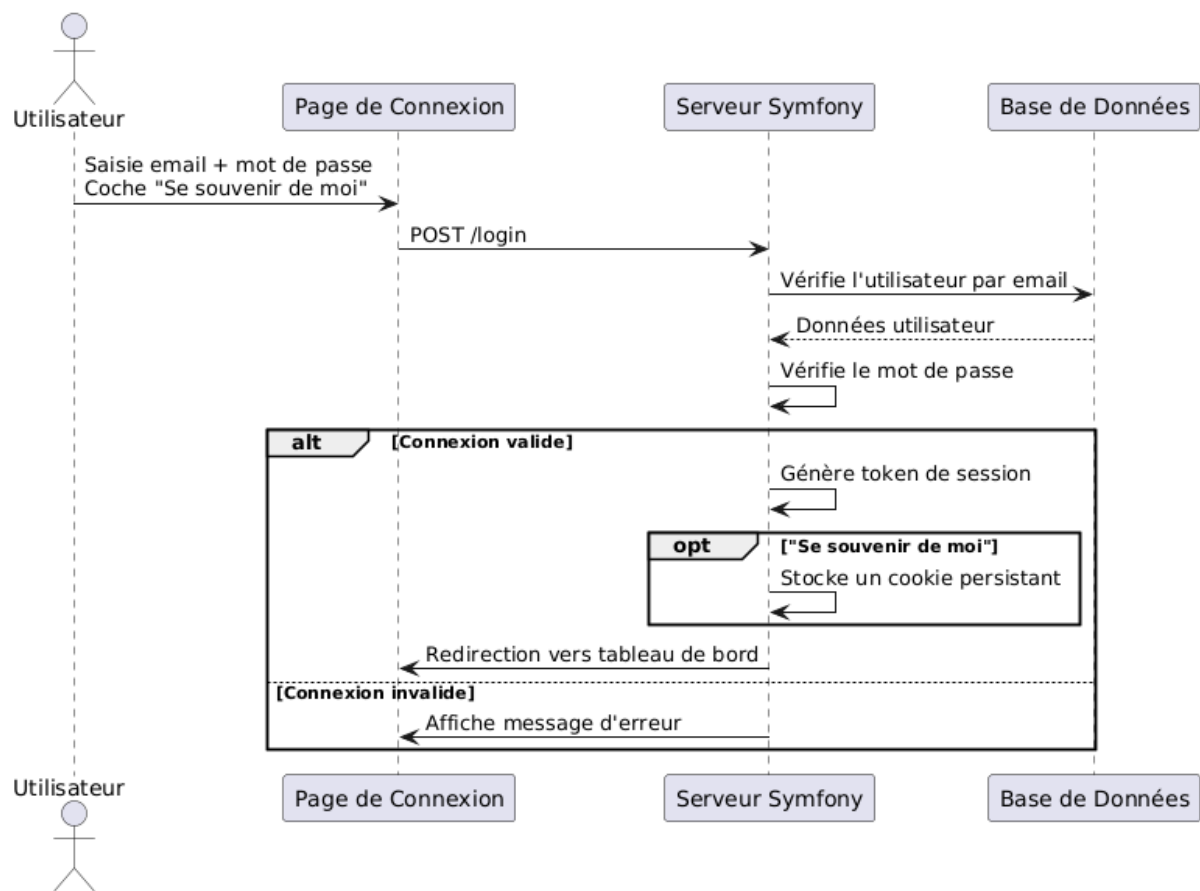


Figure 6. Diagramme séquence Connexion.

b. Admin :

i. Inscription :

L'inscription se fait via un formulaire où l'utilisateur renseigne son nom, prénom, email, mot de passe et accepte les conditions générales. Après soumission, le serveur vérifie la validité des données, notamment l'unicité de l'email. Si la validation est positive, le compte est créé et un email de confirmation est envoyé. L'utilisateur est ensuite redirigé vers la page de connexion.

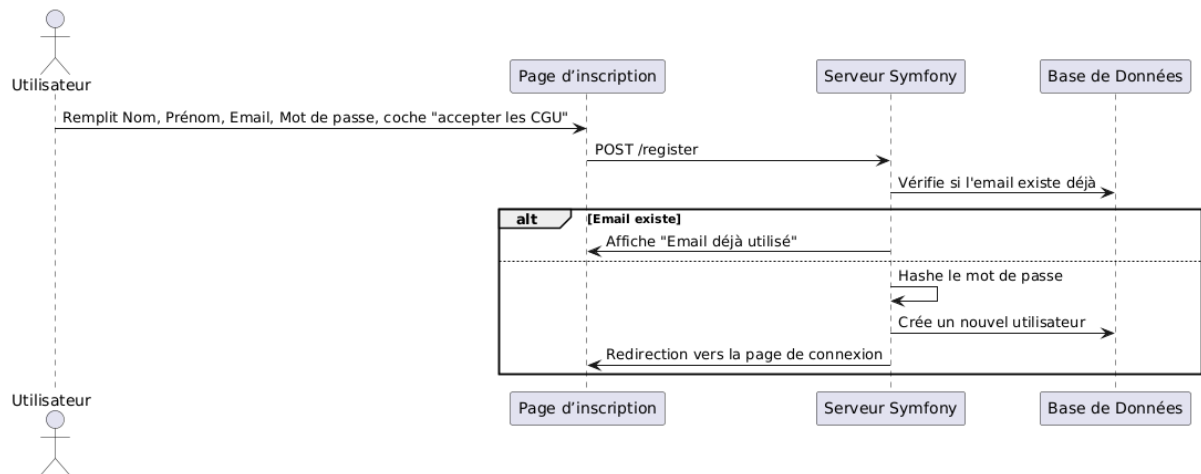


Figure 7. Diagramme de séquence Inscription.

ii. Gestion projets :

(1) Ajout projet :

Le diagramme de séquence ci-dessus décrit le processus d'ajout d'un nouveau projet par l'administrateur. Ce dernier commence par cliquer sur le bouton « Nouveau », ce qui affiche un formulaire vide. Il saisit ensuite les informations du projet et sélectionne les éléments associés comme les technologies, les hébergements et les développeurs. Lors de la soumission, le contrôleur Symfony crée une nouvelle instance de l'entité Projet, initialise ses propriétés, associe les entités liées, puis persiste le tout en base de données. Une fois l'opération réussie, un message de confirmation s'affiche à l'utilisateur.

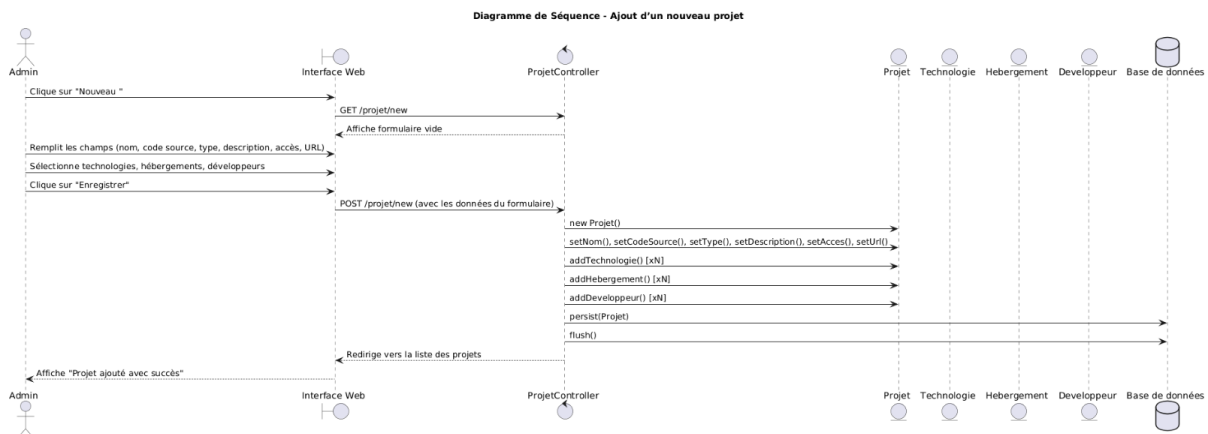


Figure 8. Diagramme de séquence Ajout projet.

(2) Modifier projet :

Ce diagramme illustre le scénario où un administrateur met à jour un projet existant. En cliquant sur l'icône « Modifier », l'interface affiche un formulaire pré-rempli avec les données du projet. L'admin modifie les informations souhaitées, puis envoie le formulaire. Le contrôleur Symfony met à jour les champs de l'entité Projet et applique les nouvelles associations. Enfin, il effectue un flush() pour sauvegarder les changements. Un message de confirmation est affiché

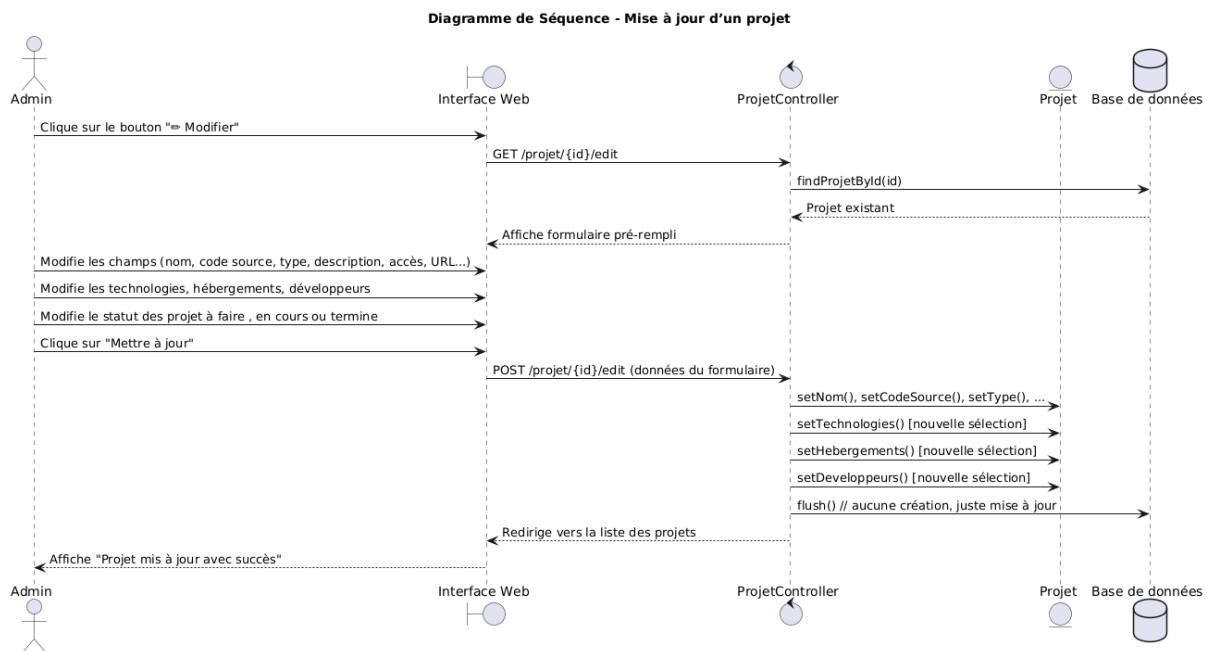


Figure 9. Diagramme de séquence modification projet.

(3) Supprimer projet :

Lorsqu'un projet est supprimé, l'admin déclenche une confirmation visuelle avant que le contrôleur n'exécute la suppression via Doctrine. La cohérence est assurée par la récupération préalable de l'objet via son identifiant.

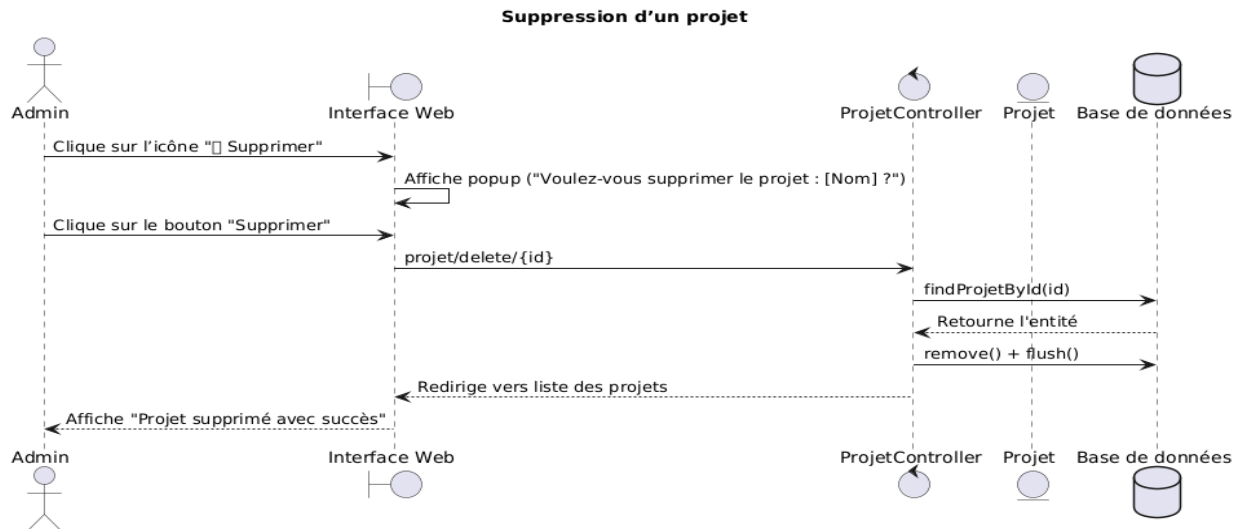


Figure 10. Diagramme de séquence suppression projet.

(4) Recherche projet :

L'administrateur saisit un nom de projet partiel dans une barre de recherche. Une requête AJAX déclenche une recherche dans la base. Les projets correspondants sont affichés dynamiquement. Si aucun résultat, un message d'erreur s'affiche.

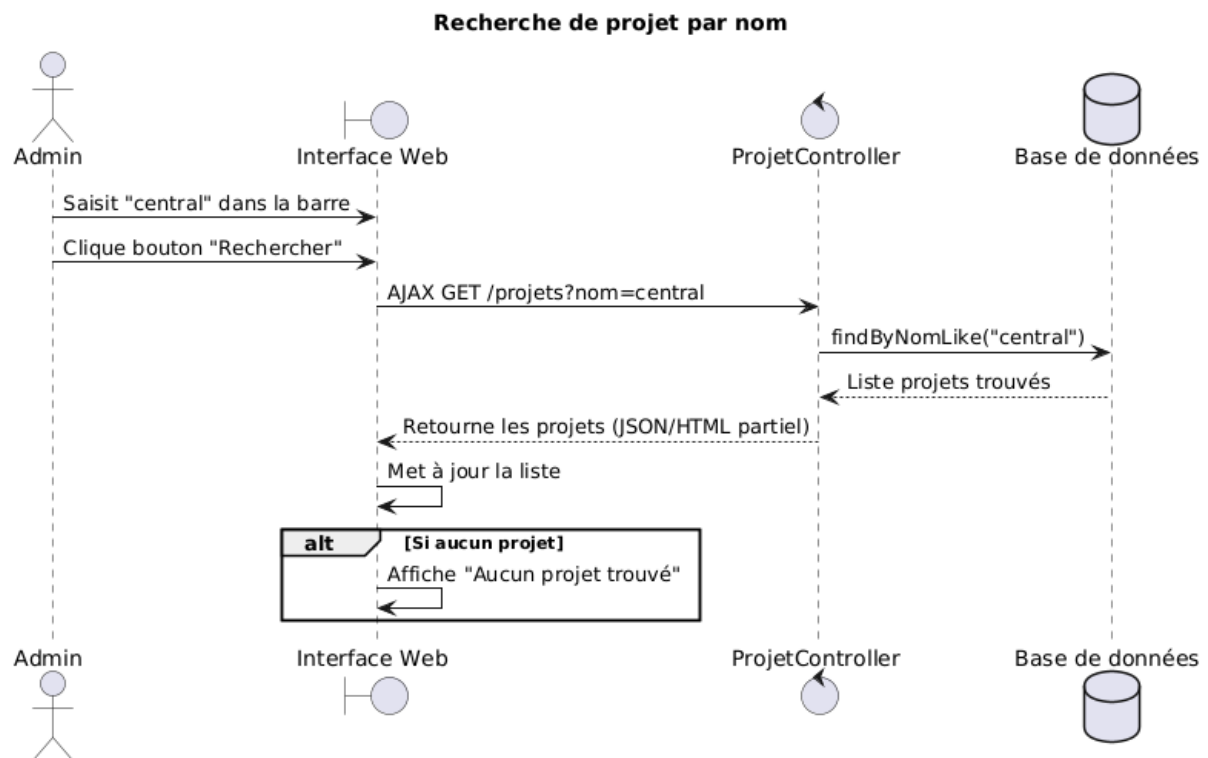


Figure 11. Diagramme de séquence rechercher projet.

(5) Voir plus :

Ce diagramme décrit le scénario où l'administrateur consulte les détails complets d'un projet via une interface web. Lors du clic sur le bouton "Voir plus", un popup dynamique s'ouvre, affichant :

- La liste des hébergements associés.
- Les technologies utilisées.
- Les développeurs impliqués.
- Les liens d'accès au code source et aux environnements.
- L'URL du projet, avec une alerte visuelle indiquant si cette URL fonctionne ou non (alerte verte = URL valide ; alerte rouge = URL invalide).
- Une requête AJAX permet de récupérer ces données depuis la base, et une vérification en temps réel est faite pour tester la disponibilité de l'URL du projet.

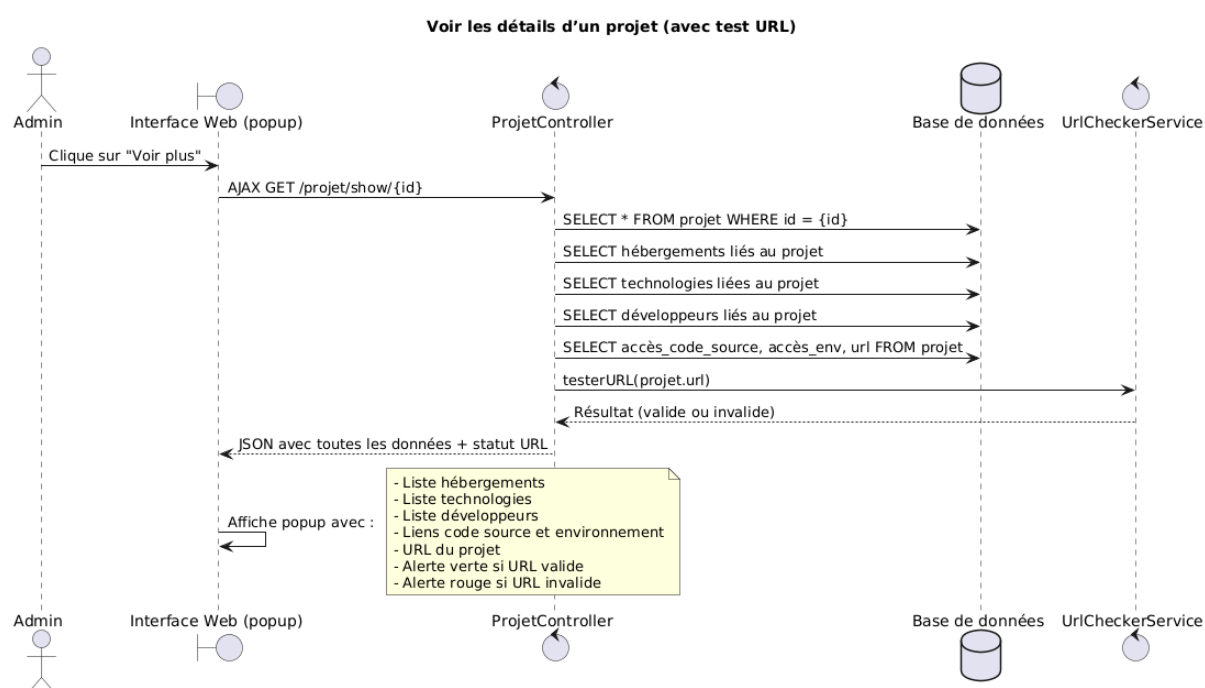


Figure 12. Diagramme de séquence plus détail projet.

(6) Ping url :

Lors de la consultation des détails d'un projet, l'administrateur dispose d'un bouton "Tester" dans la fenêtre popup qui permet de vérifier la disponibilité de l'URL associée au projet. Ce mécanisme a pour objectif de garantir que l'URL pointant vers l'environnement ou la ressource du projet est toujours fonctionnelle, ce qui est essentiel pour assurer la qualité et la fiabilité du suivi des projets.

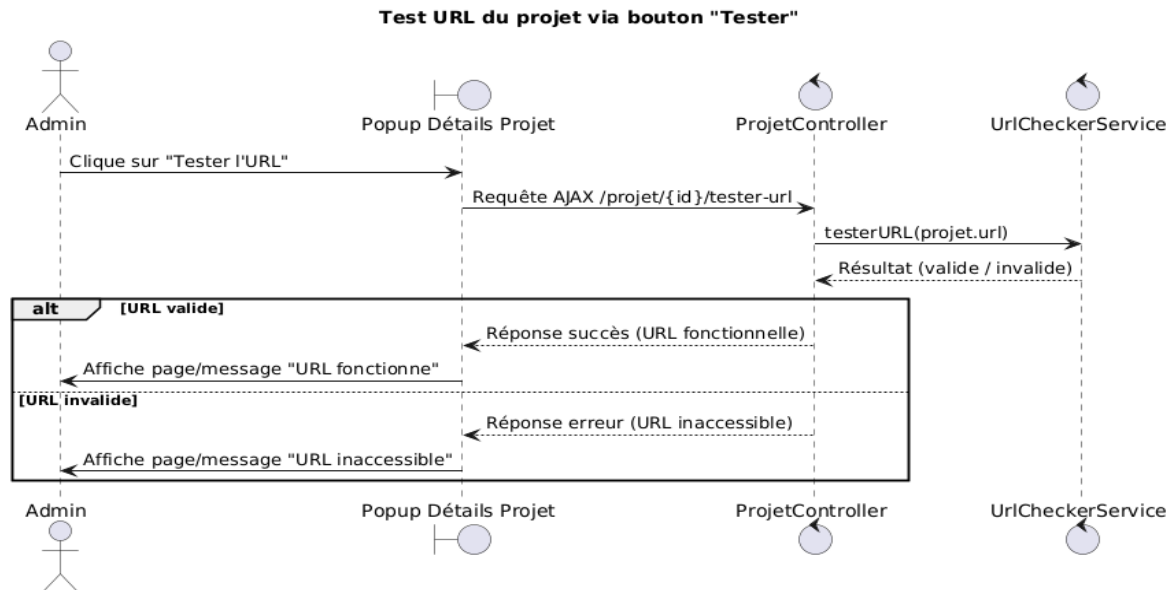


Figure 13. Diagramme de séquence ping url du projet.

iii. Gestion hébergements :

(1) Ajout hébergement :

Ce diagramme décrit le processus d'ajout d'un nouvel hébergement par un administrateur via une interface web. L'admin clique sur le bouton "Nouveau", un formulaire s'affiche, il entre les informations requises (fournisseur, date d'expiration, URL, type), puis soumet le formulaire. Les données sont ensuite validées et enregistrées dans la base de données.

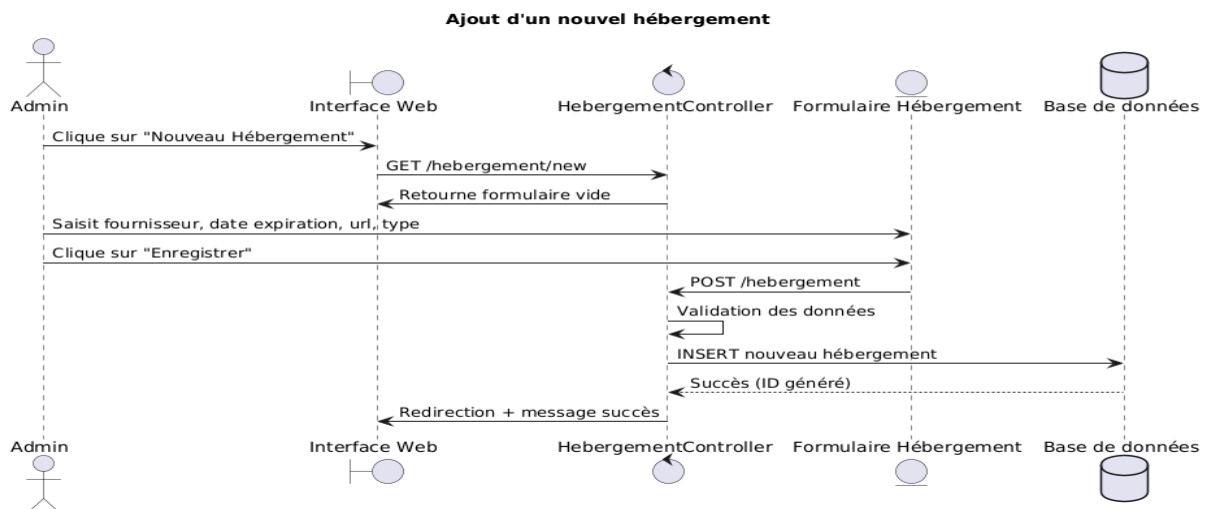


Figure 13. Diagramme de séquence Ajout hébergement.

(2) Modifier hébergement :

L'administrateur clique sur l'icône de modification d'un hébergement. Le formulaire s'ouvre avec les informations pré-remplies. Il peut modifier la date d'expiration, le fournisseur,

le type ou l'URL. Une fois le formulaire soumis, les données sont mises à jour dans la base, avec un message de confirmation.

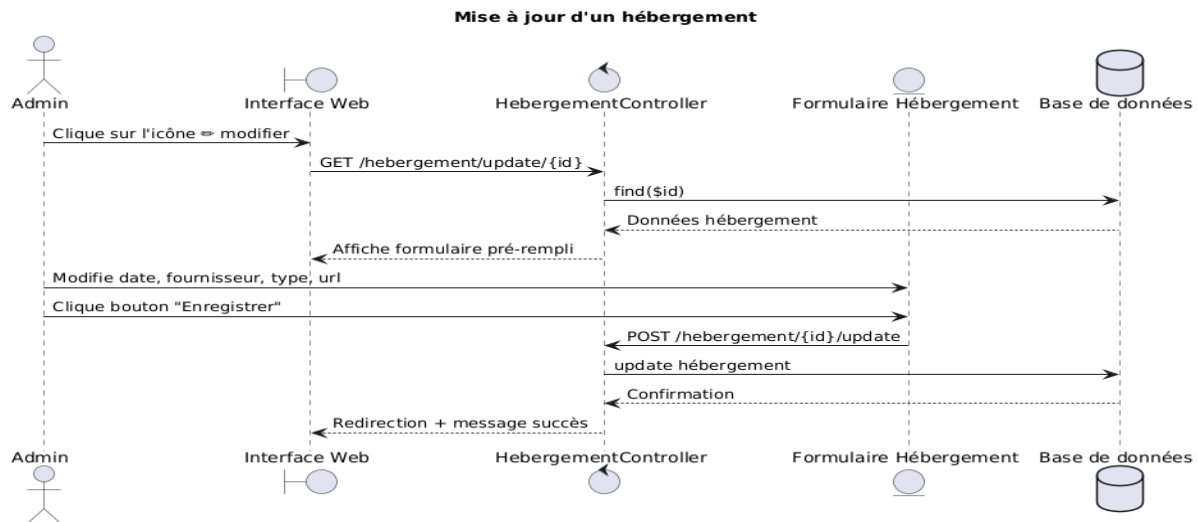


Figure 14. Diagramme de séquence modification hébergement.

(3) Supprimer hébergement :

Ce diagramme montre le processus de suppression d'un hébergement. L'admin clique sur l'icône "🗑️", un popup s'affiche avec une confirmation. Si validé, une requête DELETE est envoyée au contrôleur. Celui-ci récupère l'entité, la supprime avec Doctrine (remove() + flush()), puis redirige vers la liste actualisée.

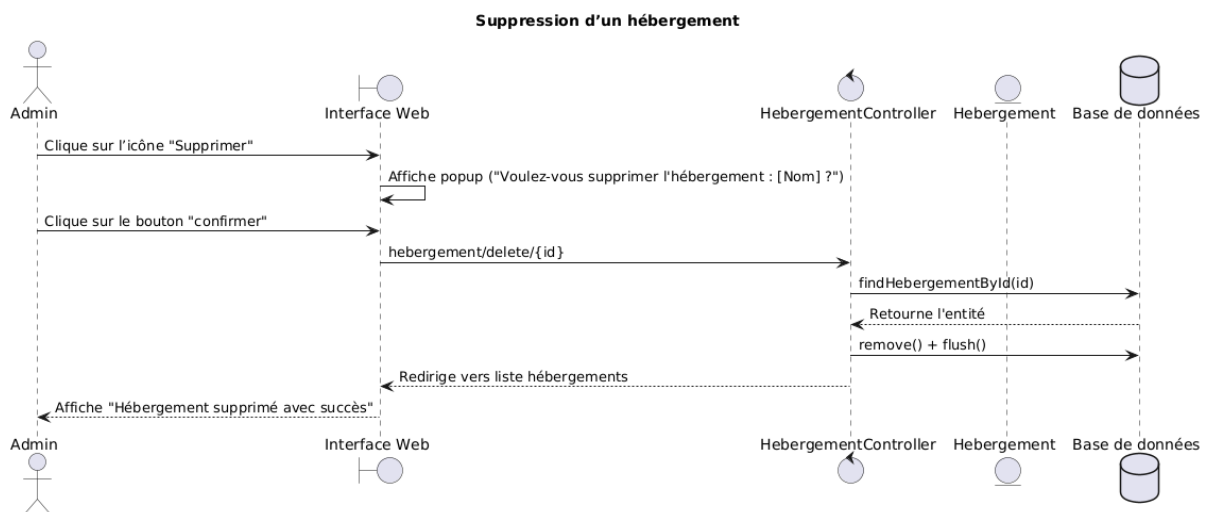


Figure 15. Diagramme de séquence suppression hébergement.

(4) Rechercher un hébergement :

La recherche d'hébergement fonctionne par filtre sur le fournisseur. Elle permet d'identifier rapidement tous les hébergements reliés à un nom de fournisseur saisi, avec affichage instantané

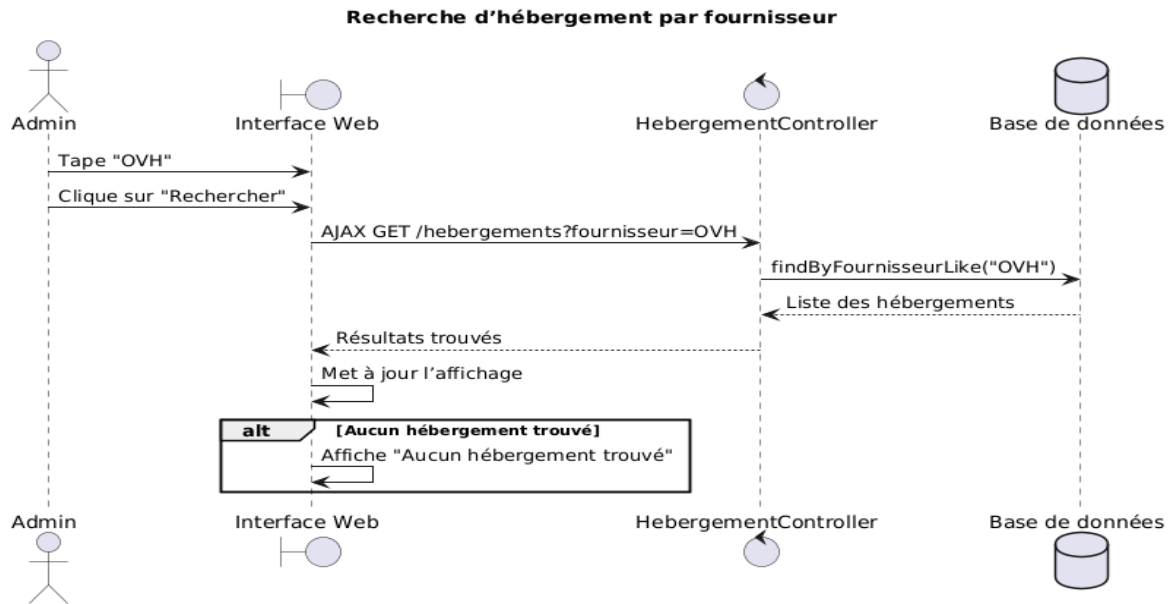


Figure 16. Diagramme de séquence rechercher hébergement.

(5) Voir plus :

Ce diagramme décrit l'interaction lorsqu'un administrateur clique sur l'icône "voir plus" à côté d'un hébergement. L'interface envoie une requête au backend pour récupérer les informations de cet hébergement, ainsi que tous les projets qui y sont hébergés. Ces données sont ensuite affichées dans un popup de détails, sans rechargement de la page.

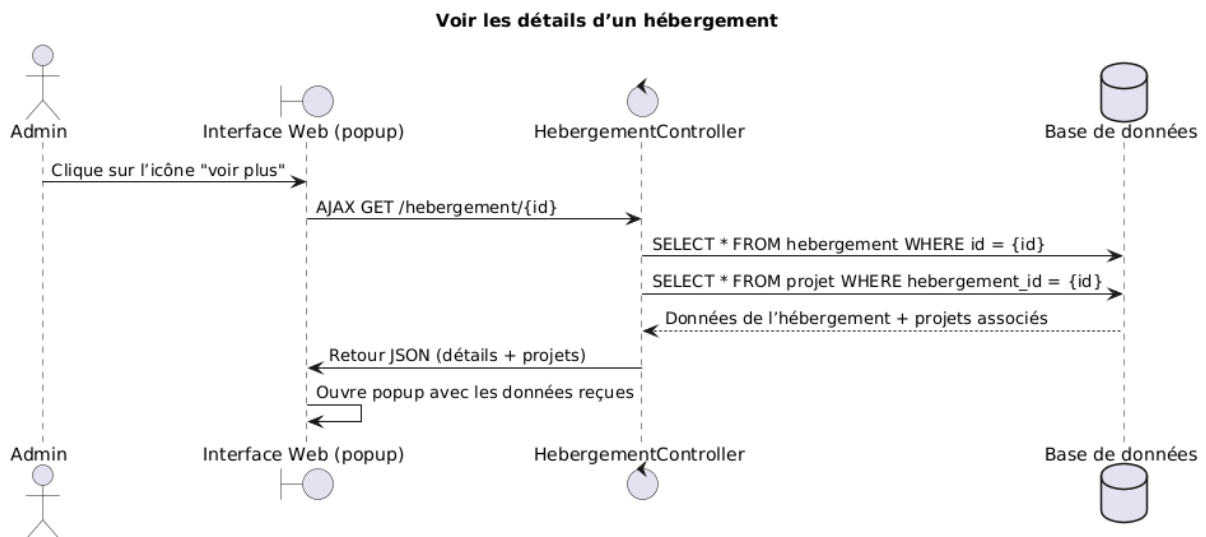


Figure 17. Diagramme de séquence plus détail hébergement.

iv. Gestion technologies :

(1) Ajout technologie :

Ce diagramme de séquence illustre le processus par lequel un administrateur ajoute une nouvelle technologie dans le système via l'interface web. L'admin clique sur le bouton "Nouvelle Technologie", un formulaire s'affiche, il saisit le nom et la version, puis clique sur

“Enregistrer”. L'application valide les données et les insère dans la base de données.

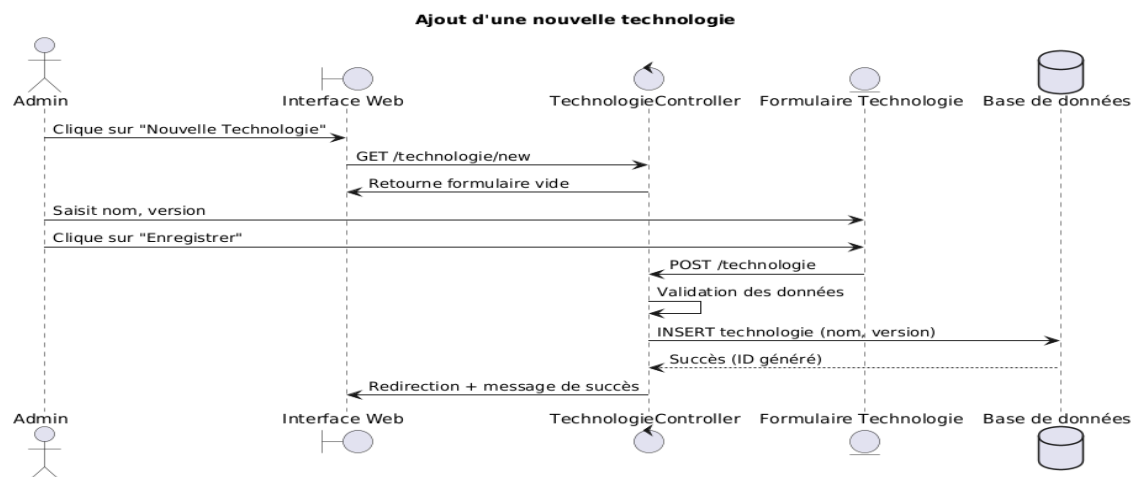


Figure 18. Diagramme de séquence Ajout technologie.

(2) Modifier technologie :

L'admin peut modifier une technologie via un formulaire pré-rempli avec son nom et sa version. Une fois les champs modifiés, la soumission déclenche une mise à jour en base de données.

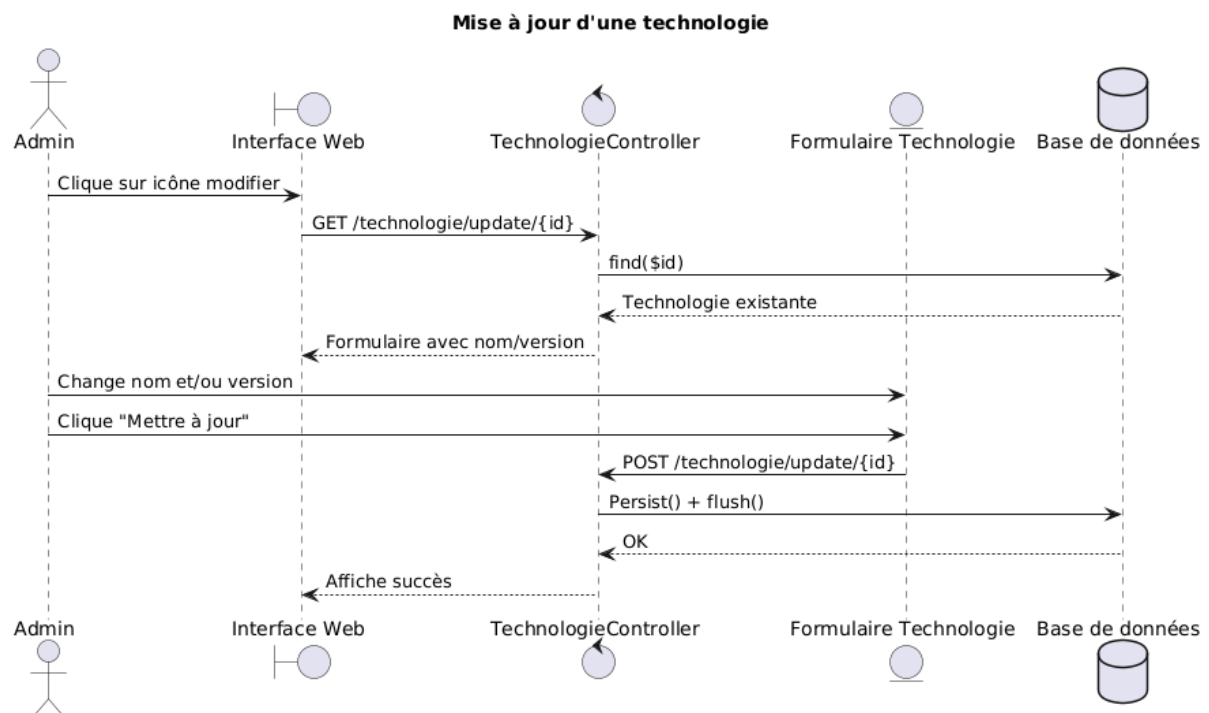


Figure 19. Diagramme de séquence modification technologie.

(3) Supprimer technologie :

L'administrateur initie la suppression d'une technologie via un clic sur une icône. Un popup de confirmation affiche le nom de la technologie ciblée. En validant, le contrôleur supprime l'entité depuis la base de données et met à jour la vue côté interface.

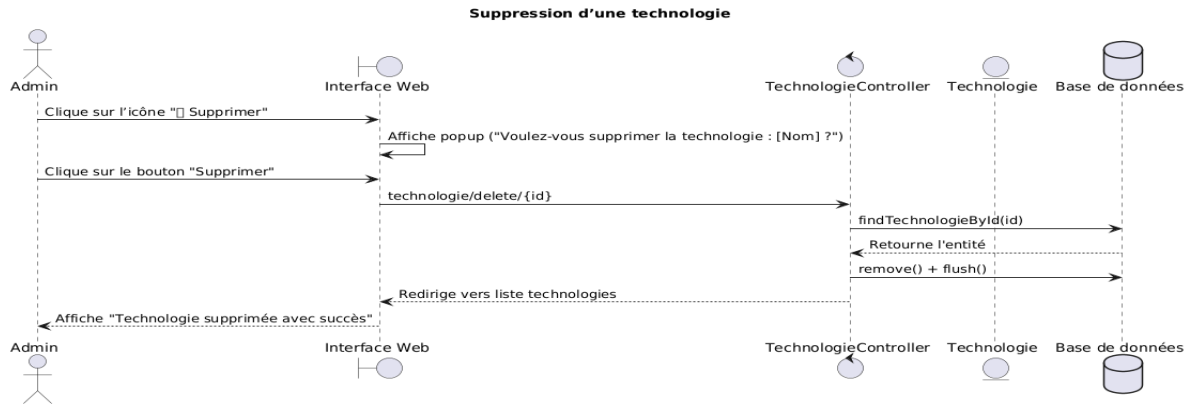


Figure 20. Diagramme de séquence suppression technologie.

(4) Rechercher technologie :

Une barre de recherche dédiée aux technologies permet de filtrer la liste affichée par nom. Elle utilise le même principe de mise à jour dynamique côté front avec requêtes AJAX, pour une meilleure ergonomie.

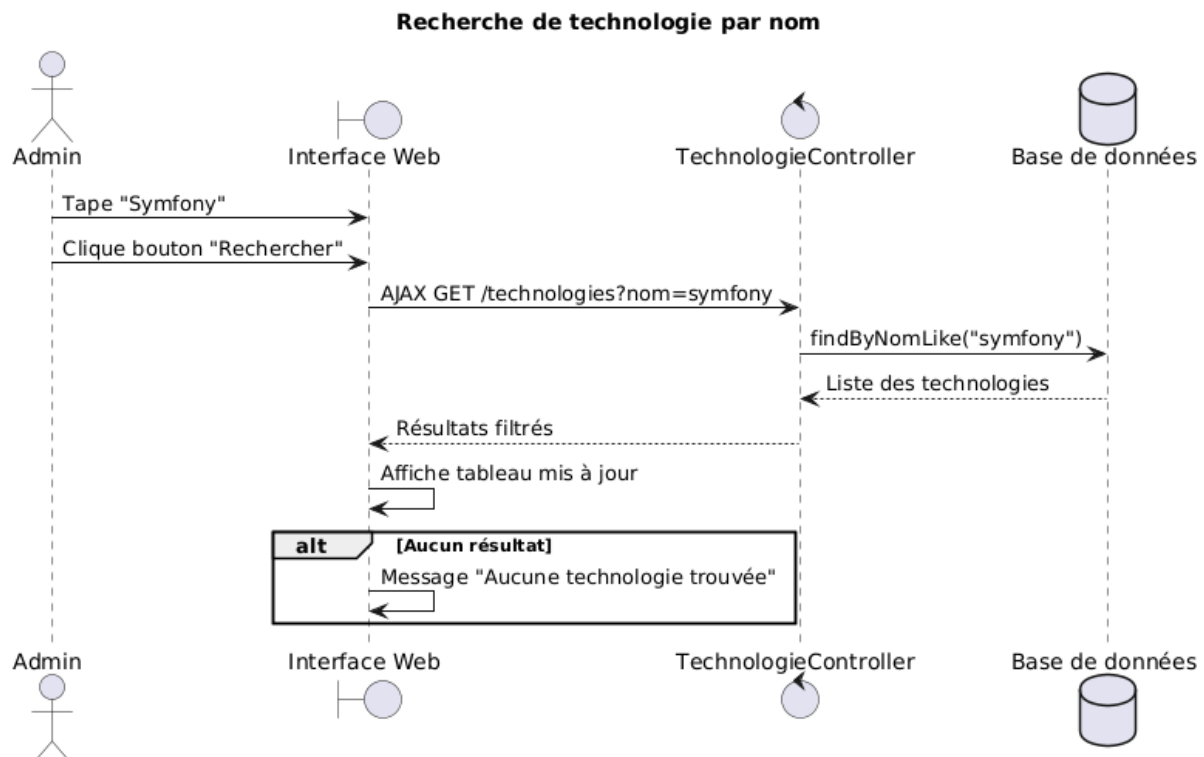


Figure 21. Diagramme de séquence recherche technologie.

(5) Voir plus :

Ce diagramme illustre l'interaction entre l'administrateur et le système lorsqu'il clique sur l'icône "voir plus" associée à une technologie. L'application web récupère les détails de la technologie sélectionnée ainsi que tous les projets qui utilisent cette technologie. Les données sont ensuite affichées dans un popup, évitant tout rechargement de page

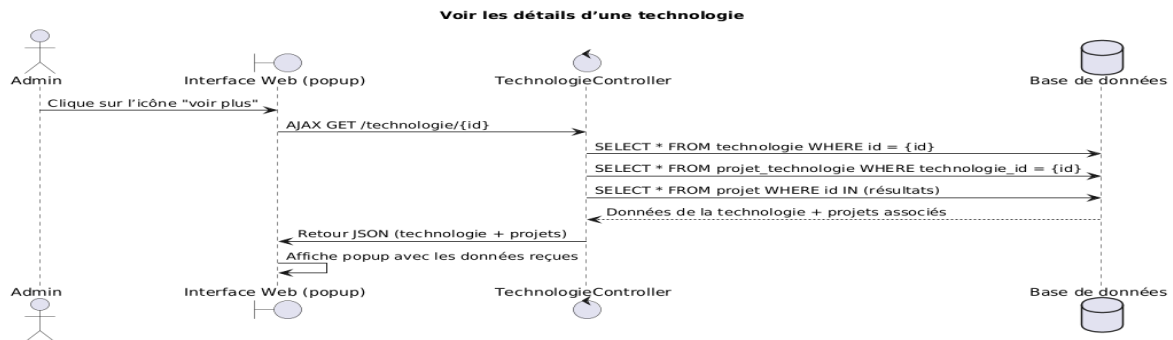


Figure 22. Diagramme de séquence plus détail technologie.

v. Gestion développeurs :

(1) Ajouter développeur :

Ce diagramme illustre comment l'administrateur ajoute un nouveau développeur dans l'application. L'admin accède au formulaire, saisit les informations du développeur, puis sélectionne un utilisateur existant (via son email), qui n'est pas encore affecté à un développeur. Une fois validé, l'application vérifie la disponibilité du User, crée l'entité Développeur, l'associe au User, et enregistre le tout en base de données.

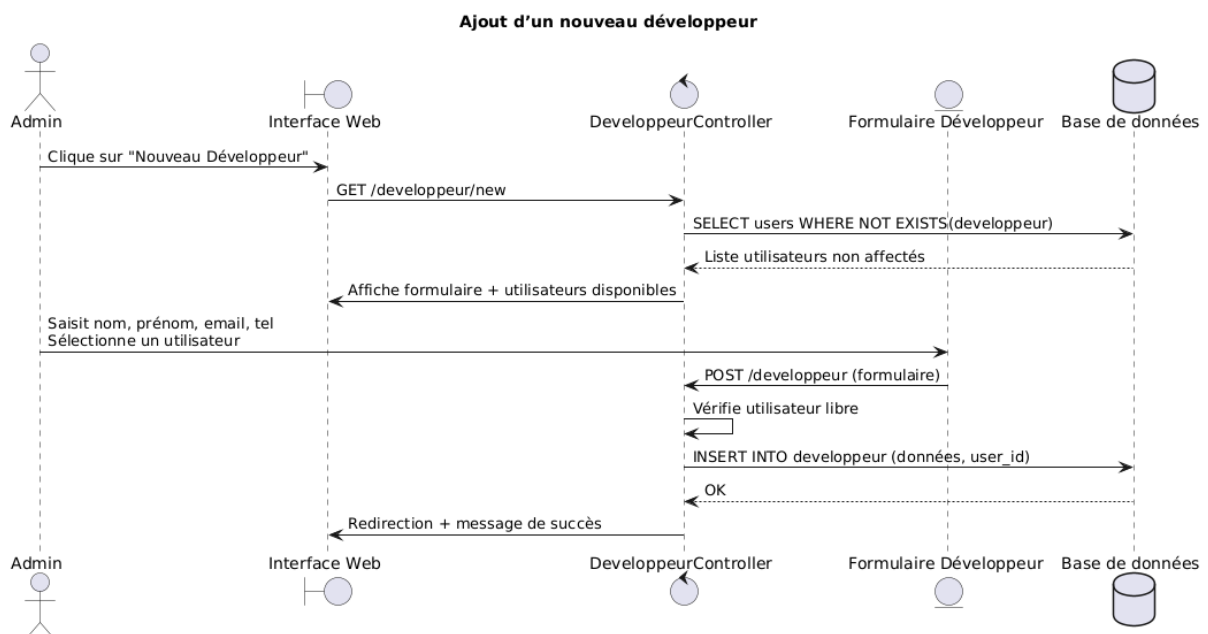


Figure 23. Diagramme de séquence Ajout développeur.

(2) Modifier développeur :

L'administrateur a la possibilité de modifier les informations d'un développeur : son nom, son email ou son numéro de téléphone. Cela s'effectue via un formulaire dynamique pré-rempli.

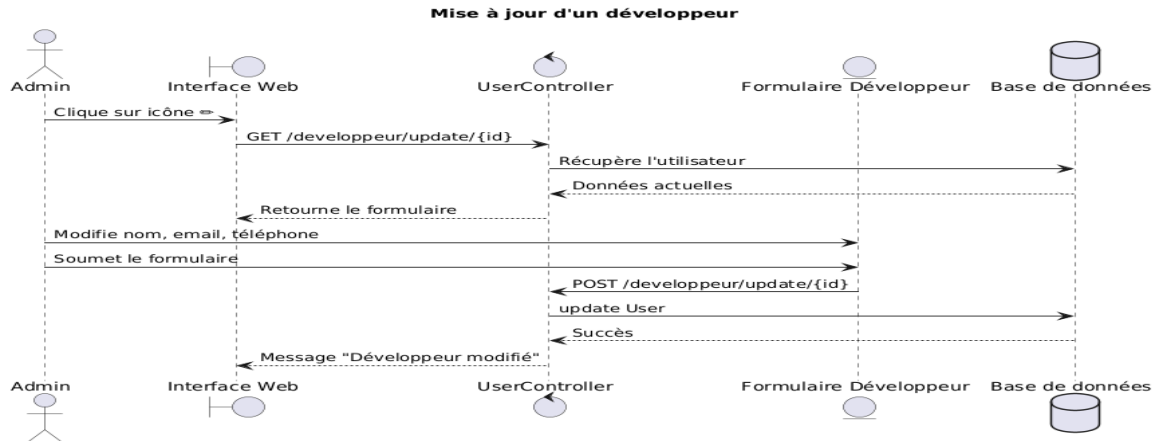


Figure 24. Diagramme de séquence modification développeur.

(3) Supprimer développeur :

Ce diagramme décrit le scénario de suppression d'un utilisateur développeur. L'admin interagit via l'interface, confirme via un popup, puis le contrôleur supprime l'entité correspondante dans la base. Un message de confirmation conclut l'opération.

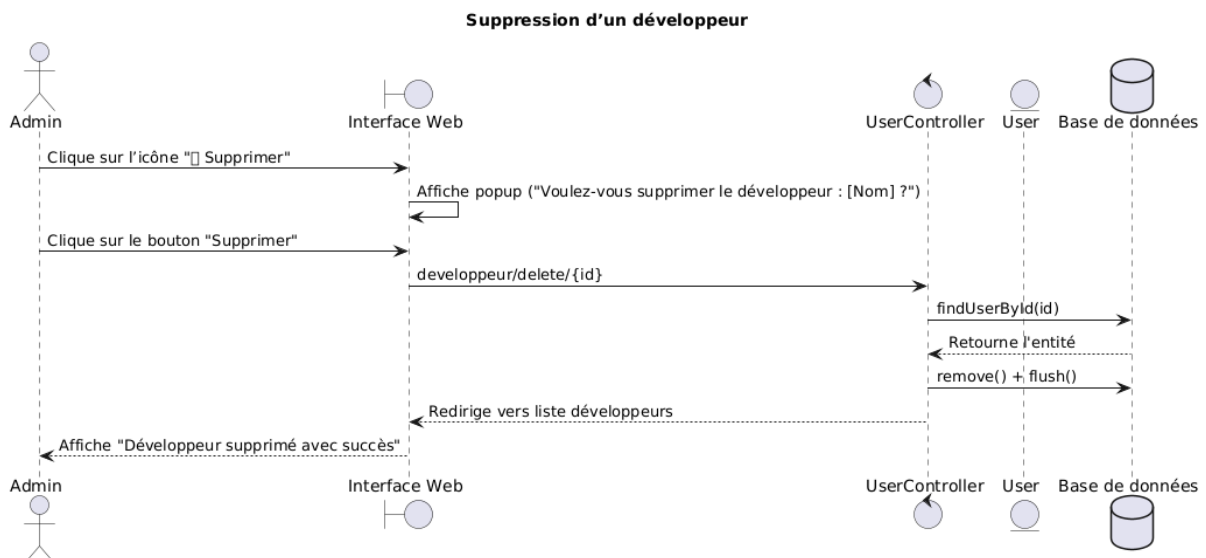


Figure 25. Diagramme de séquence suppression développeur.

(4) Rechercher développeur :

L'interface permet la recherche par nom ou prénom du développeur. La réponse est dynamique grâce à l'usage d'AJAX, et retourne les résultats en temps réel sur la même page.

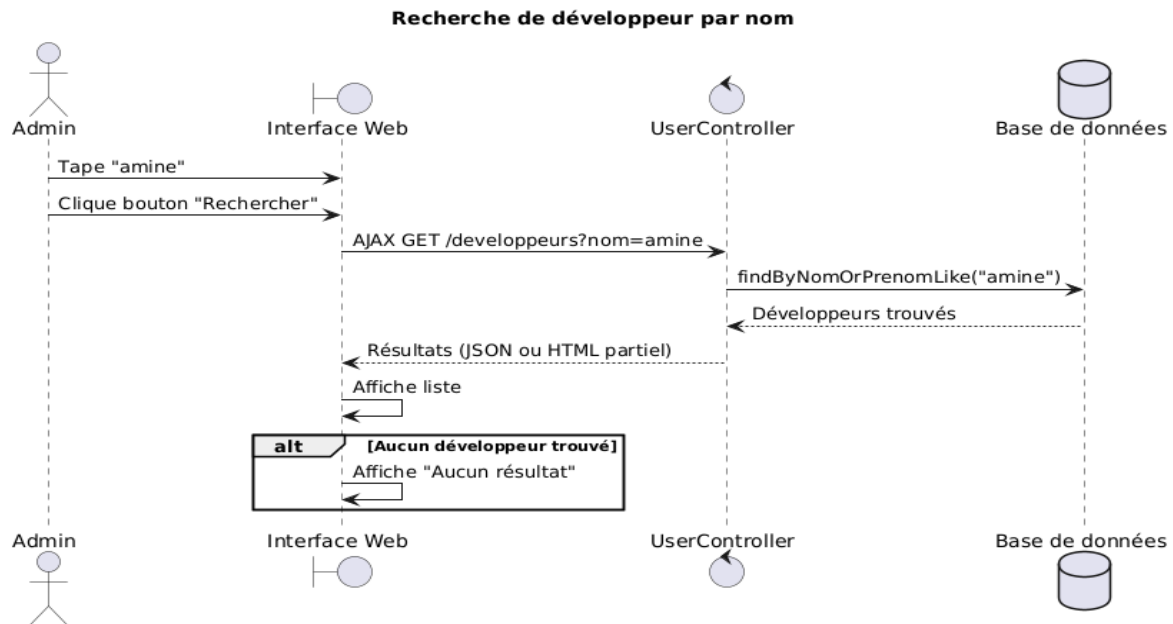


Figure 26. Diagramme de séquence recherche développeur.

(5) Voir plus :

Ce diagramme représente le processus déclenché lorsqu'un administrateur clique sur l'icône "voir plus de détails" d'un développeur dans la liste. L'interface affiche un popup contenant toutes les informations du développeur (nom, prénom, email, téléphone, etc.) ainsi que la liste des projets auxquels il est affecté. Ces projets sont récupérés dynamiquement depuis la base de données.

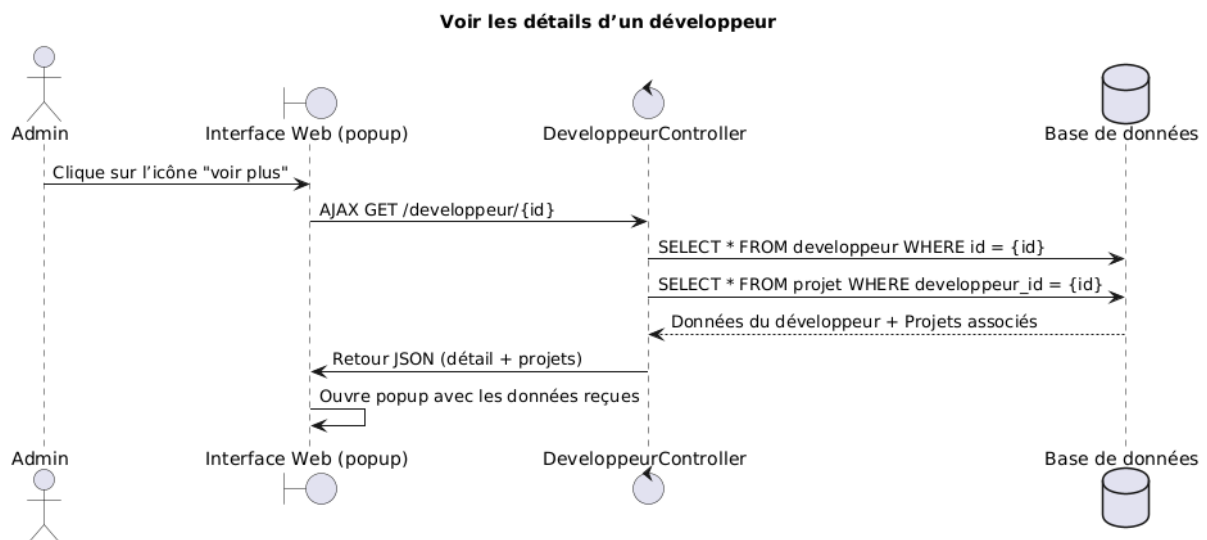


Figure 27. Diagramme de séquence recherche développeur.

vi. Recevoir Notification & Alertes :

L'application intègre un système de notification permettant à l'utilisateur d'être alerté en temps réel des événements critiques liés aux projets et aux hébergements.

Lorsqu'un projet possède une URL dont l'accès est défaillant (inaccessible ou retourne une erreur), une notification spécifique est générée pour informer l'utilisateur de ce dysfonctionnement. Cette alerte permet une réaction rapide afin de corriger le problème ou de diagnostiquer la cause.

De même, pour les hébergements, le système surveille la date d'expiration et déclenche une notification lorsqu'un hébergement approche de sa date limite, définie ici comme étant 15 jours avant l'expiration. Cette alerte préventive permet à l'utilisateur d'anticiper les renouvellements ou les actions nécessaires pour éviter toute interruption de service.

Ces notifications sont accessibles via une icône dédiée dans l'interface principale. En cliquant dessus, l'utilisateur peut consulter la liste des alertes en cours, avec des informations claires sur chaque problème (nom du projet ou de l'hébergement concerné, nature de l'alerte, échéance, etc.).

Ce système améliore la réactivité et la gestion proactive des ressources, garantissant ainsi la continuité et la qualité des services liés aux projets.

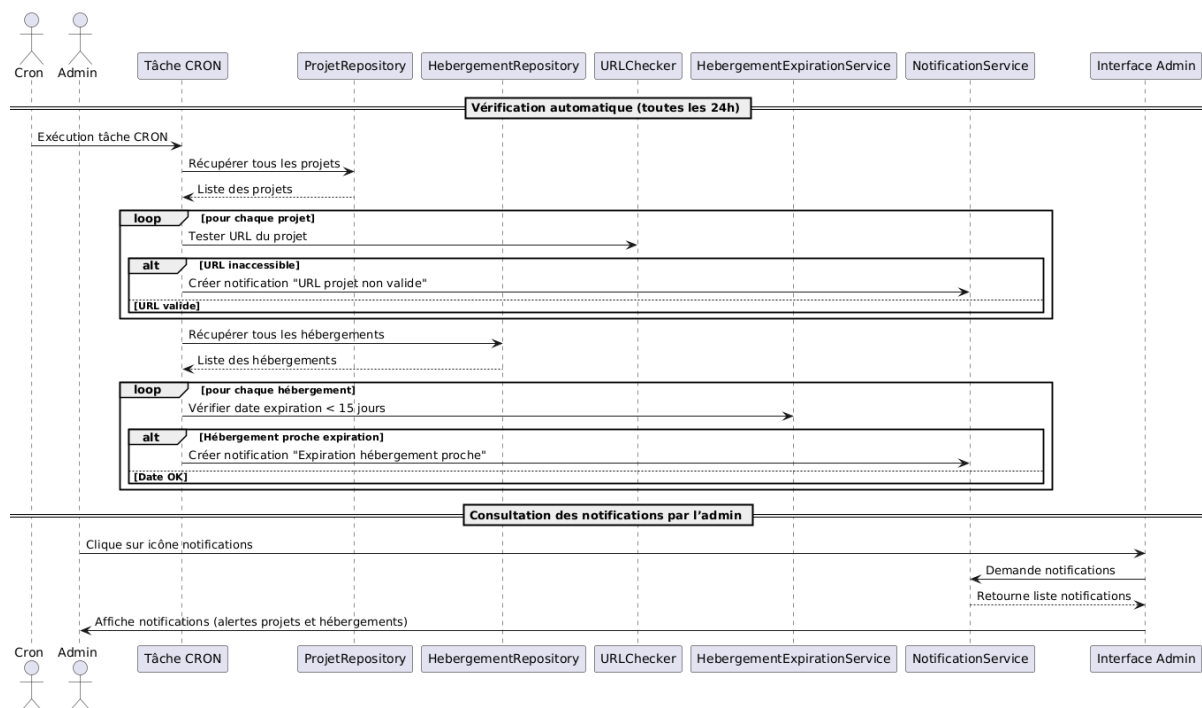


Figure 28. Diagramme de séquence notification et alerte.

c. Utilisateur :

i. Modifiée projet :

Ce diagramme illustre le scénario où un utilisateur (développeur) met à jour un projet existant. En cliquant sur l'icône « Modifier », l'interface affiche un formulaire pré-rempli avec les données du projet. L'utilisateur modifie les informations souhaitées, puis envoie le formulaire. Le contrôleur Symfony met à jour les champs de l'entité Projet et applique les

nouvelles associations. Enfin, il effectue un flush() pour sauvegarder les changements. Un message de confirmation est affiché

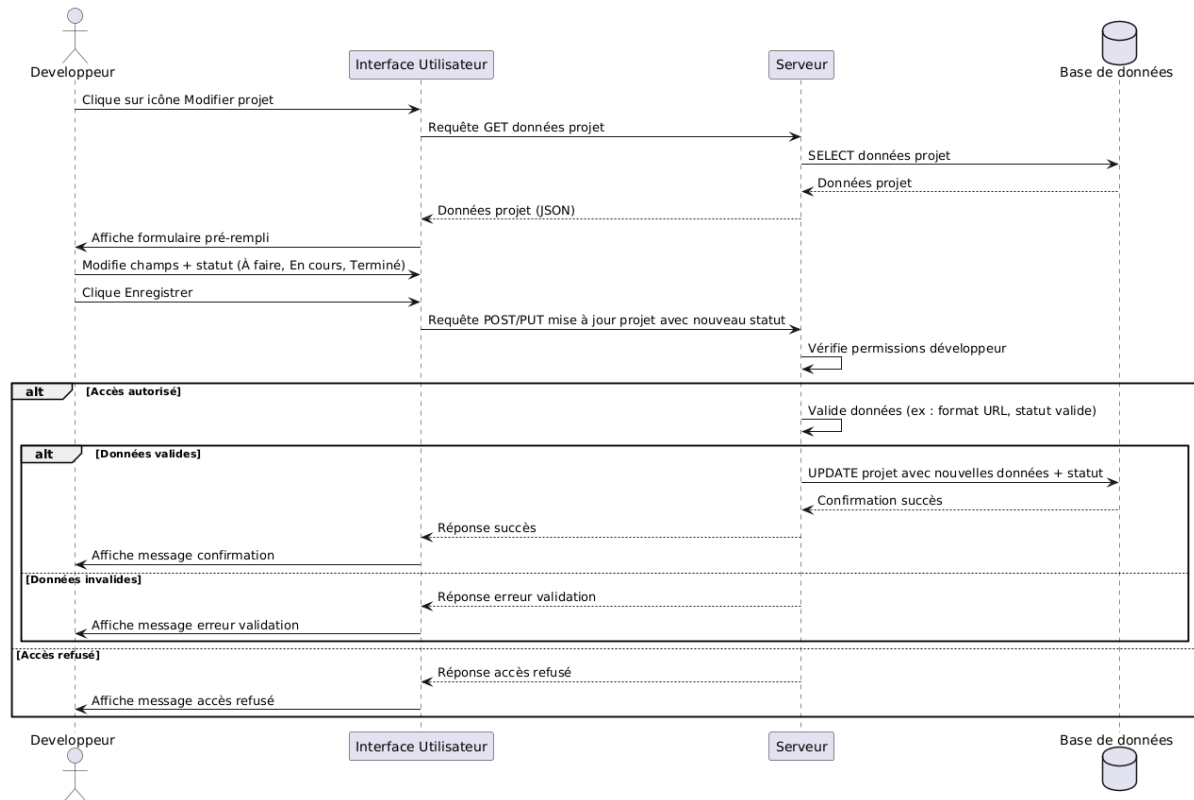


Figure 29. Diagramme de séquence modification projet user

5. Diagramme d'activités :

Le diagramme d'activités est un diagramme comportemental utilisé en UML pour représenter le flux de travail ou les enchaînements d'actions dans un système. Il décrit l'enchaînement logique des opérations réalisées par un ou plusieurs acteurs, ainsi que les conditions, décisions ou alternatives possibles

a. Inscription :

L'utilisateur remplit le formulaire d'inscription avec les informations demandées et accepte les termes et conditions. À l'envoi du formulaire, l'application vérifie la validité des données et s'assure que l'email n'est pas déjà utilisé. Si tout est conforme, le compte est créé, un email de confirmation est expédié, et l'utilisateur est redirigé vers la connexion. En cas d'erreur, un message approprié est affiché.

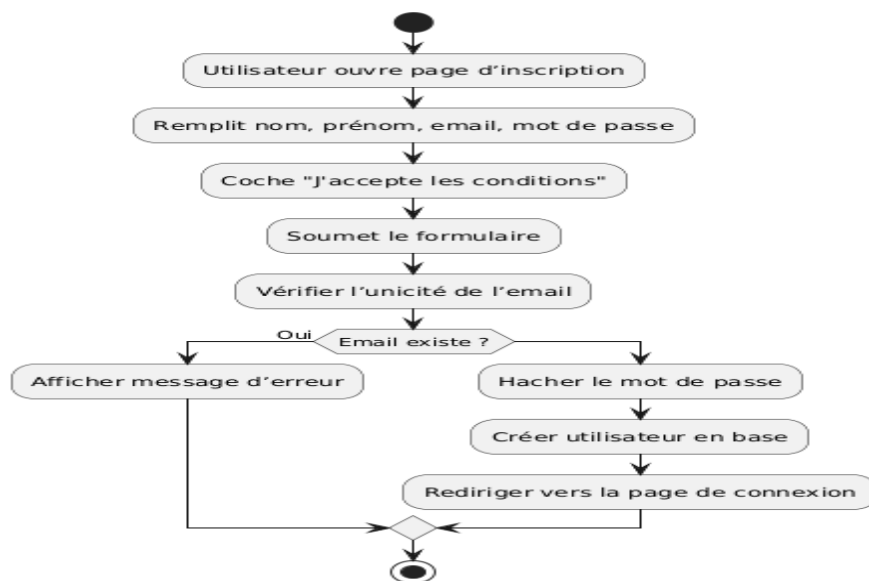


Figure 30. Diagramme d'activité inscription.

b. Connexion :

L'utilisateur saisit son email et son mot de passe dans le formulaire de connexion et peut choisir d'activer l'option "Se souvenir de moi". Après validation, l'application vérifie les données saisies. En cas de succès, la session démarre et l'utilisateur accède à son espace personnel. En cas d'échec, un message d'erreur l'informe de l'erreur de connexion. Ce processus garantit un accès sécurisé à la plateforme.

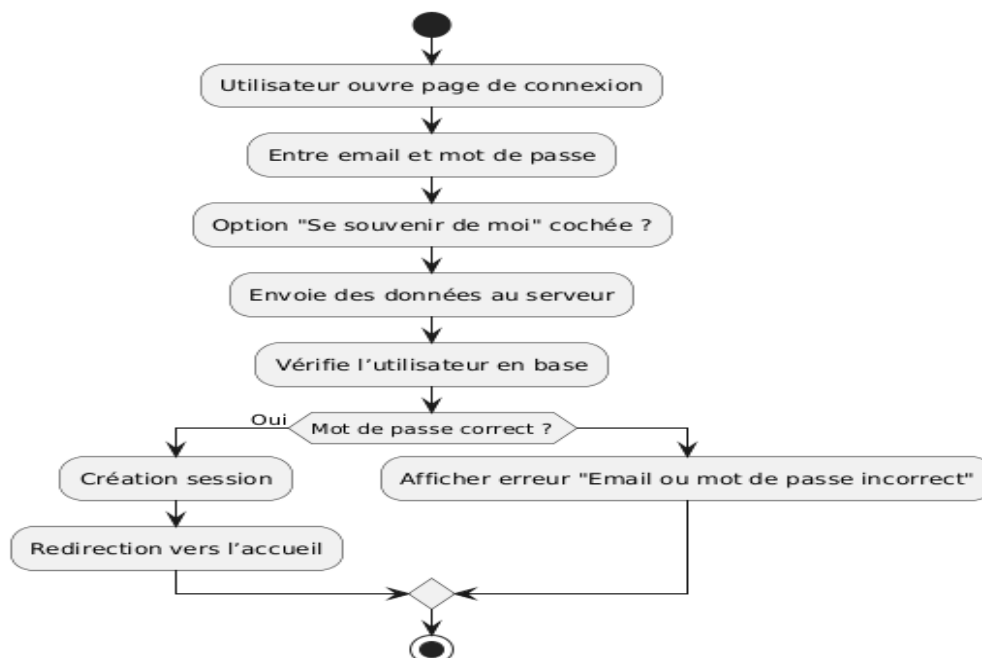


Figure 31. Diagramme d'activité connexion.

c. Gestion projets :

(1) Ajout projet :

Ce diagramme d'activité illustre les différentes étapes permettant à un administrateur d'ajouter un nouveau projet dans le système.

Le processus débute lorsque l'administrateur clique sur le bouton "**Nouveau projet**", ce qui déclenche l'affichage d'un formulaire de saisie contenant les champs nécessaires à la création du projet.

L'administrateur renseigne alors les informations telles que le nom du projet, le code source, le type de projet, la description, l'accès à l'environnement, les technologies associées, les hébergements, les développeurs affectés, ainsi que l'URL du projet.

Une fois le formulaire complété, l'administrateur soumet les données en cliquant sur le bouton "Enregistrer". Le système procède alors à une série de validations, incluant la vérification des champs obligatoires et la cohérence des données saisies.

Si une erreur est détectée, un message d'erreur est affiché, invitant l'administrateur à corriger les informations avant de pouvoir valider à nouveau.

Si toutes les validations sont réussies, le projet est enregistré dans la base de données, et une confirmation visuelle est présentée à l'administrateur pour lui indiquer que l'ajout a été effectué avec succès.

Enfin, la liste des projets est automatiquement mise à jour afin d'afficher le nouveau projet créé, ce qui permet à l'administrateur d'avoir une vue immédiate sur l'état actuel des projets gérés.

Ce diagramme met en évidence l'interaction entre l'administrateur et le système, ainsi que les contrôles nécessaires pour garantir la validité des données avant leur persistance.

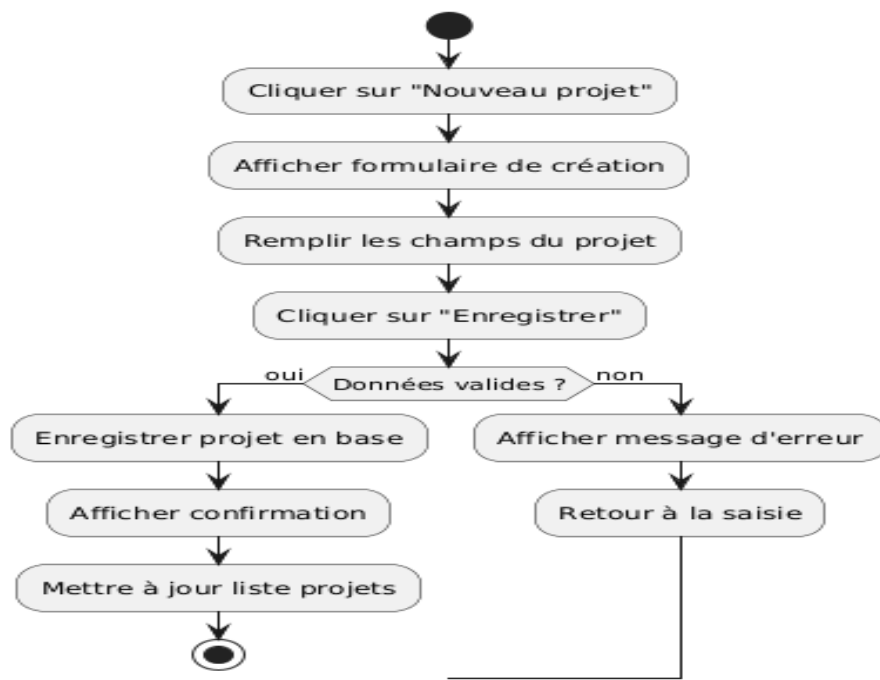


Figure 32. Diagramme d'activité Ajout projet.

(2) Modifier projet :

Ce diagramme d'activité illustre le processus complet de mise à jour des informations d'un projet dans l'application. Il commence par l'accès de l'utilisateur à la liste des projets, puis la sélection du projet à modifier via un bouton dédié.

Une fois le projet sélectionné, les données existantes sont chargées et affichées dans un formulaire pré-rempli afin de permettre à l'utilisateur d'apporter des modifications précises. Ces modifications peuvent concerner plusieurs champs du projet, notamment son nom, la description, les technologies associées, les hébergements, les développeurs affectés, ainsi que le statut du projet (par exemple : "À faire", "En cours", "Terminé").

Après modification, l'utilisateur soumet le formulaire. Le système valide alors les données saisies pour s'assurer qu'elles respectent les contraintes métier et techniques (comme la présence des champs obligatoires, formats valides, etc.).

Si les données sont valides, les modifications sont enregistrées dans la base de données, et un message de confirmation est affiché à l'utilisateur, indiquant que la mise à jour a été effectuée avec succès.

En revanche, si des erreurs sont détectées lors de la validation (champs manquants, formats incorrects...), des messages d'erreur détaillés sont affichés, et l'utilisateur est invité à corriger les informations avant de pouvoir soumettre à nouveau.

Ce flux garantit un contrôle rigoureux des modifications, assurant la cohérence des données projet tout en offrant une interface utilisateur claire et ergonomique pour la gestion des projets.

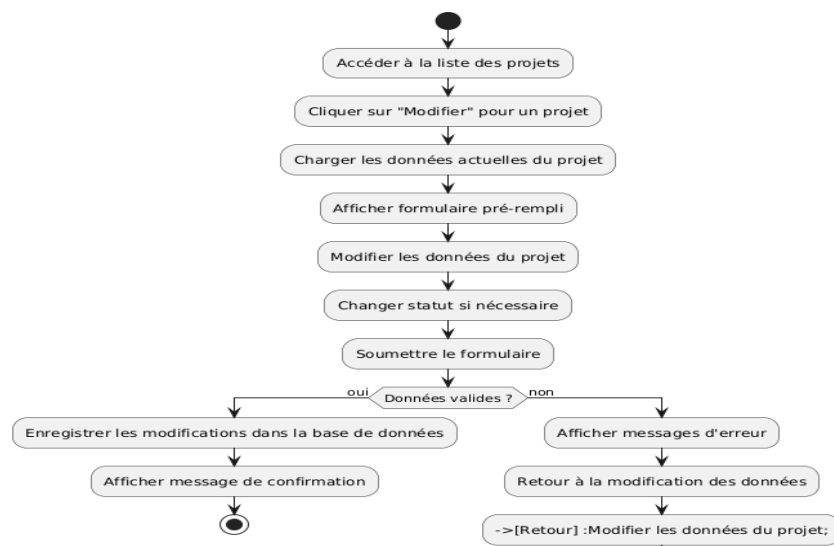


Figure 33. Diagramme d'activité modification projet.

(3) Supprimer projet :

Ce diagramme d'activité décrit le processus suivi lorsqu'un utilisateur souhaite supprimer un projet depuis l'application.

Le processus débute lorsque l'utilisateur clique sur l'icône ou le bouton de suppression associé à un projet spécifique dans la liste des projets.

Une fenêtre pop-up de confirmation s'ouvre alors, affichant un message demandant à l'utilisateur de confirmer la suppression du projet, en précisant notamment le nom du projet concerné. Cette étape est essentielle pour éviter les suppressions accidentelles.

L'utilisateur dispose de deux options :

- **Confirmer la suppression** : si l'utilisateur valide la suppression en cliquant sur le bouton de confirmation, le système procède alors à la suppression du projet dans la base de données. Cette opération peut inclure la gestion des dépendances ou relations liées au projet (par exemple, dissocier les développeurs, hébergements, technologies associées).
- **Annuler la suppression** : si l'utilisateur annule, le pop-up se ferme simplement sans aucune modification.

Après une suppression réussie, un message de confirmation est affiché à l'utilisateur pour l'informer que le projet a bien été supprimé.

Ce processus assure une gestion sécurisée et contrôlée de la suppression des projets, réduisant les risques d'erreurs tout en offrant une expérience utilisateur fluide

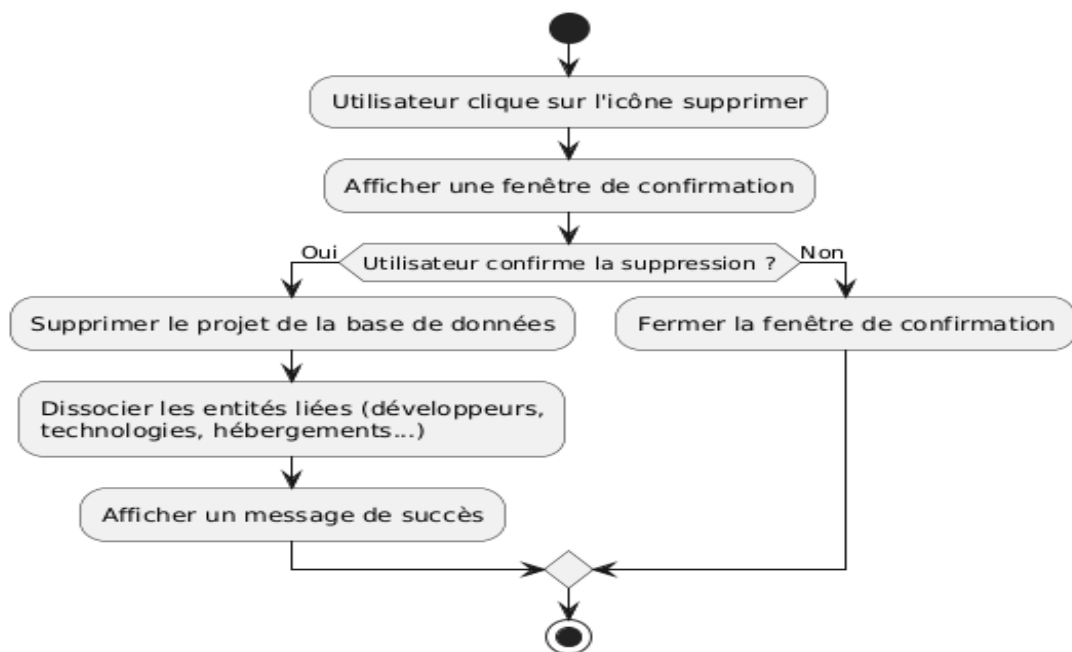


Figure 33. Diagramme d'activité suppression projet.

(4) Rechercher un projet :

La recherche permet à l'utilisateur de filtrer rapidement les projets, hébergements, développeurs ou technologies à partir d'un mot-clé saisi. Une fois la recherche lancée, une requête est envoyée au serveur pour interroger la base de données. Si des correspondances sont trouvées, elles s'affichent dynamiquement sur la même page. En cas d'erreur ou de champ vide, un message approprié est affiché pour guider l'utilisateur.

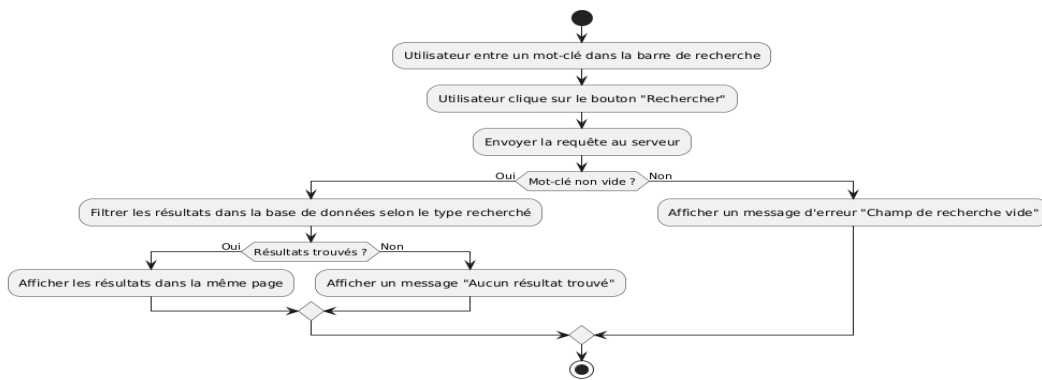


Figure 34. Diagramme d'activité rechercher projet.

d. Gestion technologies :

(1) Ajout technologie :

L'administrateur ajoute une nouvelle technologie en saisissant son nom et sa version via un formulaire. Une vérification de la validité des données est effectuée avant enregistrement dans la base. En cas de succès, un message s'affiche.

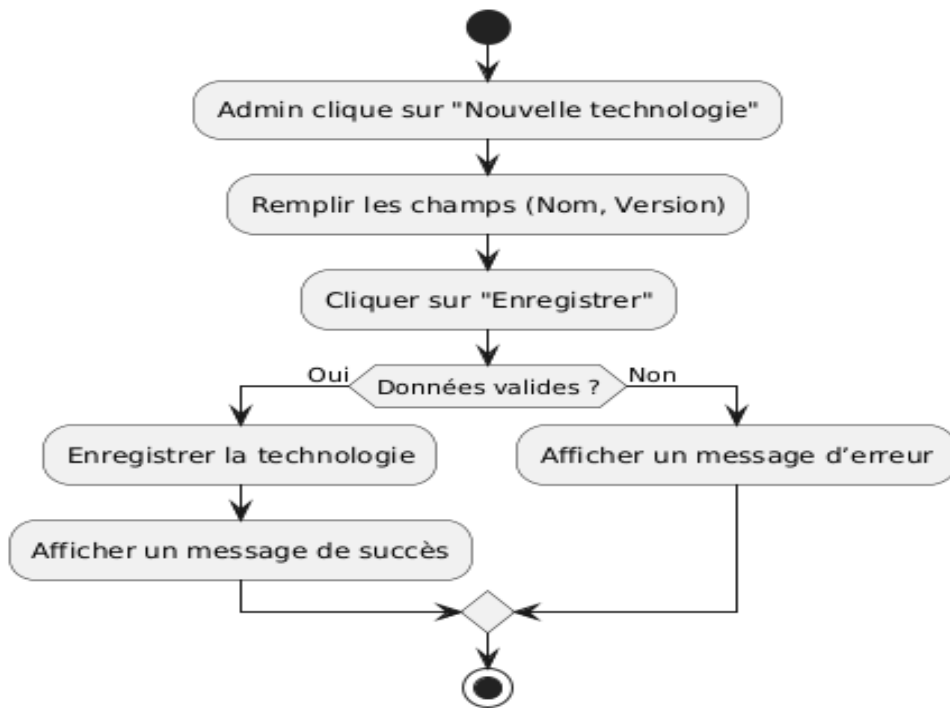


Figure 35. Diagramme d'activité ajout technologie.

(2) Modifier technologie :

L'administrateur peut modifier une technologie déjà existante. Il accède au formulaire pré-rempli, effectue les changements nécessaires, puis valide l'opération. Une validation est réalisée avant la mise à jour effective.

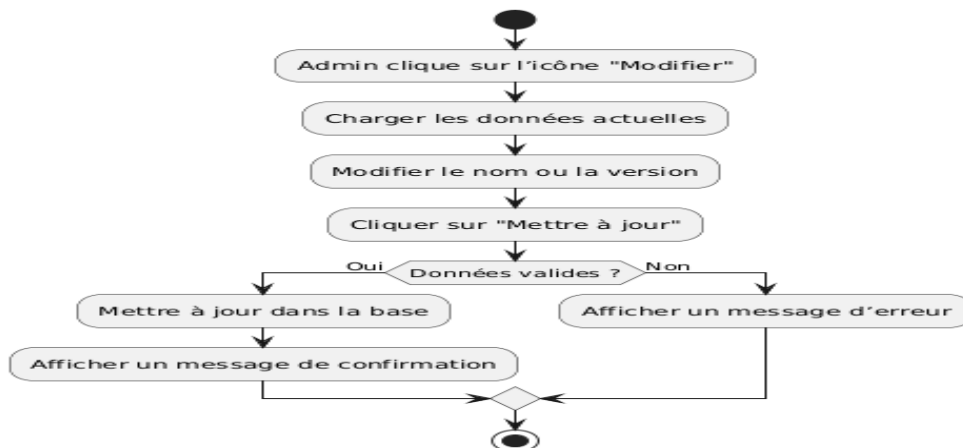


Figure 36. Diagramme d'activité modification technologie.

(3) Supprimer technologie :

Lorsqu'un administrateur clique sur l'icône de suppression, un message de confirmation s'affiche avec le nom de la technologie. Une fois validée, la suppression est effectuée et confirmée par un message à l'utilisateur.

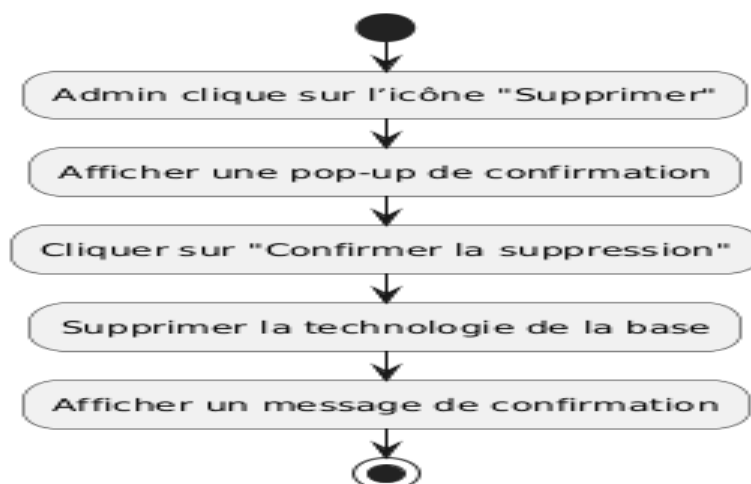


Figure 37. Diagramme d'activité suppression technologie.

(4) Rechercher une technologie :

L'administrateur recherche une technologie à partir de son nom. Un filtre dynamique permet d'afficher les résultats correspondants ou un message d'erreur si aucun résultat n'est trouvé.

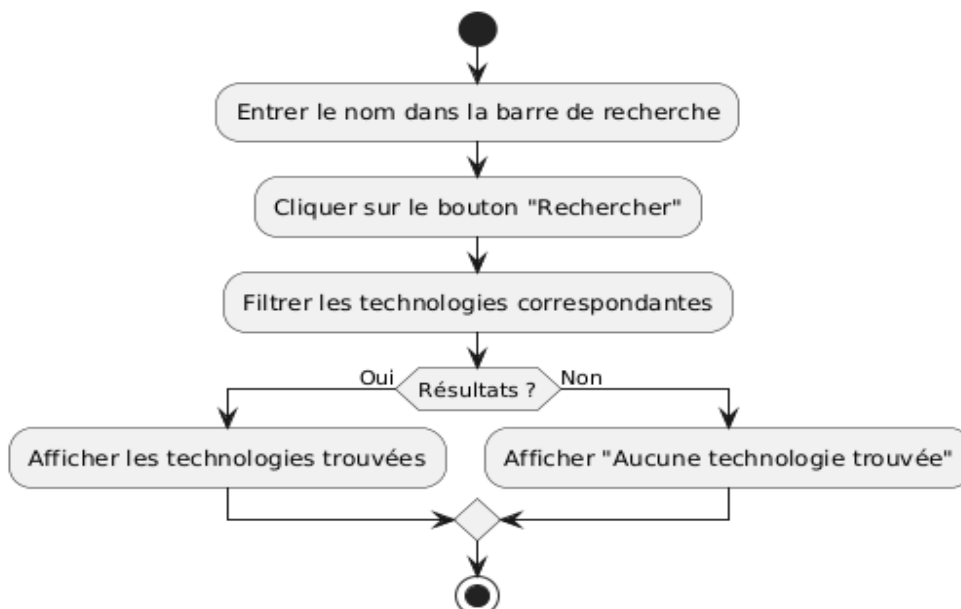
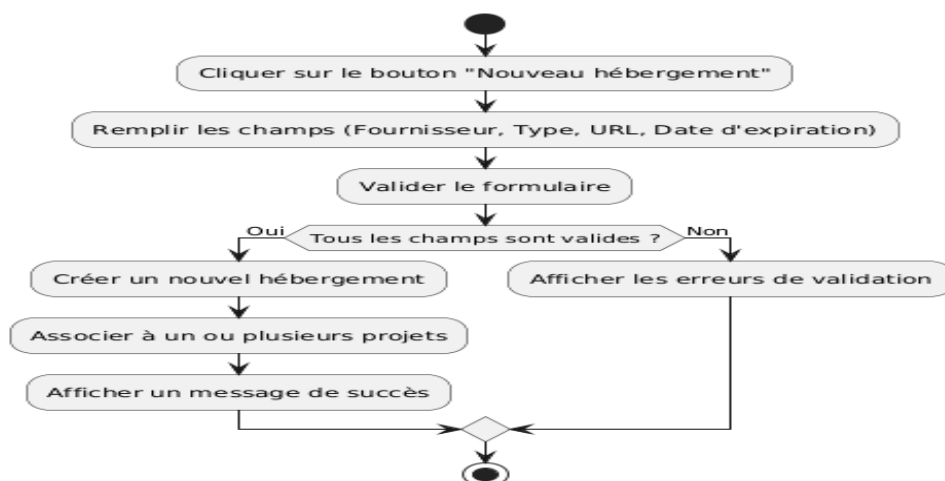


Figure 38. Diagramme d'activité recherche technologie.

e. Gestion hébergements :

(1) Ajout hébergement :

L'administrateur peut ajouter un hébergement via un formulaire contenant les champs nécessaires : fournisseur, type, URL et date d'expiration. Si les données saisies sont valides, l'hébergement est enregistré en base de données et peut être associé à un ou plusieurs projets. En cas d'erreur, un message d'alerte s'affiche.



F

Figure 39. Diagramme d'activité Ajout hébergement.

(2) Modifier un hébergement :

L'admin peut modifier les informations d'un hébergement existant en cliquant sur l'icône d'édition. Le formulaire est pré-rempli avec les données actuelles. Une fois modifié et validé, les informations sont mises à jour et un message de confirmation s'affiche.

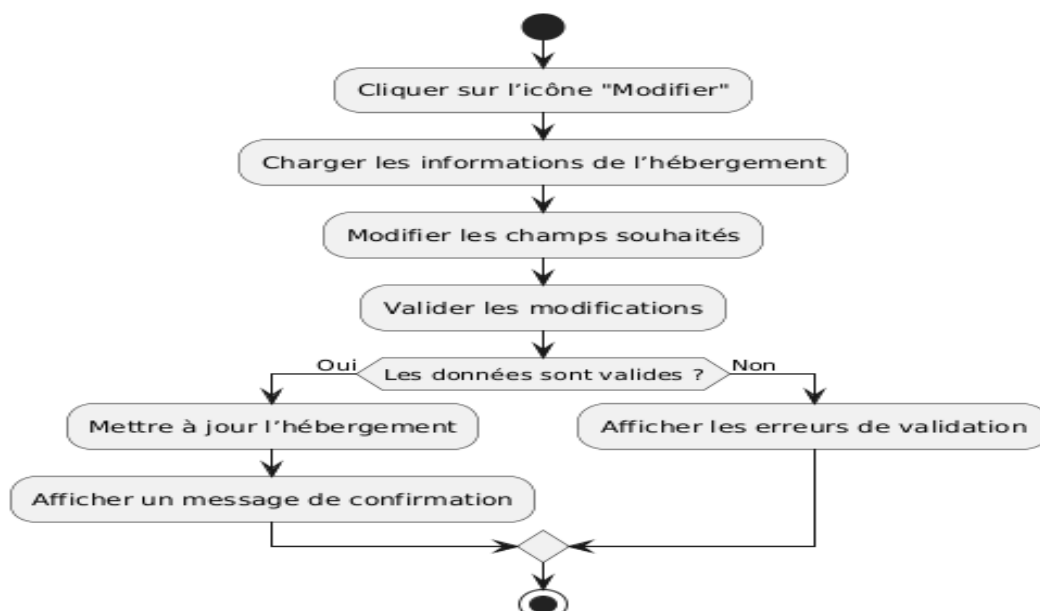


Figure 40. Diagramme d'activité modification hébergement

(3) Supprimer un hébergement :

Lorsqu'un administrateur souhaite supprimer un hébergement, une fenêtre modale s'affiche pour confirmer l'action. Si la confirmation est validée, l'hébergement est retiré de la base de données. Sinon, l'action est annulée.

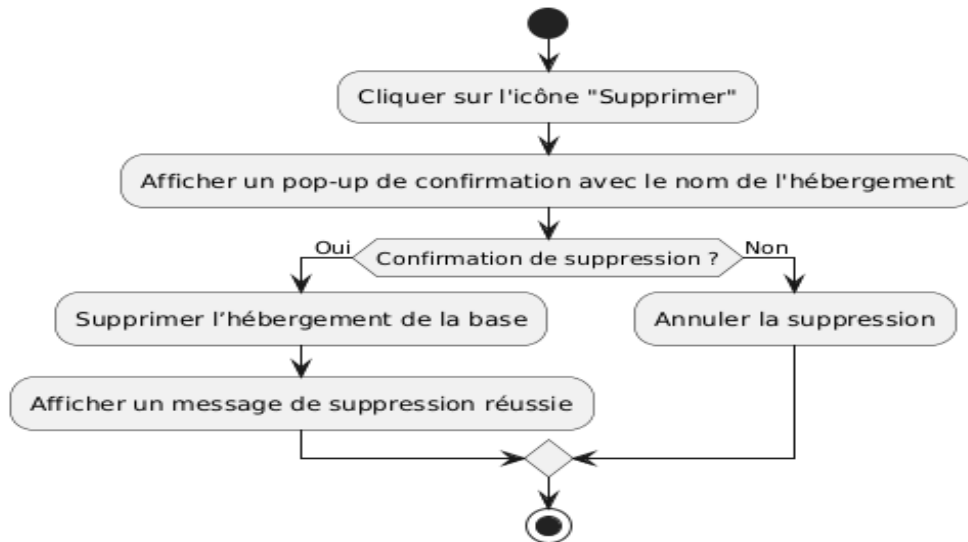


Figure 41. Diagramme d'activité suppression hébergement.

(4) Rechercher un hébergement :

L'administrateur peut rechercher un hébergement en saisissant le nom de son fournisseur dans une barre de recherche. Lorsque la recherche est lancée, le système filtre dynamiquement les hébergements correspondant au critère. Si aucun hébergement ne correspond, un message d'erreur est affiché à l'écran. Cette recherche est en temps réel et améliore l'accessibilité aux données.



Figure 42. Diagramme d'activité recherche hébergement.

f. Gestion développeurs :

(1) Ajout développeur :

Lors de l'ajout d'un développeur, l'administrateur entre les informations personnelles, puis associe un utilisateur existant non encore affecté. Une validation garantit l'unicité avant enregistrement.

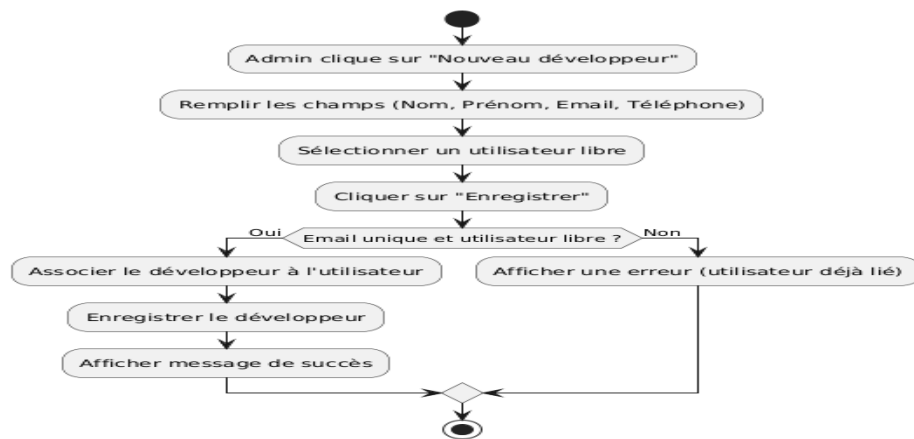


Figure 43. Diagramme d'activité ajout développeur.

(2) Modifier développeur :

L'administrateur peut mettre à jour les informations d'un développeur existant. Les modifications sont validées avant d'être enregistrées en base.



Figure 44. Diagramme d'activité modification développeur.

(3) Supprimer développeur :

Une confirmation est exigée avant la suppression d'un développeur. Si confirmée, les données du développeur sont supprimées de la base et l'utilisateur est averti.



Figure 45. Diagramme d'activité suppression développeur.

(4) Rechercher un développeur :

Grâce à un champ de recherche, l'administrateur peut rapidement retrouver un développeur en saisissant partiellement son nom. Les résultats sont affichés dynamiquement.

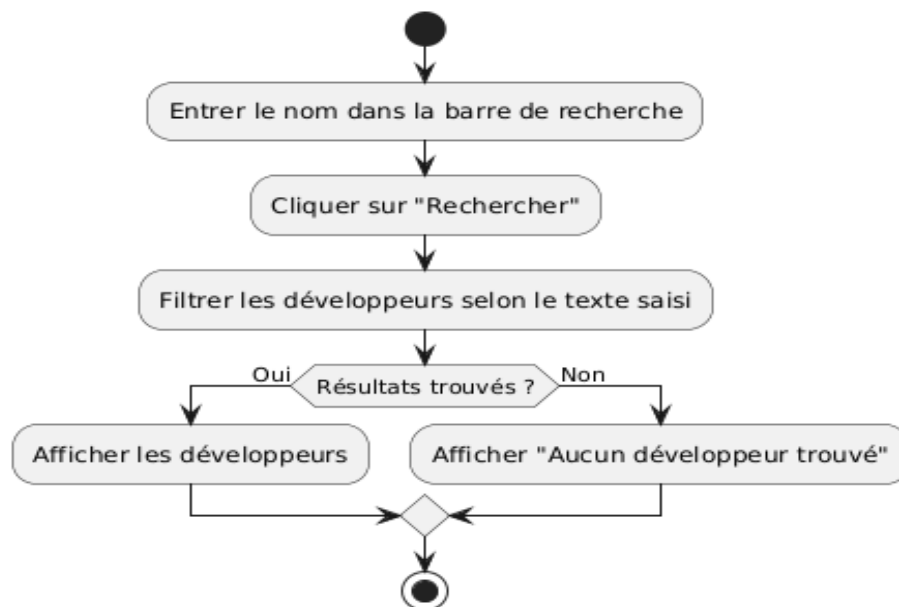


Figure 46. Diagramme d'activité recherche développeur.

g. Alerte et Notifications :

Ce diagramme d'activité illustre le processus automatisé de contrôle des URLs des projets ainsi que la surveillance des dates d'expiration des hébergements, et la gestion des notifications associées pour l'administrateur.

La procédure débute par l'exécution d'une tâche planifiée (CRON) qui s'exécute toutes les 24 heures. Cette tâche récupère la liste complète des projets, puis effectue un test d'accessibilité sur l'URL associée à chaque projet. Si une URL est inaccessible, une notification d'alerte est automatiquement créée pour signaler ce dysfonctionnement.

Ensuite, la tâche procède à la récupération de tous les hébergements. Pour chaque hébergement, elle vérifie si la date d'expiration est inférieure à 15 jours. Dans ce cas, une notification est également générée pour avertir l'administrateur du renouvellement imminent.

À l'issue de ces contrôles, la tâche CRON se termine. Côté interface utilisateur, l'administrateur dispose d'une icône dédiée aux notifications. En cliquant sur cette icône, il peut consulter la liste des alertes générées, ce qui lui permet de prendre connaissance rapidement des problèmes liés aux projets ou aux hébergements.

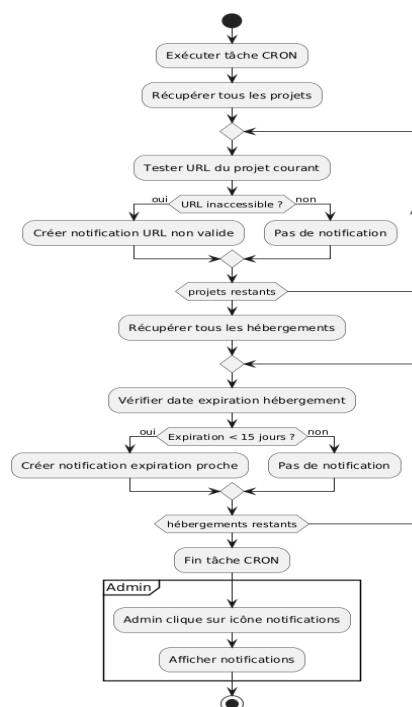


Figure 47. Diagramme d'activité alerte et notification

6. Diagramme de classe :

Le diagramme de classes est un des éléments fondamentaux du langage UML. Il permet de modéliser la structure statique d'un système, en représentant les classes, leurs attributs, leurs méthodes (ou opérations), ainsi que les relations qui les lient.

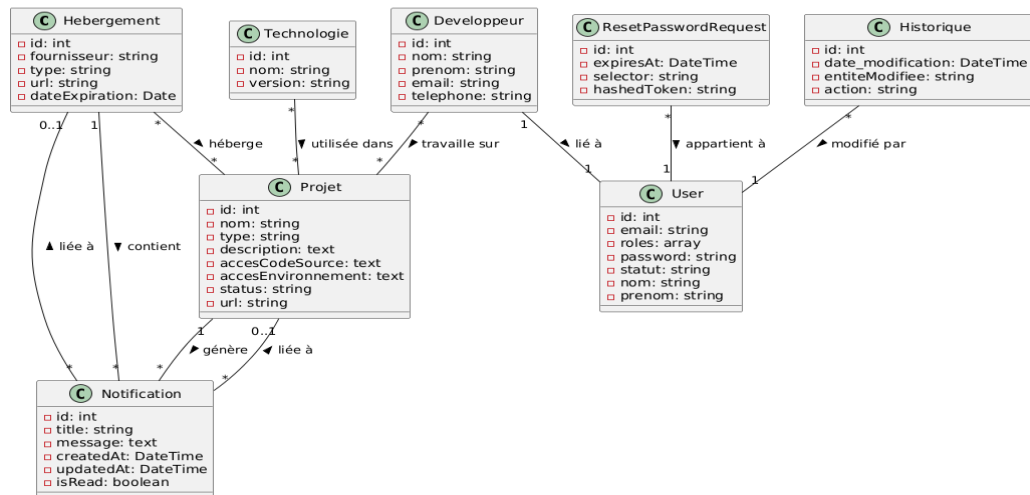


Figure 48. Diagramme de classe.

II. Besoins non fonctionnels /techniques

1. La performance

La performance mesure la rapidité d'exécution du système, notamment le temps de réponse, la vitesse de chargement, la capacité à traiter un grand volume de données ou à répondre à plusieurs utilisateurs simultanément.

Dans notre cas, l'application **Central Projet** bénéficie d'une bonne performance, aussi bien du côté de l'administrateur que de l'utilisateur. En particulier, les opérations sur le tableau de bord, telles que la recherche d'un projet, d'un développeur ou d'un hébergement, s'effectuent de manière fluide et rapide.

Cela est rendu possible grâce à plusieurs facteurs :

- L'utilisation du framework Symfony, qui intègre nativement des mécanismes de cache HTTP et d'optimisation des requêtes.
- Une base de données bien structurée, avec des index placés sur les colonnes les plus sollicitées (ex. : nom du projet, fournisseur, identifiant développeur).
- L'optimisation des requêtes Doctrine, permettant de charger efficacement les relations (projets ↔ technologies, projets ↔ hébergements, etc.) en évitant les problèmes de surcharge (n+1 queries).
- La mise en production dans un environnement optimisé (APP_ENV=prod), avec désactivation du mode debug et compilation du cache.

Ainsi, l'utilisateur bénéficie d'une expérience rapide et réactive, même en cas de forte charge de données, ce qui contribue directement à l'efficacité globale de l'application.

2. La sécurité

La sécurité garantit que le système protège les données sensibles contre les accès non autorisés, les attaques (ex : injection SQL, CSRF, XSS) et les fuites d'informations.

Dans le cas de **Central Projet**, plusieurs mesures de sécurité ont été mises en place :

- L'authentification repose sur l'utilisation d'une adresse e-mail et d'un mot de passe sécurisé, stocké sous forme de hash .
- La gestion des accès est basée sur les rôles (ex : administrateur, utilisateur), avec des restrictions d'accès appliquées via le système de Security Voter de Symfony.
- Le pare-feu Symfony (firewall) est configuré pour filtrer les routes sensibles.
- Des jetons CSRF sont utilisés dans les formulaires pour prévenir les attaques par falsification de requête.
- Les entrées utilisateurs sont filtrées et échappées afin d'éviter toute injection de code malveillant.

Ces mesures garantissent que les données des projets, des utilisateurs et des hébergements sont protégées en permanence, assurant la fiabilité et la confidentialité du système.

3. La portabilité

La portabilité désigne la capacité de l'application à fonctionner correctement sur différents environnements (systèmes d'exploitation, navigateurs, serveurs) sans nécessiter de modifications importantes.

L'application **Central Projet** a été conçue de manière à être facilement déployable sur différents environnements :

- Elle fonctionne aussi bien sur des serveurs Windows (avec XAMPP) que Linux (Apache2/Nginx).
- Elle peut être déployée localement, sur un serveur privé ou dans un environnement cloud (comme AWS ou OVH).
- L'utilisation de Docker est également possible pour automatiser le déploiement sur n'importe quel système, en assurant une cohérence de l'environnement de développement et de production.
- L'interface web est responsive et compatible avec les navigateurs modernes (Chrome, Firefox, Edge...).

Ainsi, CentralProjet bénéficie d'une bonne portabilité, ce qui facilite son installation, sa distribution et son utilisation dans différents contextes d'infrastructure.

4. La maintenance

La maintenance d'un logiciel correspond à sa capacité à être modifié, corrigé ou étendu de manière simple, rapide et sans conséquences imprévues sur les fonctionnalités existantes.

Central Projet est conçu pour être facilement maintenable, grâce à plusieurs pratiques de développement :

- Une architecture claire en MVC, respectée grâce au framework Symfony.
- Un code modulaire, structuré en services, contrôleurs et entités bien séparés.
- Une documentation technique interne (annotations, noms explicites, commentaires).
- Une utilisation cohérente des composants Symfony, ce qui facilite l'ajout de nouvelles fonctionnalités (par exemple : système de notifications avancées, ajout de filtres de recherche...).
- La possibilité de mettre en place des tests automatisés (unitaires ou fonctionnels) avec PHPUnit.

Ces aspects permettent aux développeurs actuels ou futurs d'assurer la maintenance évolutive et corrective de l'application, tout en réduisant les risques d'erreurs



Chapitre3:

Etude technique

Chapitre 3 : Etude technique

I. Architecture logicielle du système

Le projet **CentralProjet** est basé sur une **architecture logicielle en trois tiers (3-tiers)**, bien supportée par le framework **Symfony**. Cette architecture favorise la **séparation des responsabilités** entre la présentation, la logique métier et l'accès aux données, ce qui améliore la lisibilité du code, la maintenance et l'évolutivité du système.

1. Couche présentation (Frontend)

Cette couche correspond à tout ce que l'utilisateur voit et utilise dans le navigateur. Dans notre projet Symfony, elle comprend :

- Les **vues Twig** (/templates) qui génèrent le HTML dynamique ;
- Les fichiers **CSS et JS** placés dans /public ;
- L'utilisation de **Bootstrap** pour le design ;
- Et éventuellement du **JavaScript** pour améliorer l'interactivité.

2. Couche métier (Backend)

C'est le cœur de l'application. Cette couche est responsable :

- Du traitement des requêtes (via les Contrôleurs dans /src/Controller) ;
- De l'application des règles métier ;
- De la gestion de la sécurité (accès utilisateur, rôles, etc.) ;
- De la logique des services (ex : service de notification, envoi d'e-mails, vérification d'URL, etc.).

Symfony fournit une organisation claire via son système de services, événements, et middlewares, qui renforcent la séparation des responsabilités.

3. Couche d'accès aux données (Data)

Cette couche gère la persistance des données dans une base de données relationnelle MariaDB. Elle est composée de :

- Entités Doctrine (/src/Entity) représentant les tables ;
- Repositories (/src/Repository) pour interagir avec la base de données ;
- Doctrine ORM, qui permet de manipuler les données sous forme d'objets PHP.

Grâce à cette couche, la logique métier peut récupérer ou modifier les données sans dépendre du SQL brut.

II. Présentation de technologies utilisées

Afin de garantir la performance, la maintenabilité et la sécurité du système, un ensemble de technologies modernes et adaptées a été sélectionné. Le choix de ces technologies repose sur leur compatibilité, leur robustesse, leur popularité dans le développement web, ainsi que sur leur capacité à répondre aux besoins spécifiques du projet. La combinaison d'un backend puissant avec un frontend ergonomique permet de créer une application complète, intuitive et évolutive.

➤ Backend :

Le développement côté serveur a été réalisé à l'aide du langage PHP, structuré autour du framework Symfony, reconnu pour sa stabilité, sa modularité et sa conformité aux bonnes pratiques du développement moderne

❖ PHP :



PHP : Hypertext Preprocessor, plus connu sous son sigle PHP

(acronyme récursif), est un langage de programmation libre, principalement utilisé pour produire des pages. Web dynamiques via un serveur HTTP, mais pouvant également fonctionner comme n'importe quel langage interprété de façon locale. PHP est un langage impératif orienté

➤ Frontend :

L'interface utilisateur repose sur un ensemble de technologies web standards HTML5, CSS et BOOTSTRAP permettant de concevoir une interface claire, responsive et agréable à utiliser sur tous types de supports.

❖ HTML5 :



L'HyperText Markup Langague, généralement abrégé

HTML, est le format de données conçu pour représenter les pages web. C'est un langage de balisage permettant d'écrire de l'hypertexte, d'où son nom.

❖ CSS :



CSS est l'acronyme anglais de Cascading Style Sheets qui peut se traduire

par “feuilles de style en cascade “. Le CSS est un langage informatique utilisé sur l’internet pour mettre en forme les fichiers HTML ou XML. Ainsi, les feuilles de style, aussi appelé les fichiers.

➤ **JavaScript & JQuery :**

Pour enrichir l’interactivité de l’interface utilisateur et offrir une navigation fluide et dynamique, le projet intègre des technologies côté client telles que **JavaScript** et sa bibliothèque **jQuery**. Ces outils permettent de manipuler la page web en temps réel, de réagir aux actions de l’utilisateur, et d’interagir efficacement avec le serveur via AJAX.

❖ **JavaScript :**



JavaScript est un langage de programmation de scripts principalement employé dans les pages web interactives mais aussi pour les serveurs.

❖ **JQuery :**



JQuery est une bibliothèque JavaScript libre et multiplateforme créée pour faciliter l’écriture de scripts côté client dans le code HTML des pages web. La première version est lancée en janvier 2006 par John Resig.

➤ **AJAX & JSON :**

Dans le but d’améliorer l’expérience utilisateur et de rendre l’application plus fluide et réactive, des technologies de communication asynchrone ont été mises en œuvre. Parmi celles-ci, **AJAX** et **JSON** jouent un rôle central dans les échanges de données entre le client (navigateur) et le serveur. Elles permettent de charger ou d’envoyer des informations sans avoir à recharger complètement la page, ce qui optimise les performances et la réactivité de l’interface

❖ **AJAX :**

AJAX est une technique de développement web qui permet d’envoyer ou de recevoir des données depuis un serveur **de manière asynchrone**, c’est-à-dire **sans recharger la page entièrement**.

Dans ton projet, AJAX est utilisé pour actualiser dynamiquement certaines parties de l’interface (par exemple, l’ajout ou la mise à jour de données dans une table) tout en assurant une communication fluide avec le backend Symfony.

❖ **JSON**



JSON est un format de données léger et facile à lire, utilisé pour **échanger de**

informations entre le client et le serveur. Il est particulièrement bien adapté au JavaScript, car les objets JSON peuvent être directement manipulés comme des objets JavaScript. Dans ce projet, les données récupérées ou envoyées via AJAX sont **formatées en JSON**, ce qui facilite leur traitement côté client et leur interprétation côté serveur.

III. Patrons de conception utilisés

1. Définition :

Les patrons de conception sont des solutions standardisées et éprouvées, destinées à résoudre des problèmes récurrents de conception dans le développement logiciel.

Ils offrent un cadre méthodologique pour organiser et structurer le code, permettant d'améliorer la qualité, la réutilisabilité, et la maintenance des applications.

En utilisant ces patrons, les développeurs bénéficient d'un vocabulaire commun et d'une base solide facilitant la compréhension et la collaboration au sein des équipes de développement.

2. Types de patrons de conception :

a. Patrons architecturaux :

Les patrons architecturaux sont des modèles conceptuels à grande échelle qui définissent l'organisation globale d'une application logicielle.

Ils déterminent la séparation des responsabilités entre les différents composants ainsi que la manière dont ces composants interagissent pour assurer le bon fonctionnement du système.

Parmi les patrons architecturaux les plus utilisés, on retrouve :

- **MVC (Model-View-Controller)** : ce patron sépare l'application en trois parties distinctes — le modèle qui gère les données et la logique métier, la vue qui s'occupe de l'affichage et de l'interface utilisateur, et le contrôleur qui gère les interactions entre l'utilisateur et le modèle. Cette séparation facilite la modularité, la maintenance et la testabilité du système.
- **Microservices** : ce modèle divise l'application en plusieurs services autonomes et indépendants, chacun responsable d'une fonction métier spécifique. Les services communiquent via des interfaces bien définies, ce qui améliore la scalabilité, la flexibilité et la facilité de déploiement.
- **Architecture en couches** : l'application est organisée en couches distinctes, telles que la couche présentation, la couche métier et la couche accès aux données. Chaque couche est responsable d'une fonction spécifique et communique uniquement avec la

couche immédiatement adjacente, assurant ainsi une meilleure organisation et une séparation claire des responsabilités.

L'adoption de ces patrons permet d'améliorer la qualité globale du logiciel, en rendant l'application plus modulaire, évolutive et facile à maintenir.

b. Patrons de création :

Les patrons de création concernent les mécanismes permettant de créer des objets de manière flexible et adaptée au contexte, tout en masquant la complexité liée à leur instanciation.

Ces patrons facilitent la gestion de la création d'objets, améliorent la réutilisabilité du code et garantissent souvent un meilleur contrôle sur le processus de construction.

Parmi les principaux patrons de création, on distingue :

- **Singleton** : garantit qu'une classe ne possède qu'une seule instance accessible globalement au sein de l'application, évitant ainsi la multiplication inutile des objets.
 - **Factory Method** : définit une interface pour créer un objet, mais laisse aux sous-classes le soin de décider de la classe concrète à instancier, permettant ainsi une flexibilité accrue.
 - **Abstract Factory** : fournit une interface pour créer des familles d'objets liés sans spécifier leurs classes concrètes, favorisant la cohérence entre les objets créés.
 - **Builder** : sépare la construction d'un objet complexe de sa représentation, permettant d'obtenir différentes représentations via un même processus de construction.
-
- **Prototype** : crée un nouvel objet en copiant un objet existant, évitant ainsi des instanciations coûteuses ou complexes.

c. Patrons structurels :

Les patrons structurels définissent la manière d'assembler des classes ou des objets afin de former des structures plus grandes, modulaires et flexibles.

Ils facilitent la composition et l'organisation des éléments logiciels tout en simplifiant les interactions entre eux.

Parmi les principaux patrons structurels, on peut citer :

- **Adapter** : permet de faire fonctionner ensemble des interfaces incompatibles en convertissant l'interface d'une classe en une autre attendue par les clients.
- **Decorator** : ajoute dynamiquement des fonctionnalités ou responsabilités supplémentaires à un objet, sans modifier sa structure de base.

- **Facade** : fournit une interface unifiée et simplifiée à un ensemble complexe de classes ou de sous-systèmes, facilitant ainsi leur utilisation.
- **Composite** : permet de composer des objets en structures arborescentes pour traiter de manière uniforme les objets simples comme les compositions d'objets.
- **Proxy** : contrôle l'accès à un autre objet, en pouvant ajouter des fonctionnalités telles que le contrôle d'accès, la mise en cache ou la gestion des ressources.
- **Flyweight** : optimise l'utilisation de la mémoire en partageant des objets communs lorsqu'il y a un grand nombre d'instances similaires.
- **Bridge** : sépare une abstraction d'une implémentation afin que les deux puissent évoluer indépendamment.

d. **Patrons comportementaux :**

Les patrons comportementaux concernent la communication entre objets ainsi que la répartition des responsabilités.

Ils permettent d'organiser les interactions, de gérer le flux des opérations, et d'assurer une collaboration flexible et cohérente entre les différents objets d'un système.

Parmi les principaux patrons comportementaux, on trouve :

- **Observer** : établit une relation de dépendance un-à-plusieurs entre objets, où plusieurs observateurs sont automatiquement notifiés et mis à jour lorsqu'un objet sujet change d'état.
- **Strategy** : permet de définir une famille d'algorithmes interchangeables que l'on peut modifier dynamiquement sans changer le contexte qui les utilise.
- **Command** : encapsule une requête ou une action sous forme d'objet, ce qui permet de la paramétrer, de la stocker ou de la transmettre.
- **State** : permet à un objet de modifier son comportement lorsque son état interne change, donnant ainsi l'illusion que l'objet change de classe.
- **Mediator** : centralise les communications entre plusieurs objets afin de réduire les dépendances directes entre eux.
- **Chain of Responsibility** : transmet une requête à une chaîne d'objets jusqu'à ce qu'un objet capable de traiter la requête intervienne.
- **Template Method** : définit le squelette d'un algorithme en déléguant certaines étapes à des sous-classes qui peuvent les personnaliser.
- **Iterator** : offre un moyen d'accéder séquentiellement aux éléments d'une collection sans exposer sa représentation interne.
- **Visitor** : permet de séparer un algorithme des objets sur lesquels il opère, facilitant ainsi l'ajout de nouvelles opérations sans modifier les classes des objets.
- **Memento** : capture et externalise l'état interne d'un objet sans violer son encapsulation, afin de permettre sa restauration ultérieure.

3. Les patrons de conception utilisés :

a. Patrons Comportemental :

Les patrons comportementaux définissent la manière dont les objets interagissent et communiquent entre eux, en répartissant clairement les responsabilités. Ils facilitent la communication, la coordination et la flexibilité du comportement des objets dans l'application.

Dossier / Fichier	Rôle dans le projet CentralProjet	Type de patron
src/EventSubscriber/NotificationSubscriber.php	Notifie automatiquement plusieurs services lors d'un événement déclenché (ex : nouvelle notification)	Observer
src/Command/CheckUrlsCommand.php	Encapsule une commande pour vérifier l'état des URLs via la console	Command
src/Service/UrlCheckerServiceUrlService.php	Permet de changer dynamiquement la méthode de vérification des URLs	Strategy
src/Service/HistoriqueService.php	Capture et restaure l'état des entités lors des modifications (historique des actions)	Memento / State
src/Service/NotificationService.php	Centralise les interactions entre les différents services de notification	Mediator
src/Form/	Définit une structure de formulaire avec certaines étapes personnalisables	Template Method
src/Repository/	Permet de parcourir les collections d'entités sans exposer la structure interne	Iterator

b. Patrons de création :

Les patrons de création se concentrent sur la manière de créer les objets de manière souple, tout en masquant la complexité de l'instanciation. Dans mon projet, plusieurs composants exploitent ces patrons pour rendre le code plus modulaire, maintenable et réutilisable.

Le tableau ci-dessous présente les dossiers/fichiers du projet Symfony dans lesquels ces patrons de création sont appliqués :

Dossier / Fichier	Rôle / Exemple concret	Type de patron
src/Service/EmailService.php	Service unique pour l'envoi d'e-mails – instancié une seule fois et partagé	Singleton, Factory
src/Service/HebergementExpirationService.php	Service qui crée dynamiquement des vérifications selon les paramètres reçus	Factory Method
src/Service/HistoriqueService.php	Génère des objets historiques à partir d'événements capturés	Builder, Factory

src/Service/NotificationService.php	Centralise et crée différents types de notifications	Factory, Builder
src/Service/UrlCheckerServiceUrlService.php	Crée dynamiquement des processus de vérification de lien	Factory, Singleton
src/Form/	Crée des objets de formulaire liés à des entités complexes (ex. projets, développeurs)	Builder, Factory Method
src/Entity/	Possibilité de cloner ou dupliquer des entités (patron de clonage)	Prototype

c. Patrons architectural :

Dans le cadre de ce projet, j'ai appliqué le patron de conception architectural **MVC**, qui est largement utilisé dans les frameworks web tels que Symfony. Ce patron permet de séparer les responsabilités entre la gestion des données (modèle), la logique de traitement (contrôleur), et l'affichage (vue). Cette séparation favorise la maintenabilité, la clarté du code et facilite les tests.

Le tableau suivant présente les composants de l'application associés à chacun des rôles du modèle MVC :

Dossier / Fichier	Rôle dans MVC
src/Controller/	Contrôleurs de l'application
src/Entity/	Modèle : représentation des données métier
src/Form/	Vue : liaison entre entité et formulaire
templates	Vue : interface utilisateur via Twig
src/Repository/	Accès aux données du modèle

d. Patrons structurel :

Les patrons structurels définissent comment les classes et objets s'organisent pour former des structures flexibles et réutilisables. Leur but est d'assurer une meilleure organisation du code en facilitant la composition, l'extension et l'intégration des composants.

Dossier / Fichier	Rôle / Exemple concret	Type de patron structurel
src/Service/EmailService.php	Fournit une interface simple pour envoyer des e-mails .	Facade, Adapter
src/Service/NotificationService.php	Ajoute dynamiquement différents canaux de notification (mail, interface)	Decorator, Facade
src/Twig/	Personnalisation de l'affichage via des extensions Twig (ex. filtres)	Decorator, Adapter
src/Repository/	Abstraction de l'accès à la base de données via Doctrine	Proxy, Bridge

src/Form/	Structure les formulaires en composants réutilisables, liés aux entités	Composite, Builder
-----------	---	--------------------

4. Schémas globaux des patrons utilisés :

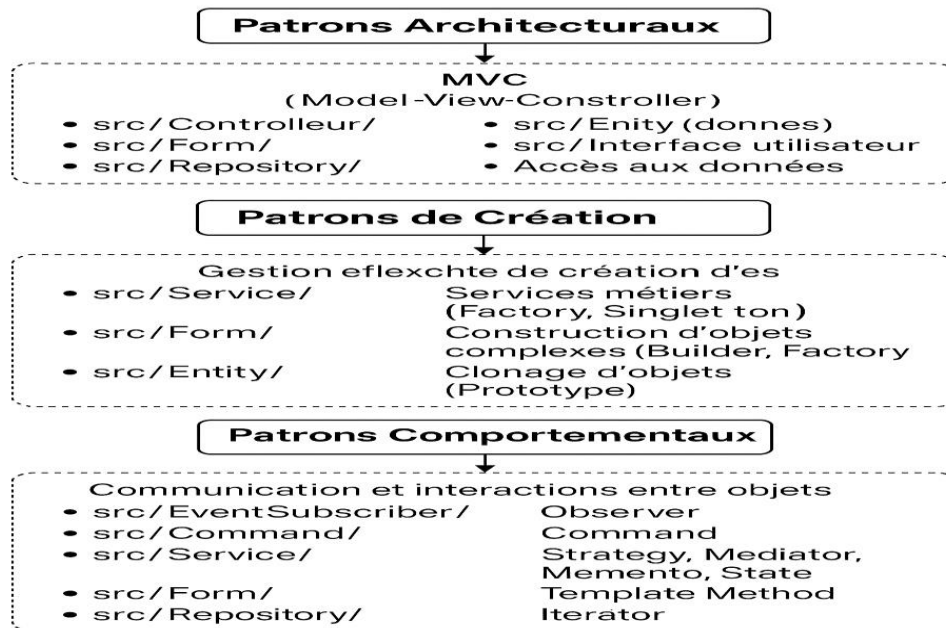


Figure 49. Schéma des patrons utilisés.

IV. Le framework adopté

Dans le cadre de ce projet, le choix du framework constitue une étape cruciale, car il conditionne à la fois la structure, la maintenance et l'évolution future de l'application. Un framework permet de standardiser le développement, d'accélérer la production grâce à des outils et des bibliothèques intégrées, et d'assurer une meilleure organisation du code.

Le framework adopté a été sélectionné en fonction de plusieurs critères essentiels tels que la facilité d'apprentissage, la communauté active, la compatibilité avec les besoins fonctionnels du projet, ainsi que la performance et la sécurité. Cette section présente une vue d'ensemble du framework choisi, ses principales caractéristiques, et les raisons qui ont motivé ce choix stratégique pour le bon déroulement du développement.

❖ SYMFONY 7:



Symfony 7 est la dernière version majeure du framework PHP Symfony, un ensemble de composants et d'outils open-source destinés au développement d'applications web. Symfony 7 s'appuie sur les dernières avancées du langage PHP (version 8.2 et plus) pour offrir un cadre robuste, performant et modulable. Ce framework facilite la création d'applications évolutives grâce à une architecture basée sur des composants réutilisables, une

gestion avancée des routes, des services et une intégration simplifiée avec les bases de données et les API.

Grâce à ses améliorations en termes de sécurité, de performance et d'expérience développeur, Symfony 7 est largement utilisé pour développer des applications web modernes, professionnelles et maintenables sur le long terme.

❖ **BOOTSTRAP :**



Bootstrap est une collection d'outils utiles à la création du design (graphisme, animation et interactions avec la page dans le navigateur, etc.) de sites et d'applications web. C'est un ensemble qui contient des codes HTML et CSS, des formulaires, boutons, outils de navigation et d'autres éléments interactifs, ainsi que des extensions JavaScript en option. C'est l'un des projets les plus populaires sur la plate-forme de gestion de développement GitHub.

V. **Les outils de développement utilisés**

➤ **Environnement de développement intégré (IDE) :**

- ❖ **Visual Studio Code** : un éditeur de code source léger et performant, offrant de nombreuses extensions dédiées à Symfony, PHP, Twig, et CSS. Visual Studio Code facilite l'écriture, le débogage et la gestion du code grâce à son interface intuitive et ses fonctionnalités avancées d'auto-complétions et de coloration syntaxique

➤ **Gestionnaire de dépendances :**

- ❖ **Composer** : outil essentiel pour la gestion des dépendances PHP dans Symfony. Composer permet d'installer, de mettre à jour et de gérer facilement les bibliothèques nécessaires au projet.

➤ **Gestion de version :**

- ❖ **Git** : système de contrôle de version distribué utilisé pour suivre l'historique des modifications du code, gérer les branches et faciliter la collaboration.
- ❖ **GitHub** : plateforme d'hébergement de dépôt Git permettant de sauvegarder et partager le code source du projet.

➤ **Test :**

- ❖ **PHPUNIT** : est un framework de test unitaire pour le langage PHP. Il permet aux développeurs d'écrire et d'exécuter des tests automatisés afin de vérifier que chaque unité de code (fonction, classe, méthode) fonctionne correctement. PHPUnit facilite ainsi la détection précoce des erreurs et améliore la qualité et la maintenance du code.

VI. **Choix du SGBDR**

Le choix du Système de Gestion de Base de Données Relationnelle (SGBDR) est une étape cruciale pour assurer la performance, la fiabilité et la facilité de maintenance de l'application.

Pour ce projet, j'ai opté pour **MySQL/MariaDB**, qui est un SGBDR open source très populaire et largement supporté par Symfony. Ce choix est motivé par plusieurs raisons

- **Compatibilité avec Symfony** : MySQL/MariaDB s'intègre parfaitement avec le framework Symfony, notamment via Doctrine ORM, facilitant la gestion des entités et des relations.
- **Performance et fiabilité** : Ces SGBDR offrent des performances stables et une bonne gestion des transactions, ce qui est essentiel pour assurer la cohérence des données.
- **Communauté et support** : MySQL et MariaDB bénéficient d'une large communauté active, offrant un grand nombre de ressources, de documentations et d'outils.
- **Facilité d'installation et d'utilisation** : Ces bases de données sont faciles à installer dans un environnement local avec des outils comme XAMPP, et disposent d'interfaces graphiques conviviales telles que PhpMyAdmin.
- **Flexibilité et évolutivité** : MySQL/MariaDB supportent une grande variété de types de données et permettent de gérer des bases volumineuses, ce qui assure la montée en charge future de l'application.



Chapitre 4 : Réalisation/Mise en œuvre de la solution

Chapitre 4 : Réalisation/Mise en œuvre de la solution

I. Structure du projet

La structure du projet **CentralProjet** a été pensée pour organiser de manière claire et efficace l'ensemble des fichiers et dossiers nécessaires au bon fonctionnement de l'application. Cette organisation facilite la navigation dans le code, la maintenance et l'évolution future du projet. Elle respecte les conventions du framework Symfony, en séparant les différentes responsabilités telles que la gestion des contrôleurs, des entités, des templates, des configurations et des tests.

La capture d'écran suivante illustre l'arborescence complète du projet, montrant la répartition des différents composants essentiels à son architecture.

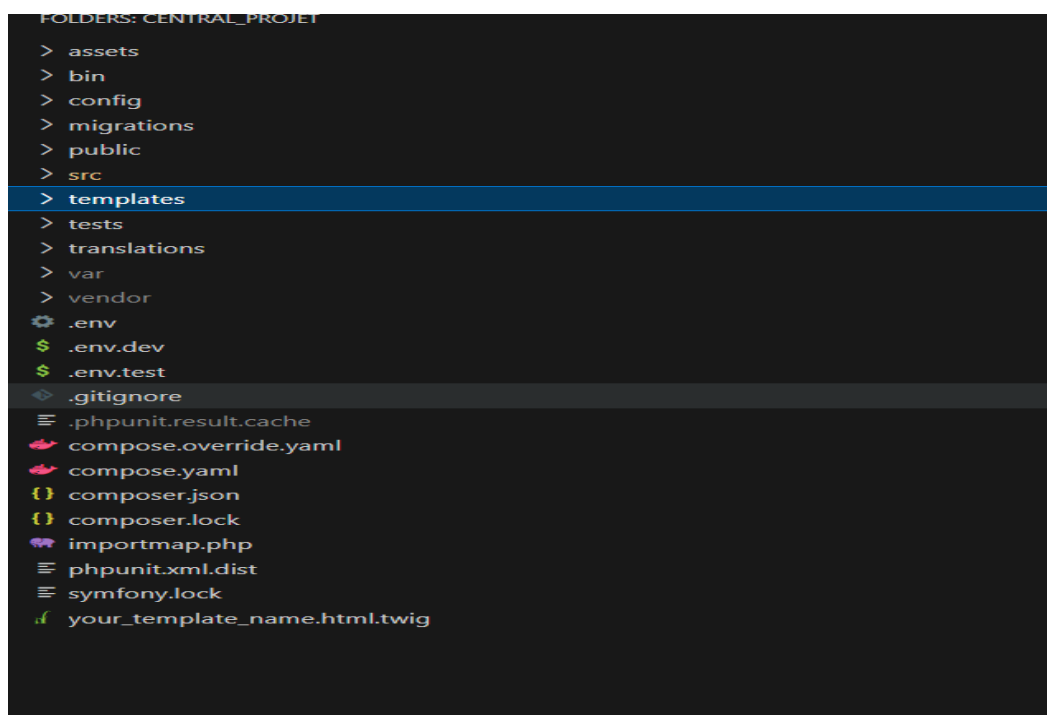


Figure 50 Structure du projet complète 1

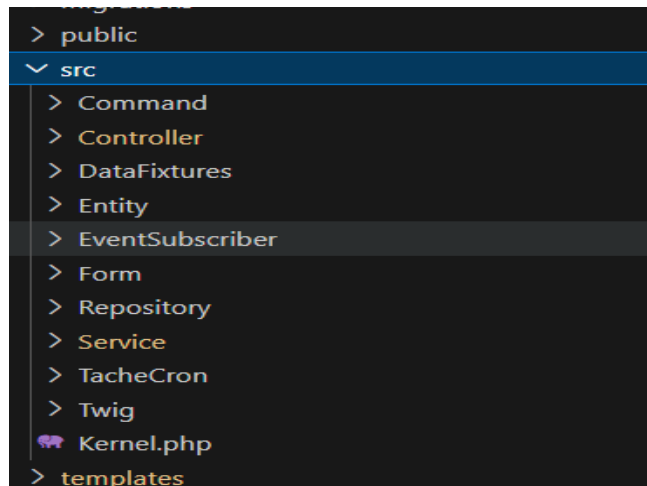


Figure 51 Structure du projet /src

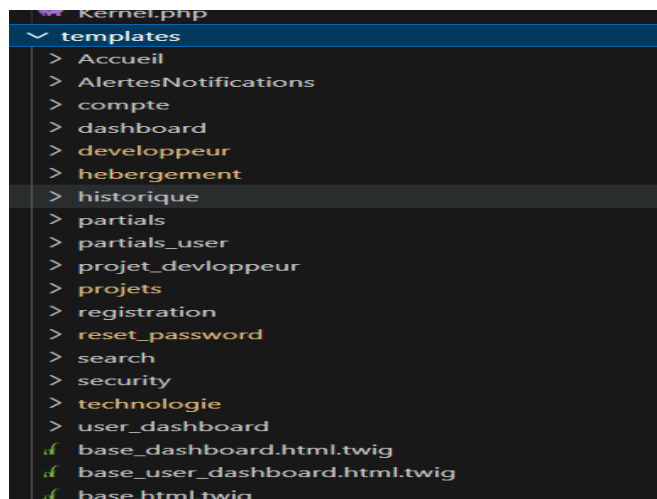


Figure 52 Structure du projet /template.

II. Schéma de la base de données

Le schéma de base de données constitue une représentation visuelle et logique de la structure des données utilisées par l'application. Il permet de modéliser les tables, leurs colonnes, ainsi que les relations entre elles (clé primaire, clé étrangère, cardinalités, etc.). Cette étape est essentielle dans la conception d'un système d'information, car elle garantit une organisation cohérente, efficace et évolutive des données.

Le schéma facilite la compréhension globale du fonctionnement de la base tant pour les développeurs que pour les administrateurs. Il permet également de détecter d'éventuelles incohérences ou redondances avant la mise en œuvre technique. Dans le cadre de ce projet, le schéma a été élaboré à partir de la structure exportée de la base de données MySQL, puis transformé en diagramme relationnel afin de mieux illustrer les différentes entités du système et leurs interactions.

Voici une brève description des tables clés utilisées dans la base de données :

1. Table User :

- **id :**
identifiant du projet.
- **nom :**
nom du projet.
- **type :**
type de projet ('web', 'mobile', 'desktop').
- **description :**
description détaillée.
- **url :**
lien vers le projet déployé.
- **acces_code_source :**
lien ou méthode d'accès au code source.
- **acces_environnement :**
accès aux environnements de test ou production.
- **status :**
état actuel du projet.
- **createdAt, updatedAt :**
suivi des dates de création et de modification.

2. Table Développeur :

- **id :**
identifiant du développeur.
- **nom, prenom :**
nom complet.
- **email :**
contact du développeur.
- **telephone :**
numéro de téléphone.
- **user_id :**
clé étrangère pointant vers la table 'user'.

- **createdAt, updatedAt :**
suivi des dates de création et de modification.

3. **Table Projet :**

- **id :**
identifiant du projet.
- **nom :**
nom du projet.
- **type :**
type de projet ('web', 'mobile', 'desktop').
- **description :**
description détaillée.
- **url :**
lien vers le projet déployé.
- **acces_code_source :**
lien ou méthode d'accès au code source.
- **acces_environnement :**
accès aux environnements de test ou production.
- **status :**
état actuel du projet.
- **createdAt, updatedAt :**
suivi des dates de création et de modification

4. **Table Hébergement :**

- **id :**
identifiant de l'hébergement.
- **fournisseur :**
nom du prestataire (ex : OVH, Hostinger...).
- **type :**
type d'hébergement (partagé, VPS, cloud...).
- **url :**

adresse du service hébergé.

- `date_expiration` :

date d'expiration du service.

- `createdAt, updatedAt` :

dates de création et de dernière mise à jour.

5. **Table Technologie :**

- `id` :

identifiant unique.

- `nom` :

nom de la technologie (ex : Symfony, React...).

- `version` :

version utilisée.

- `createdAt, updatedAt` :

dates d'ajout et de mise à jour.

6. **Table Notification :**

- `id`:

identifiant de la notification.

- `title`:

titre de la notification.

- `message`:

contenu de la notification.

- `is_read` :

statut de lecture ('true' ou 'false').

- `hebergement_id` :

lien avec un hébergement concerné.

- `projet_id` :

lien avec un projet concerné.

- `created_at, updated_at` :

dates de création et de mise à jour.

7. **Table Historique :**

- **id :**
identifiant de l'action.
- **user_id :**
utilisateur ayant réalisé l'action.
- **action :**
type d'opération effectuée (ajout, modification, suppression...).
- **entite_modifier :**
nom de l'entité concernée (projet, utilisateur...).
- **date_modification :**
date de l'action.

III. Présentation des interfaces de l'application/Captures écrans

1. Page d'accueil :

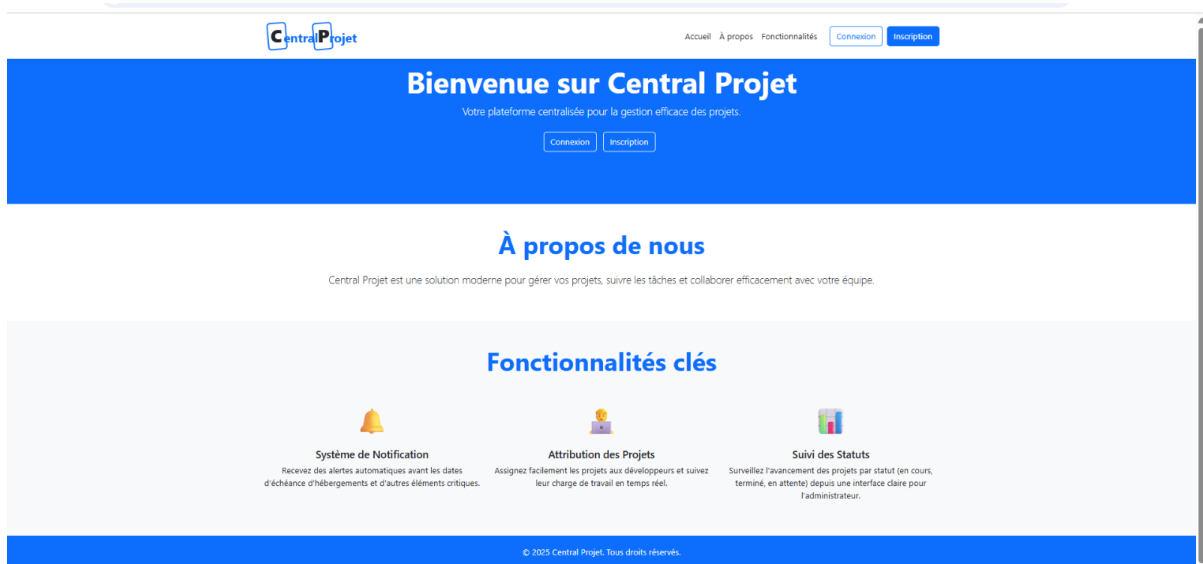


Figure 54. Page d'accueil.

2. Page Register :

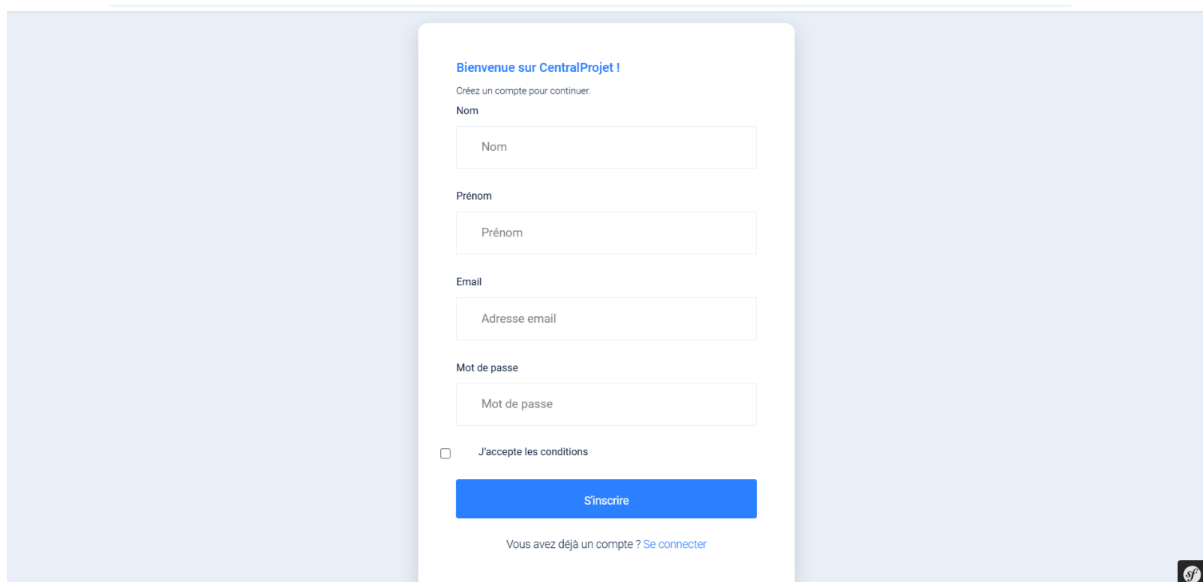
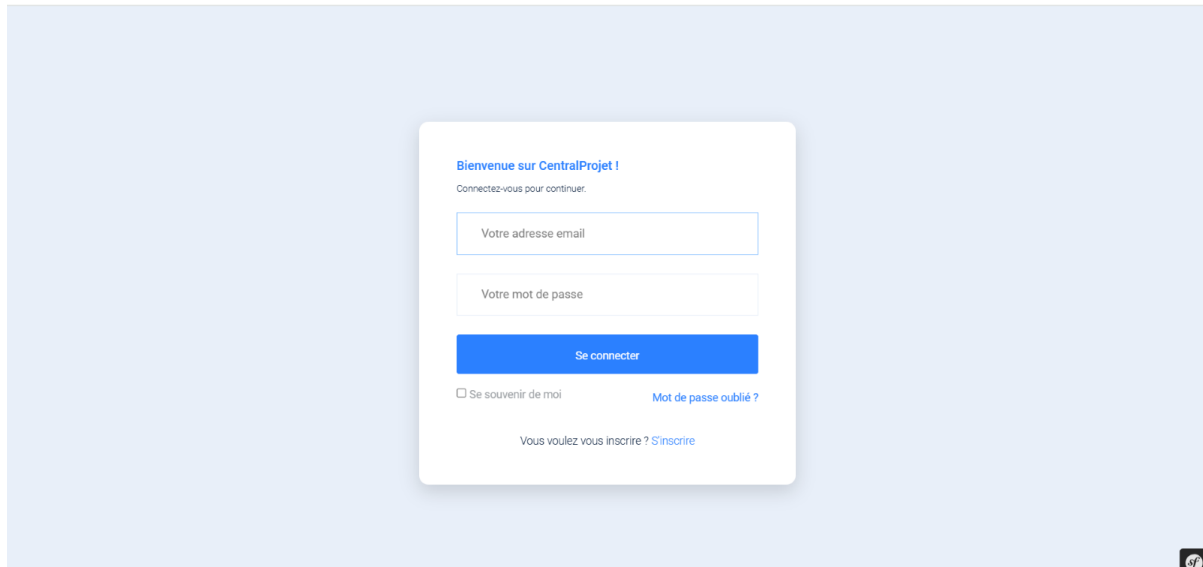


Figure 55. Page Inscription.

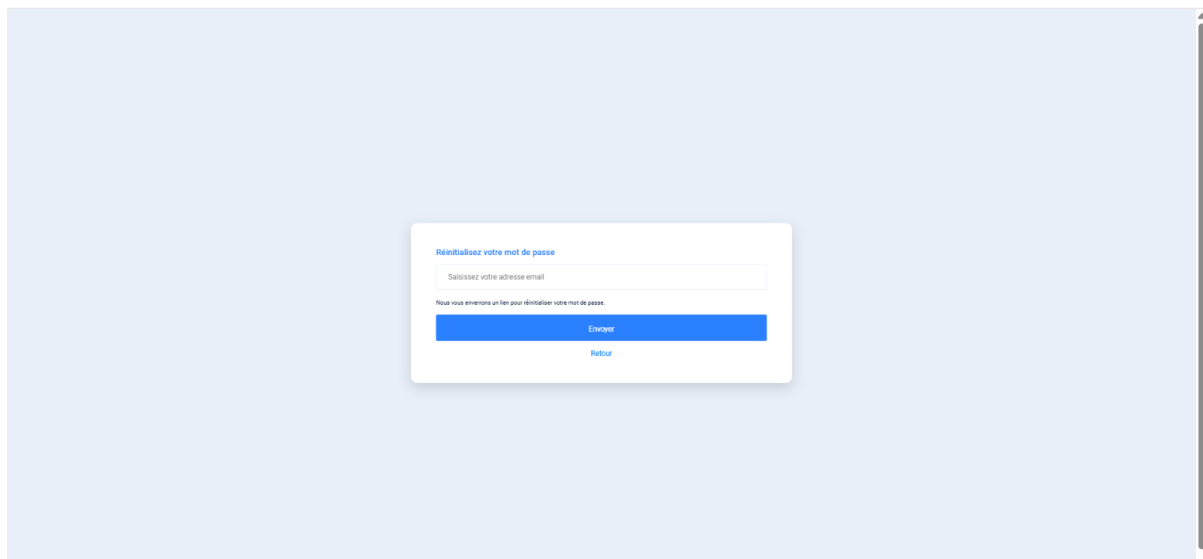
3. Page login :



The screenshot shows a login form titled "Bienvenue sur CentralProjet !". Below the title, it says "Connectez-vous pour continuer." There are two input fields: "Votre adresse email" and "Votre mot de passe". A blue button labeled "Se connecter" is positioned below the password field. Underneath the button, there is a checkbox labeled "Se souvenir de moi" and a link "Mot de passe oublié ?". At the bottom of the form, it says "Vous voulez vous inscrire ? S'inscrire". The entire form is centered on a light blue background.

Figure 56. Page connexion.

4. Page réinitialiser password :



The screenshot shows a password reset form titled "Réinitialisez votre mot de passe". Below the title, it says "Saisissez votre adresse email". There is a single input field for the email address. Below the input field, it says "Nous vous enverrons un lien pour réinitialiser votre mot de passe." There are two buttons: a blue button labeled "Envoyer" and a link labeled "Retour". The entire form is centered on a light blue background.

Figure 57. Page de réinitialisation du mot de passe.

5. Interface Admin :

a. Dashboard :

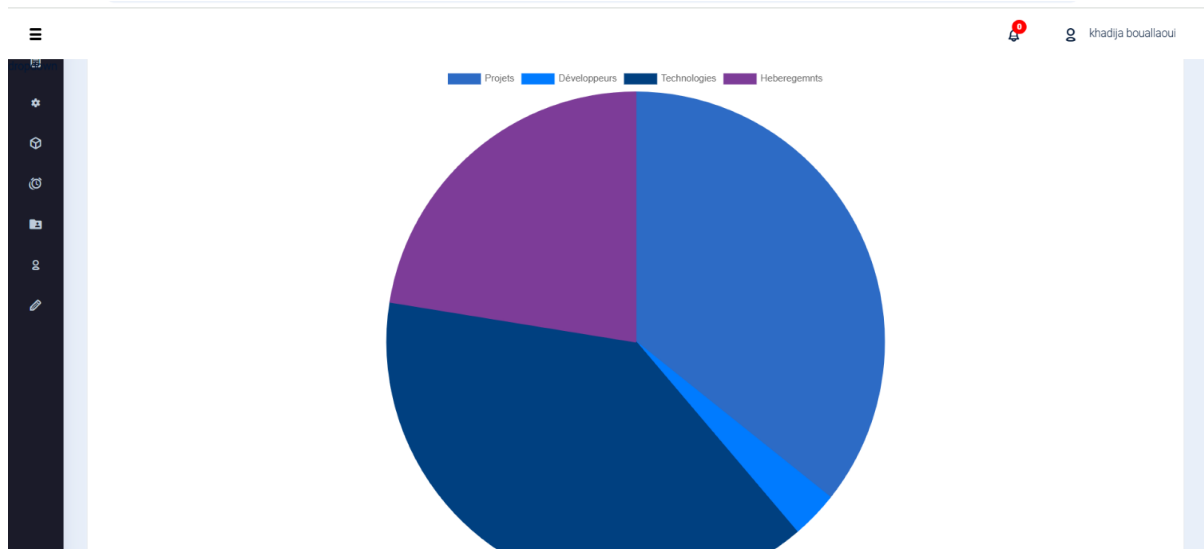


Figure 58. Page Dashboard admin.

b. Gestion Projets :

1) Page principale :

The screenshot shows the main project management page. It features a search bar at the top with the text 'Rechercher' and a magnifying glass icon. To the right of the search bar is a button labeled '+ Nouveau'. Below the search bar is a table with the following columns: Nom, Type, Description, Statut, and Actions. The table contains seven rows of project data.

Nom	Type	Description	Statut	Actions
projet2	web	azertyu	En cours	
projet 3	web	azertyu	A faire	
projet1	web	azertyu	Terminé	
projet 4	web	azertyu	En cours	
project 5	web	azertyu	En cours	
project 6	web	azertyu	Terminé	
project 7	web	azertyu	Terminé	

Figure 59. Page principale projet.

2) Pop-up ajout :

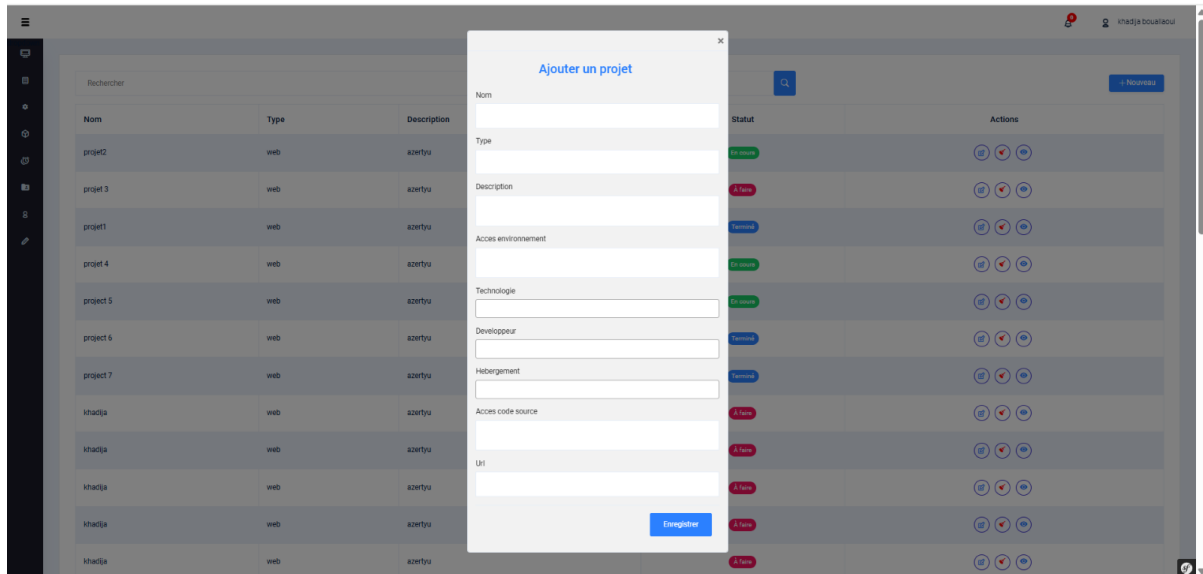


Figure 60. Pop-up ajout projet.

3) Pop-up modification :

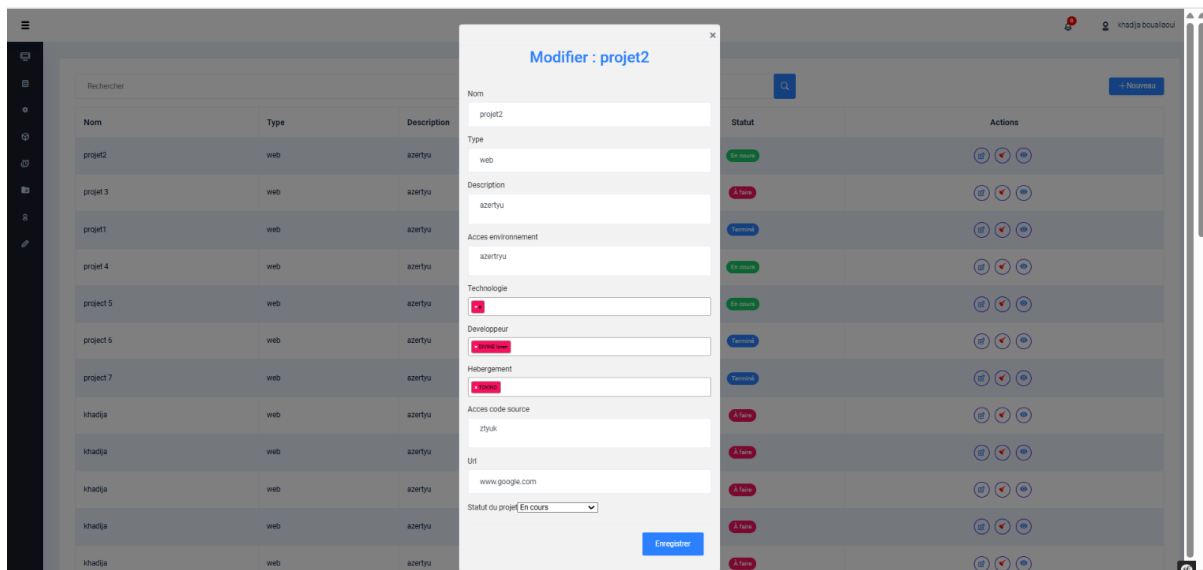


Figure 61. Pop-up modification projet

4) Pop-up suppression :

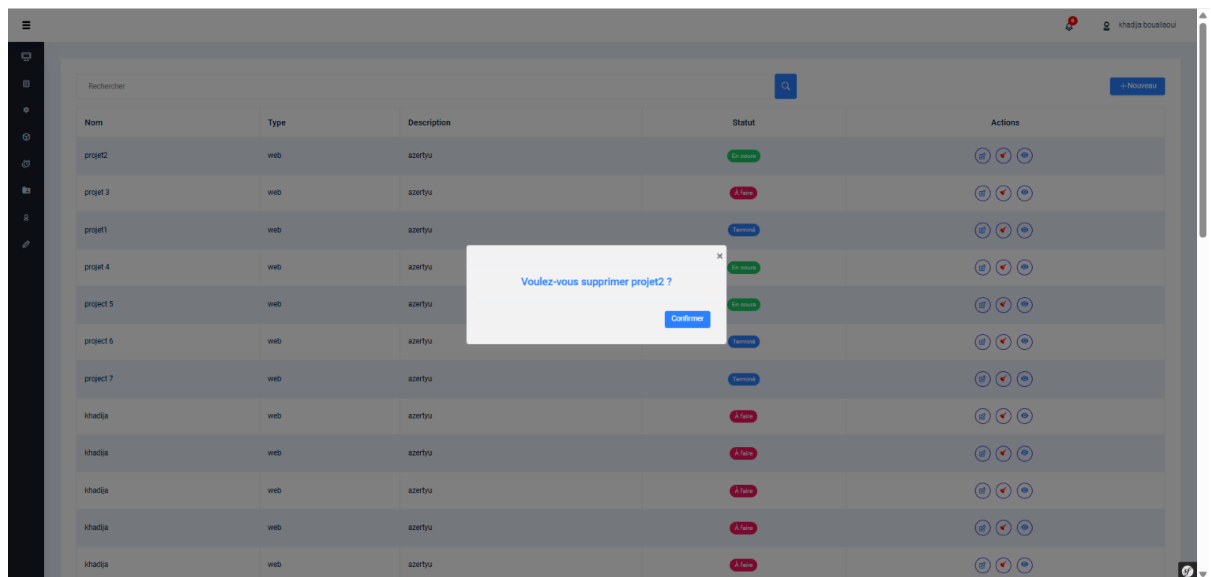


Figure 62. Pop-up suppression projet.

1) Page de recherche :

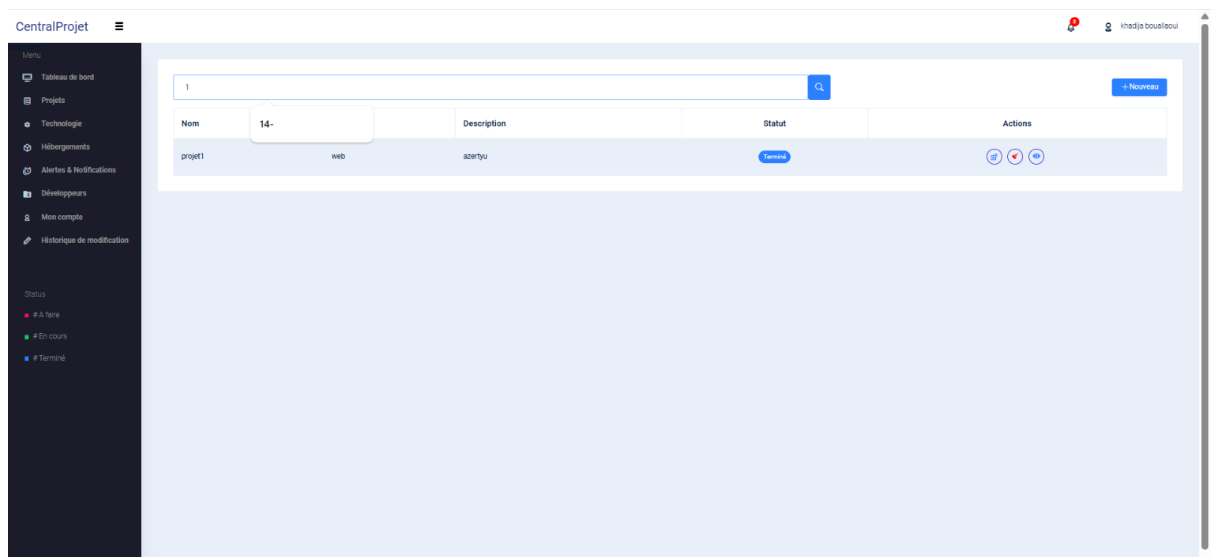


Figure 63. Page de recherche plein

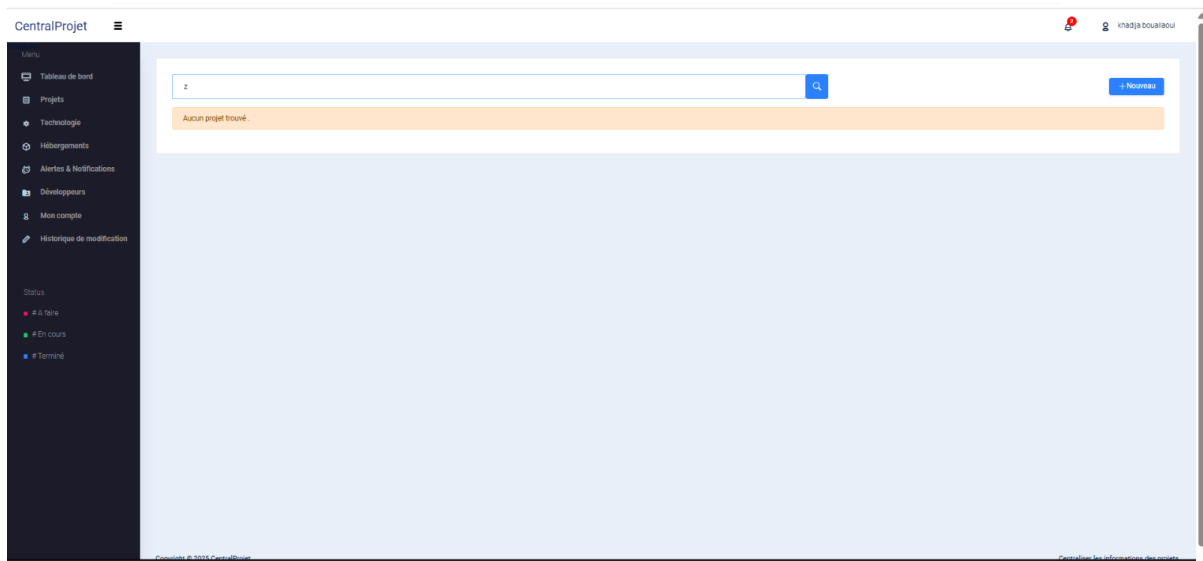


Figure 64. Page de recherche vide.

5) Pop-up voir plus de détails :

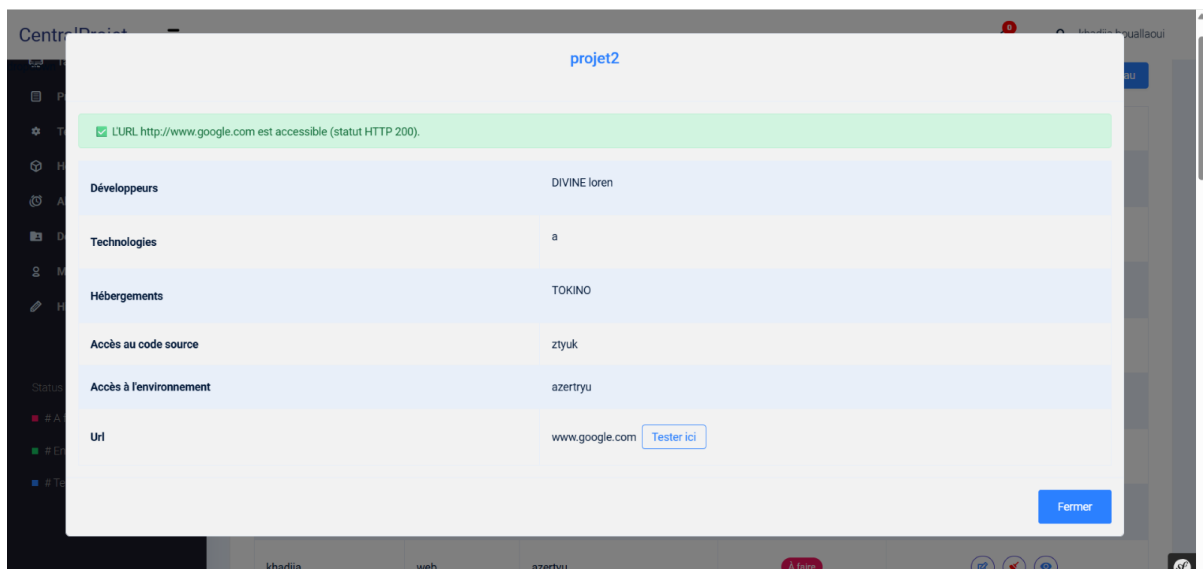


Figure 65. Pop-up plus détail du projet.

6) Page d'erreur Url :

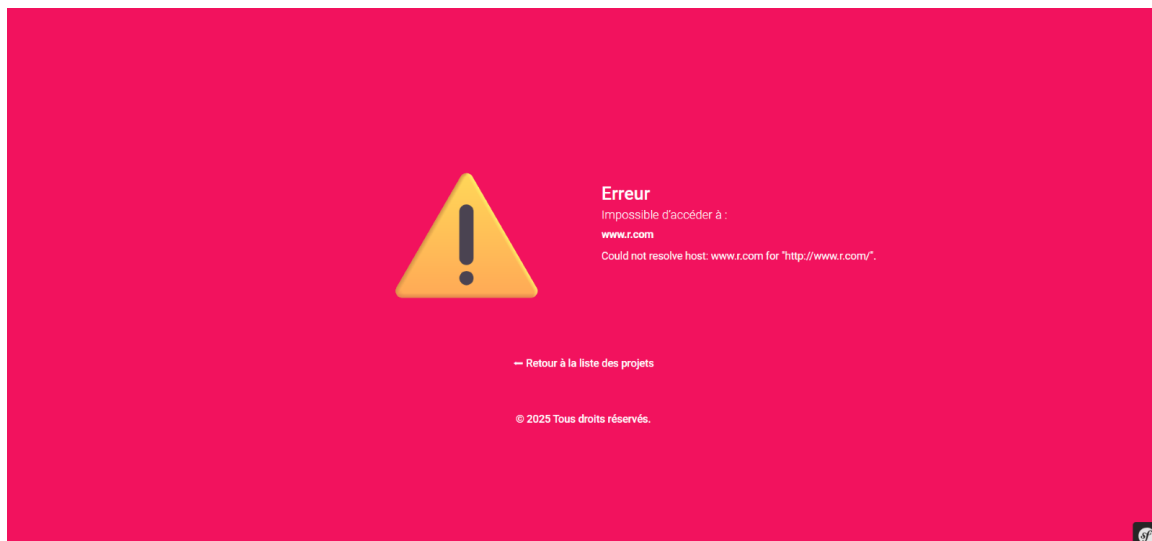


Figure 66. Page erreur du teste url du projet.

7) Page de succès Url :

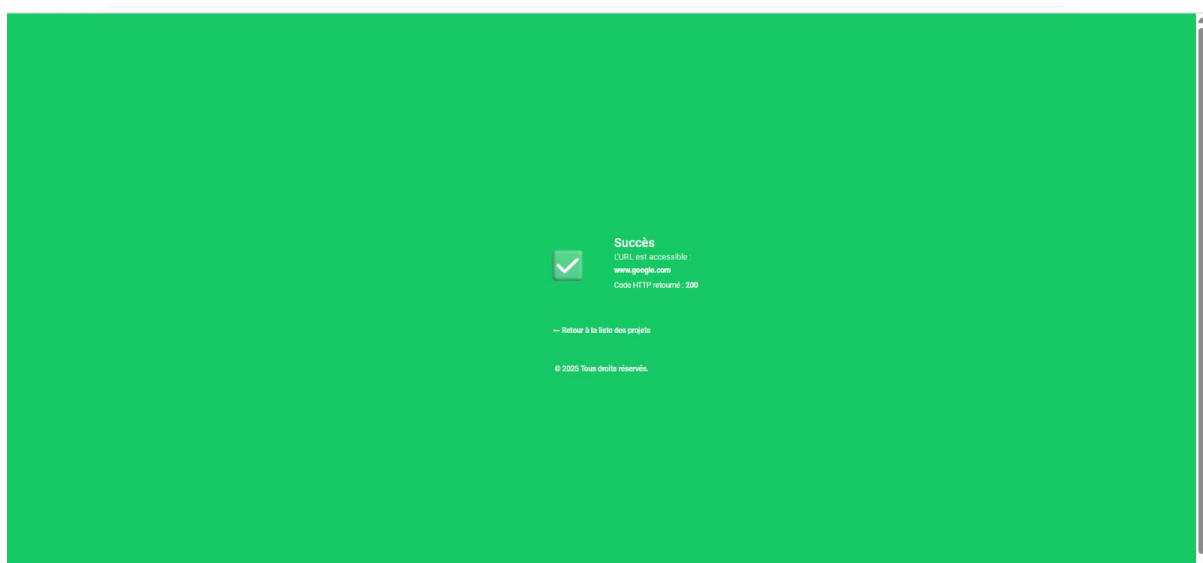
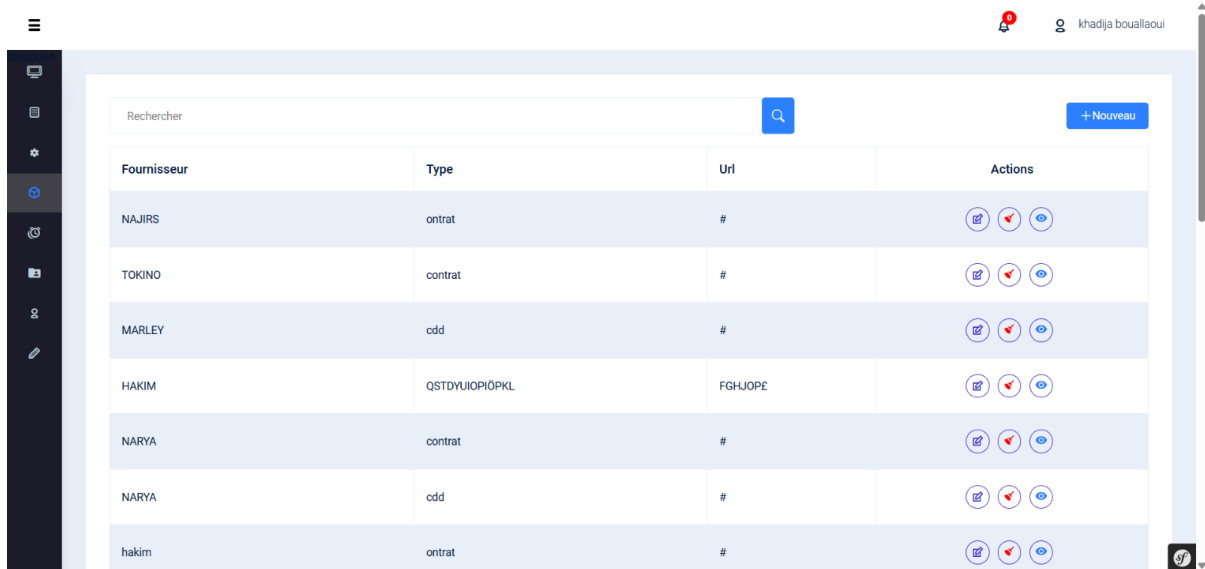


Figure 67. Page succès du teste url du projet.

c. Gestion Hébergements :

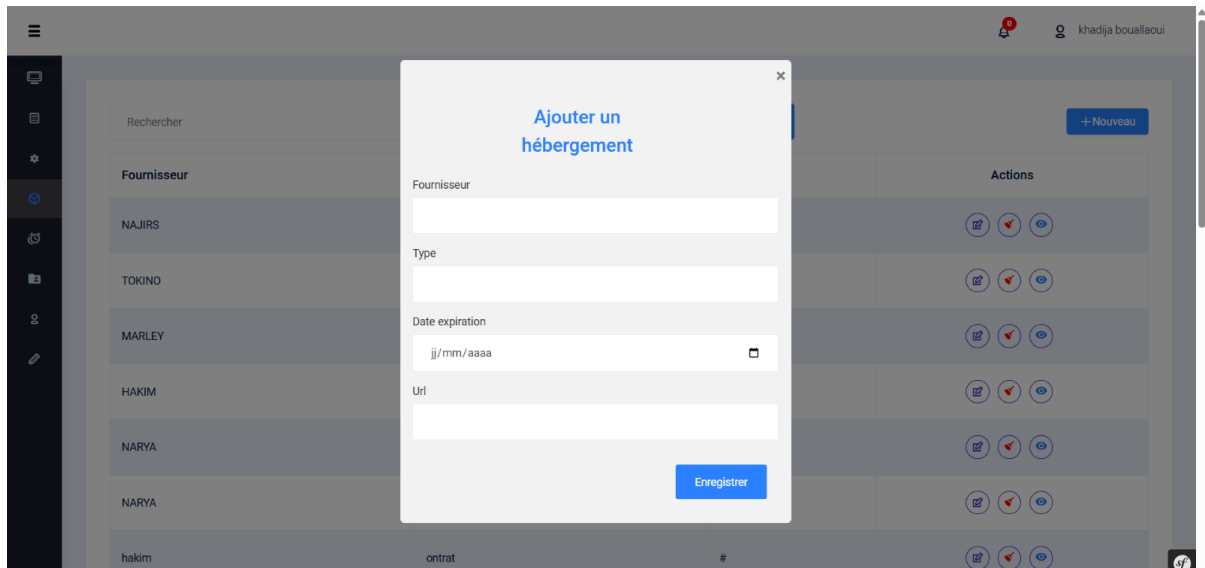
1. Page principale :



Fournisseur	Type	Url	Actions
NAJIRS	ontrat	#	  
TOKINO	contrat	#	  
MARLEY	cdd	#	  
HAKIM	QSTDYUIOPÏÖPKL	FGHJOPE	  
NARYA	contrat	#	  
NARYA	cdd	#	  
hakim	ontrat	#	  

Figure 68. Page principale hébergement.

2. Pop-up ajout :



Ajouter un hébergement

Fournisseur

Type

Date expiration

Url

Enregistrer

Figure 69. Pop-up ajout hébergement.

3. Pop-up modification :

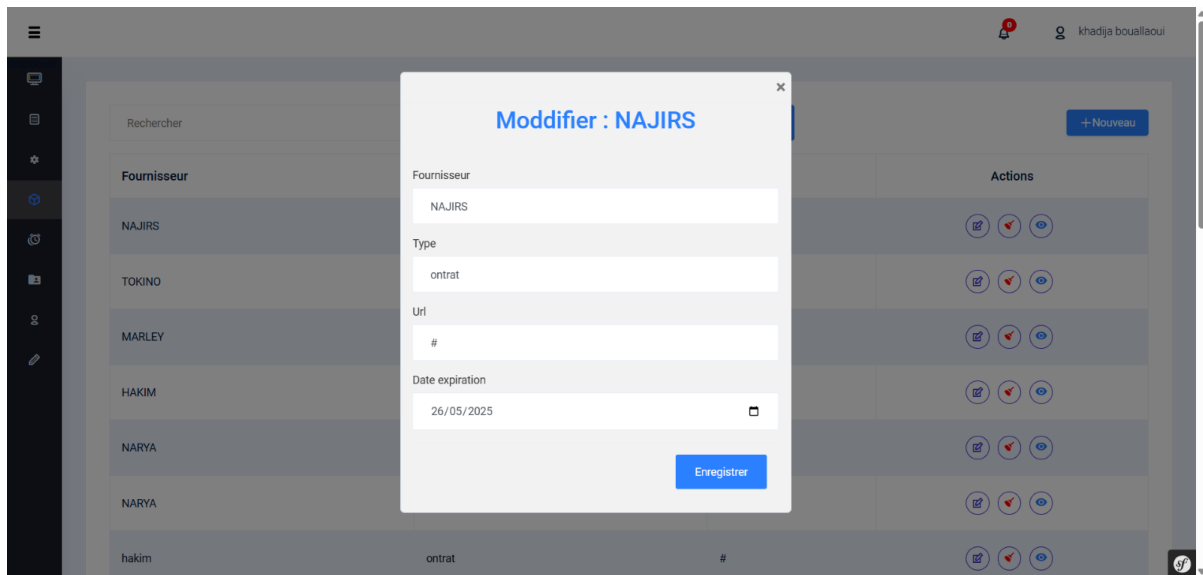


Figure 70. Pop-up modification hébergement.

4. Pop-up suppression :

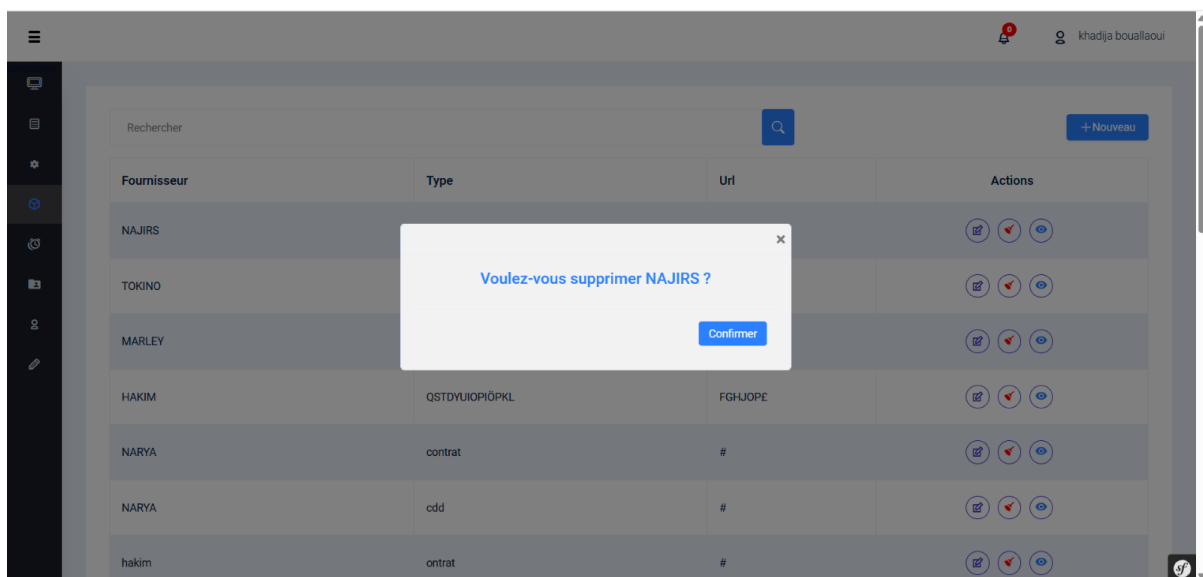


Figure 71. Pop-up suppression hébergement.

5. Page de rechercher :

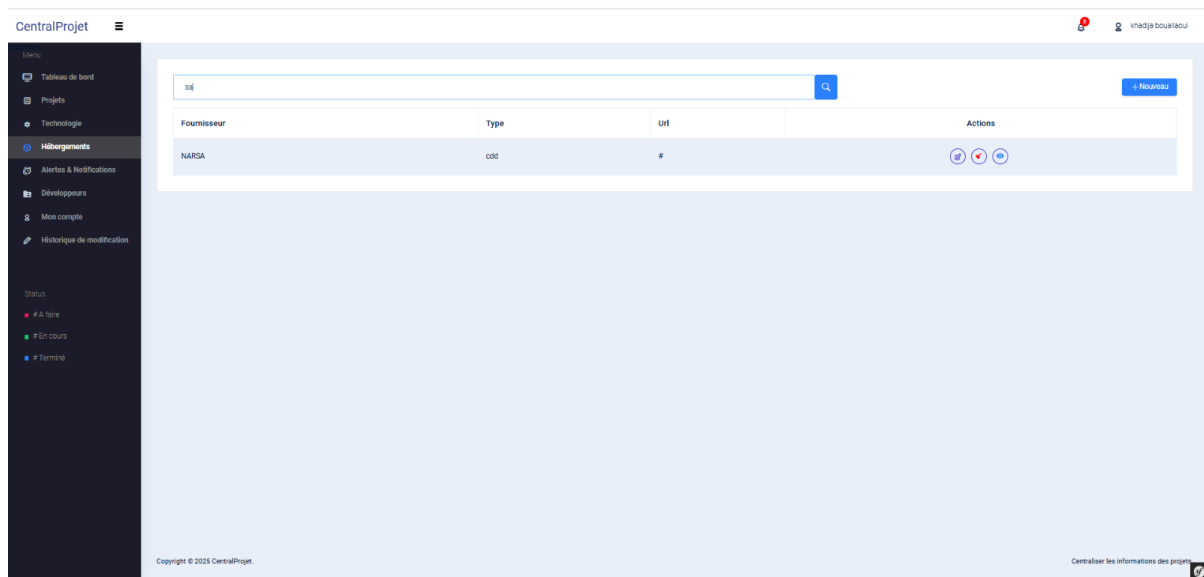


Figure 72. Page recherche hébergement pleine.

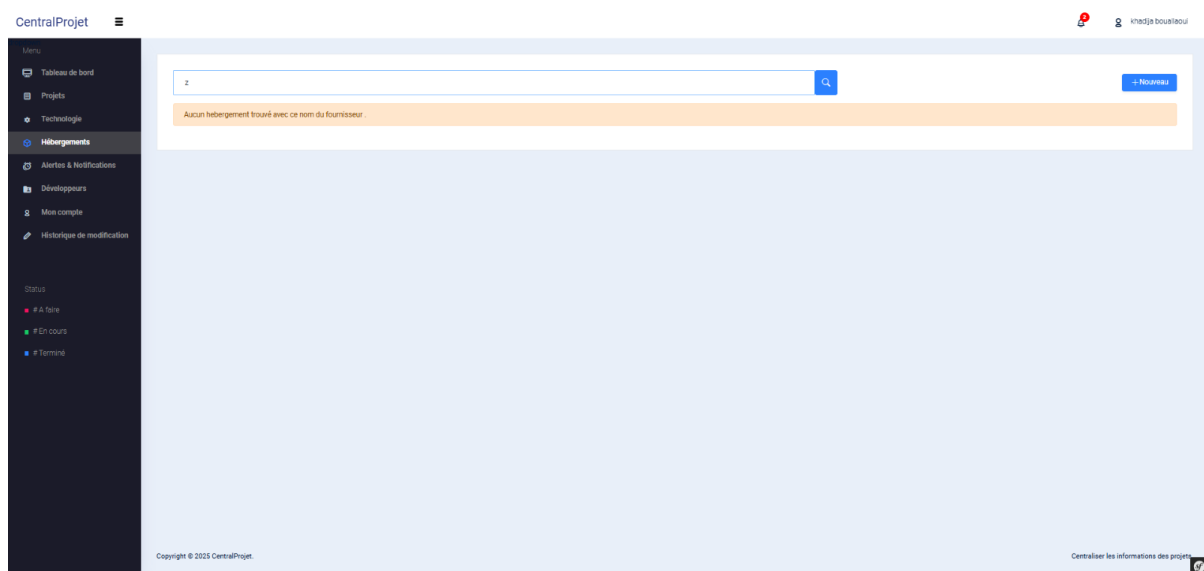


Figure 73. Page recherche hébergement vide.

6. Pop –up voir plus de détails :

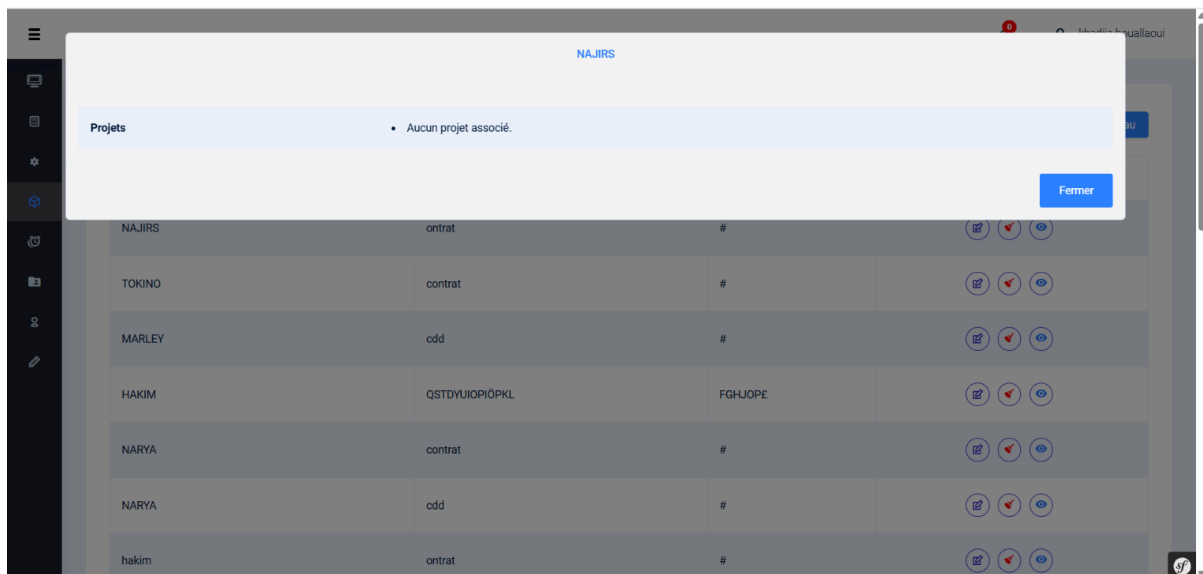


Figure 74. Pop-up plus détail hébergement.

d. Gestion Technologie :

1. Page principale :

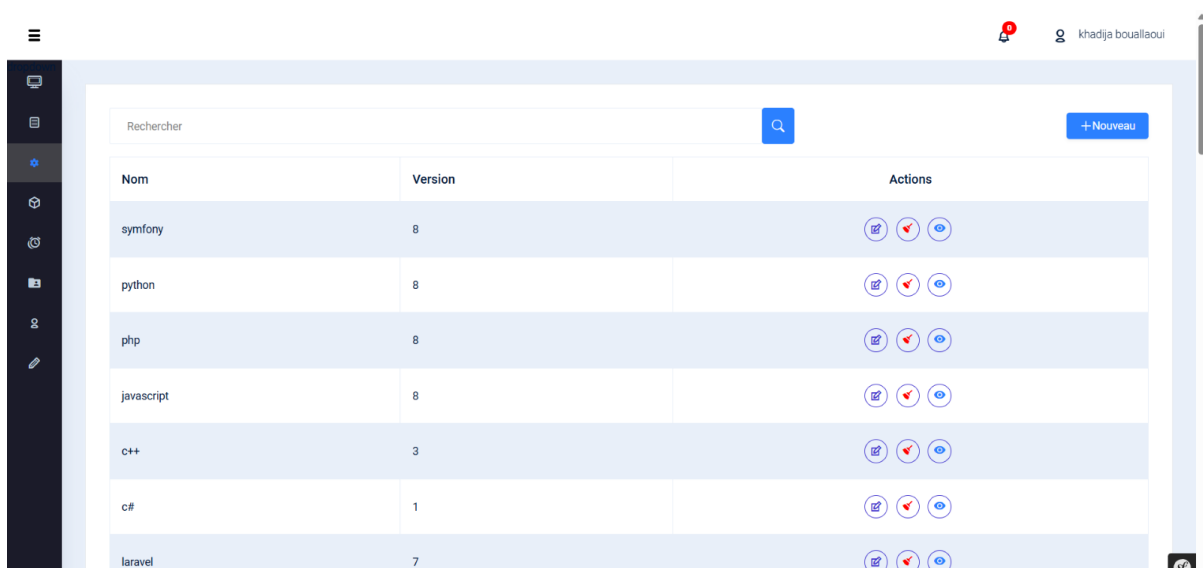


Figure 75. Page principale technologie.

2. Pop -up ajout :

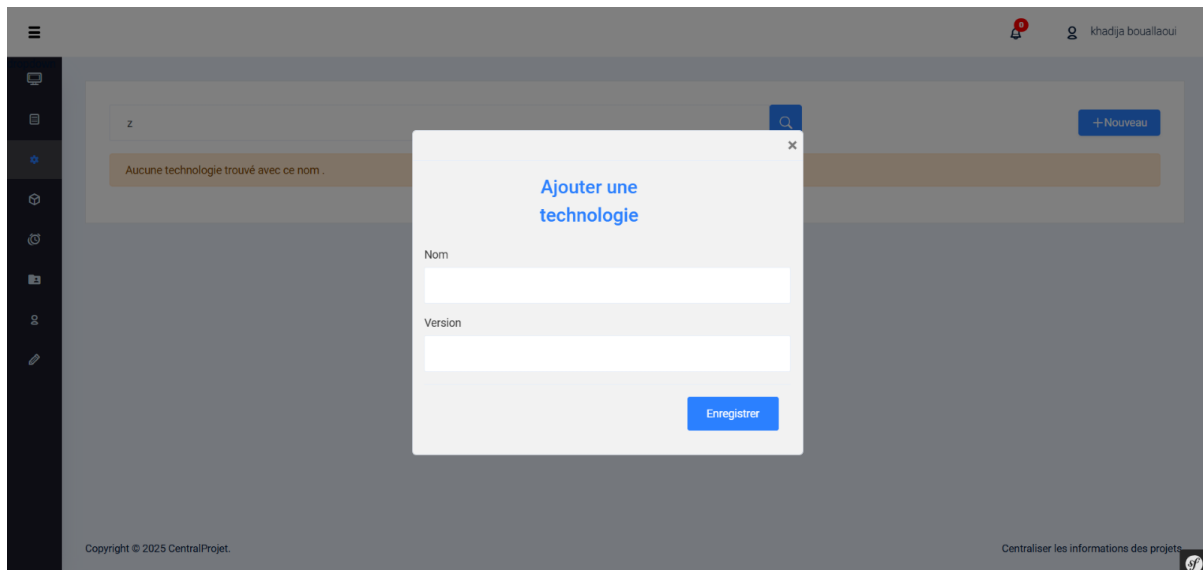


Figure 76. Pop-up ajout technologie.

3. Pop-up modification :

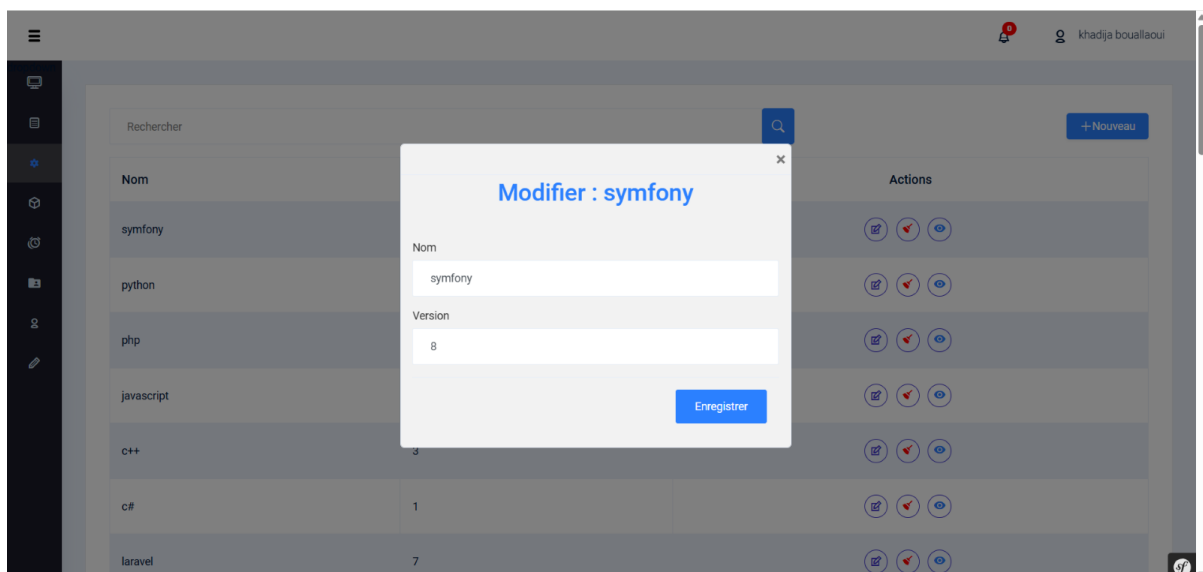


Figure 77. Pop-up modification technologie.

4. Pop-up suppression :

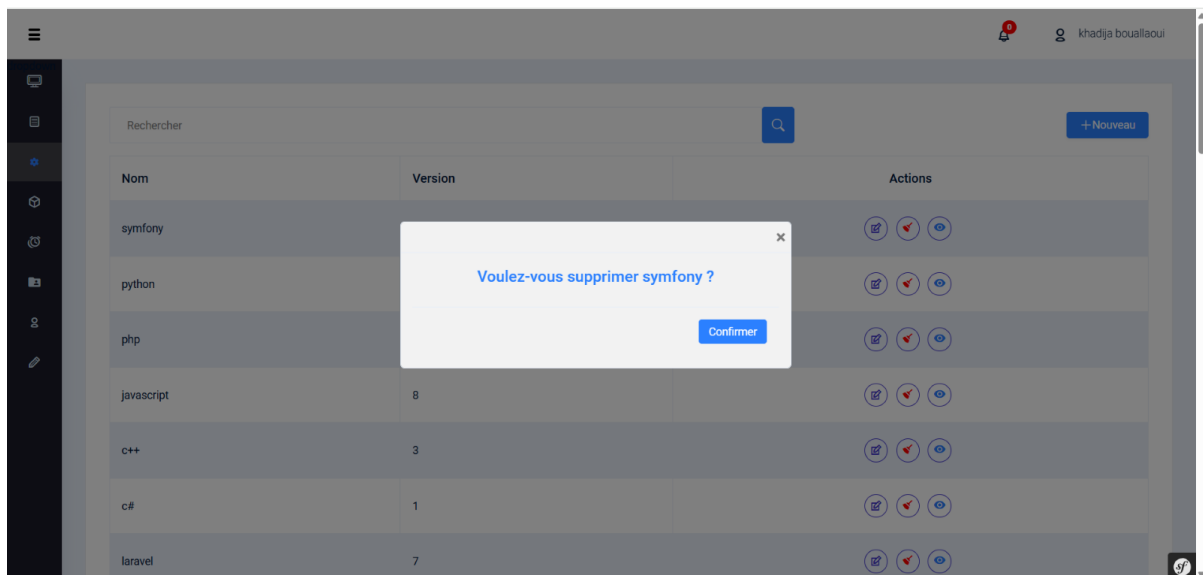


Figure 78. Pop-up suppression technologie.

5. Page de rechercher :

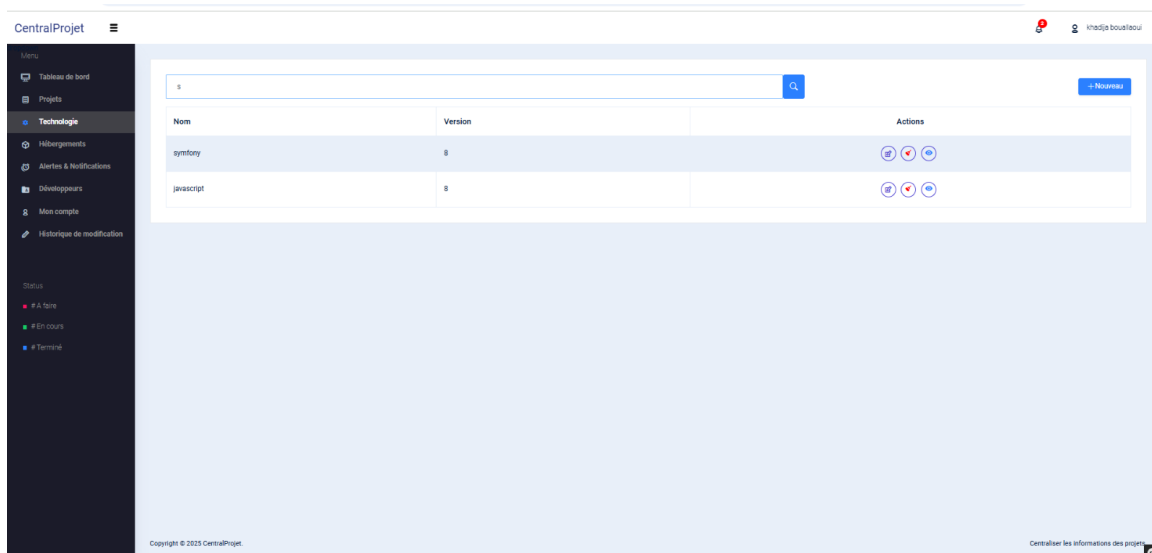


Figure 79. Page de recherche technologie pleine.

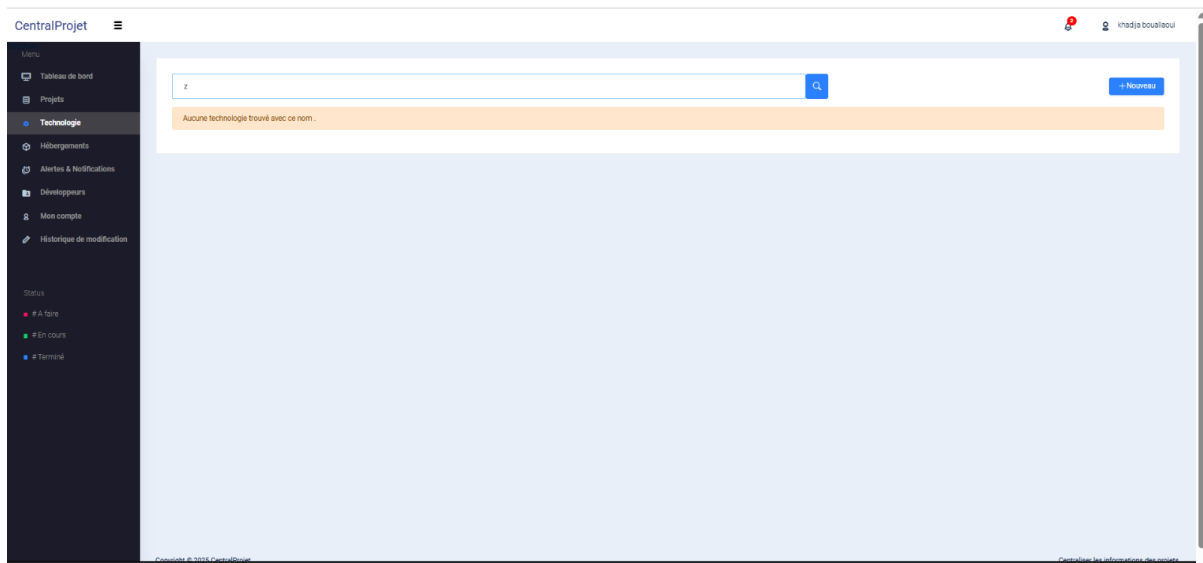


Figure 80. Page de recherche technologie vide.

6. Pop-up voir plus de détails :

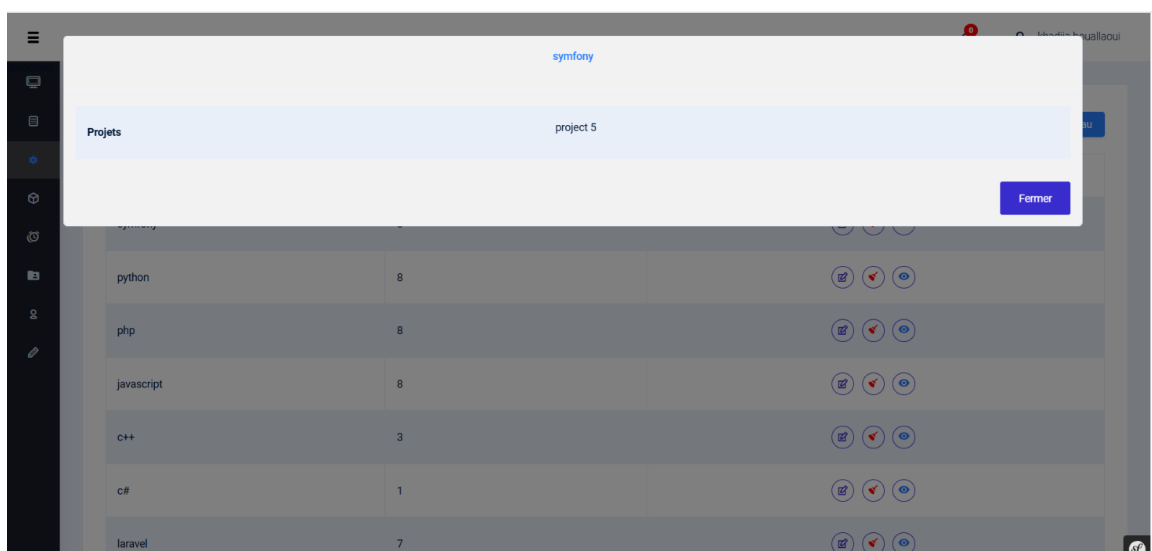


Figure 81. Pop-up plus détail technologie.

e. Gestion Développeur :

1. Page principale :

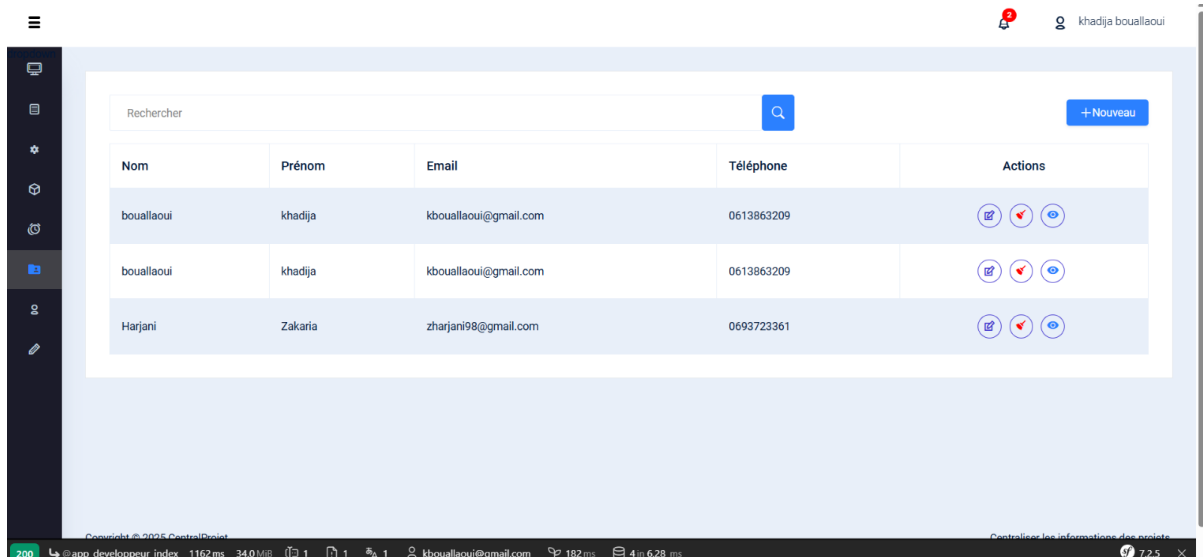


Figure 82. Page principale développeur.

2. Pop-up ajout :

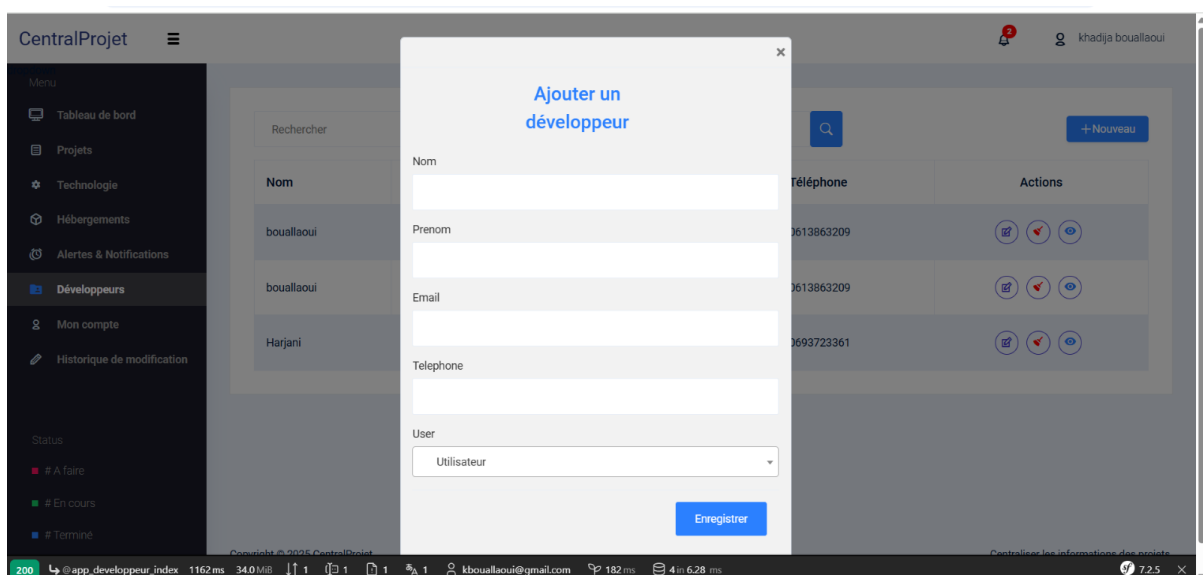


Figure 83. Pop-up ajout développeur.

3. Pop-up modification :

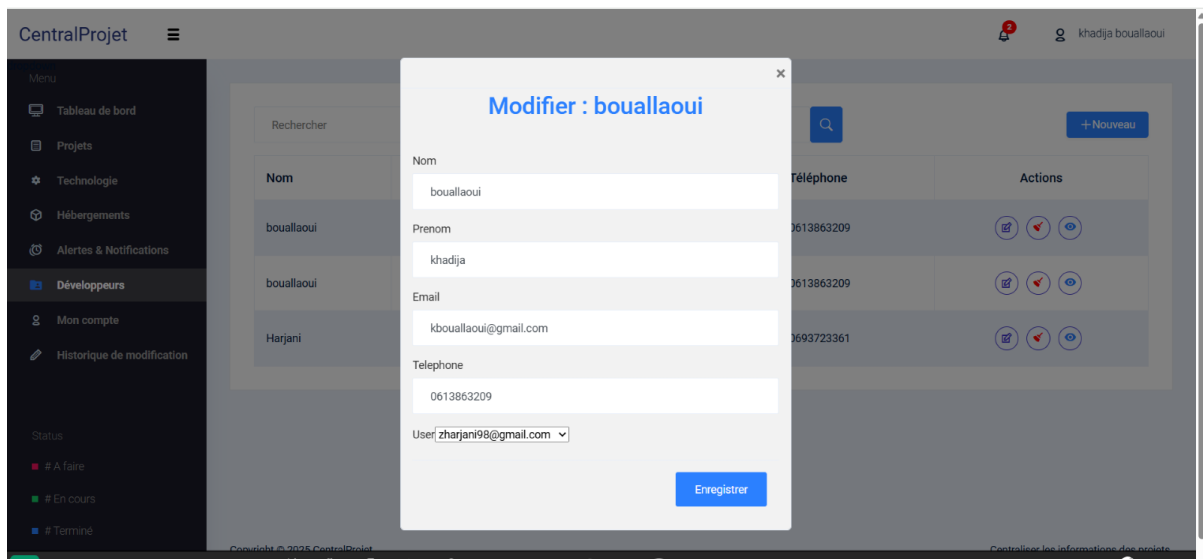


Figure 84. Pop-up modifier développeur.

4. Pop-up suppression :

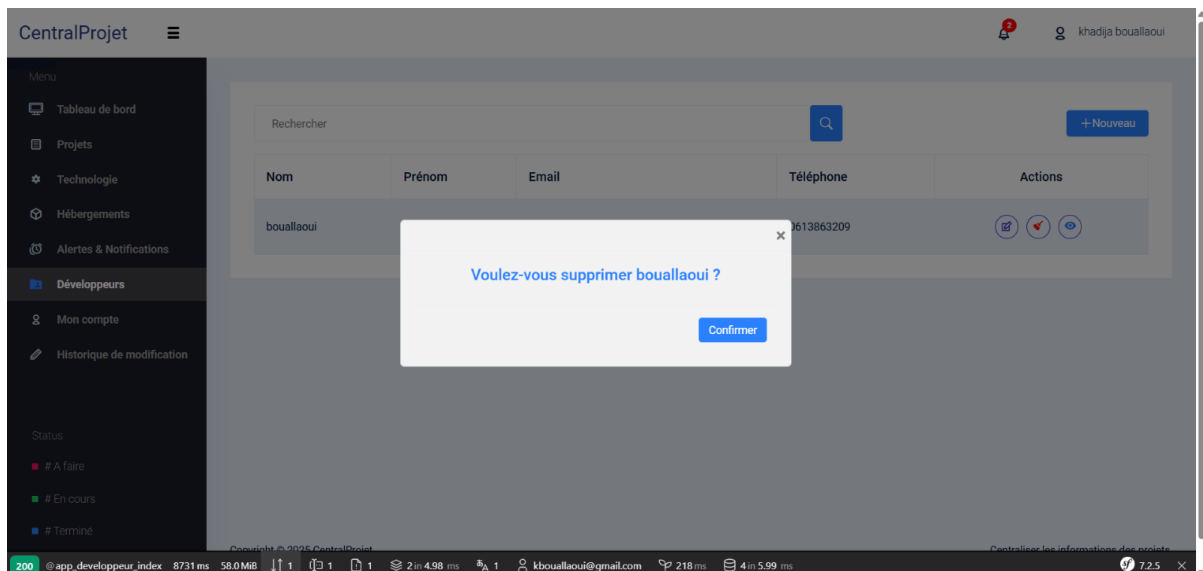


Figure 85. Pop-up suppression développeur.

5. Page de rechercher :

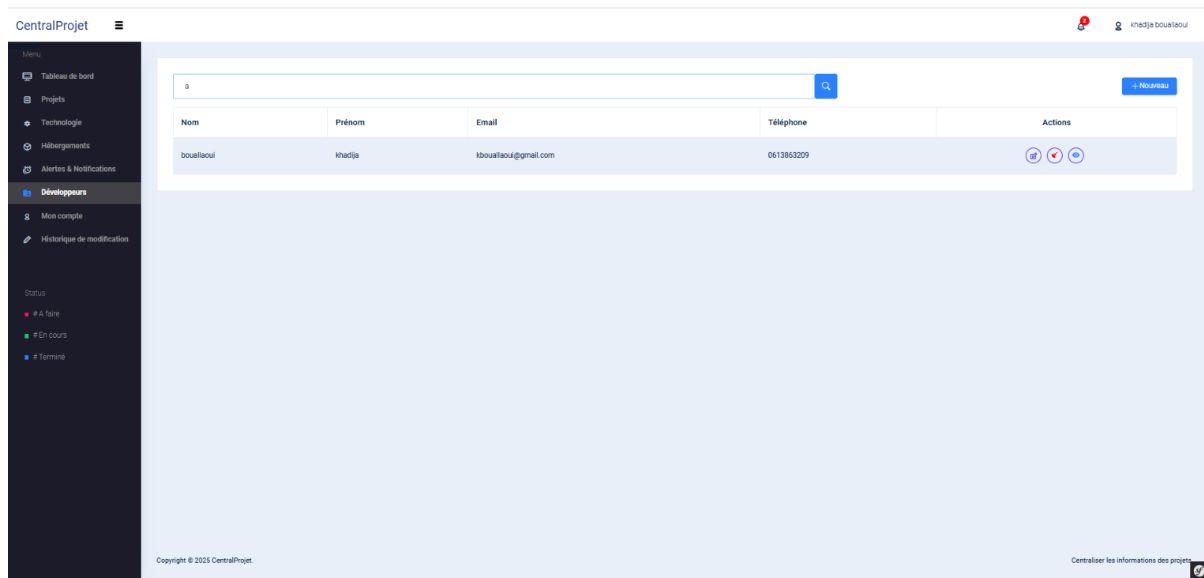


Figure 86. Page de recherche développeur pleine.

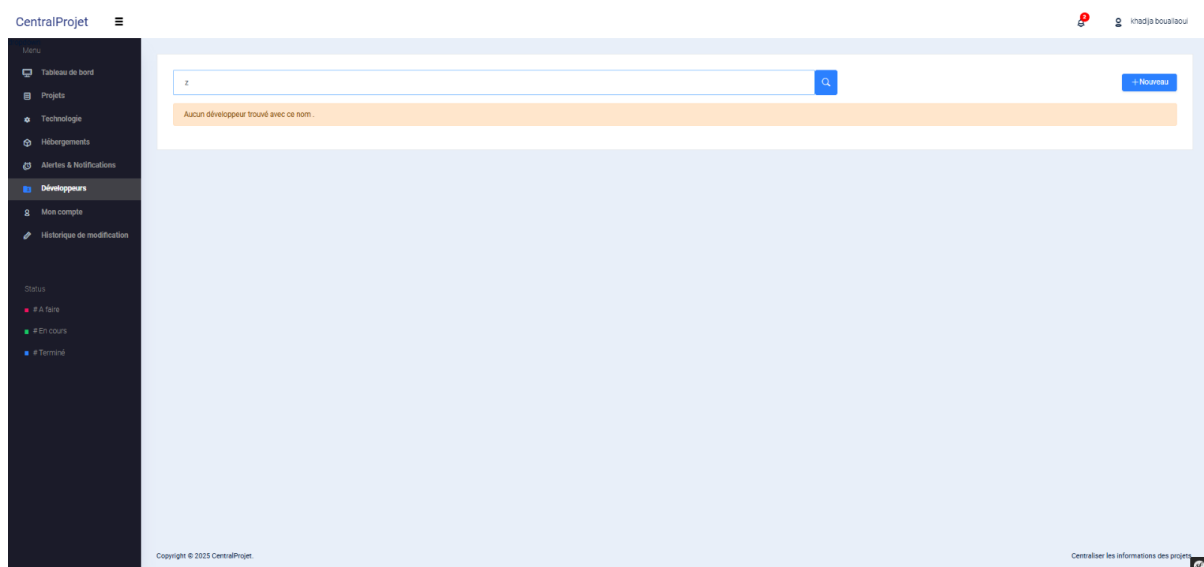


Figure 87. Page de recherche développeur vide.

6. Pop –up voir plus de détails :

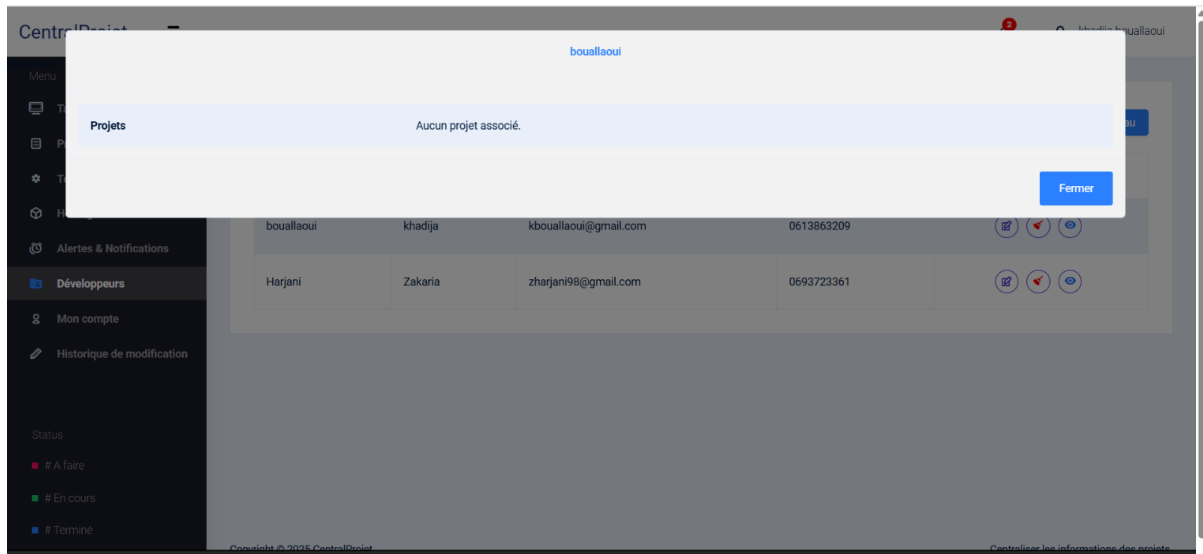


Figure 88. Pop-up plus détails développeur.

f. Consultation Compte :

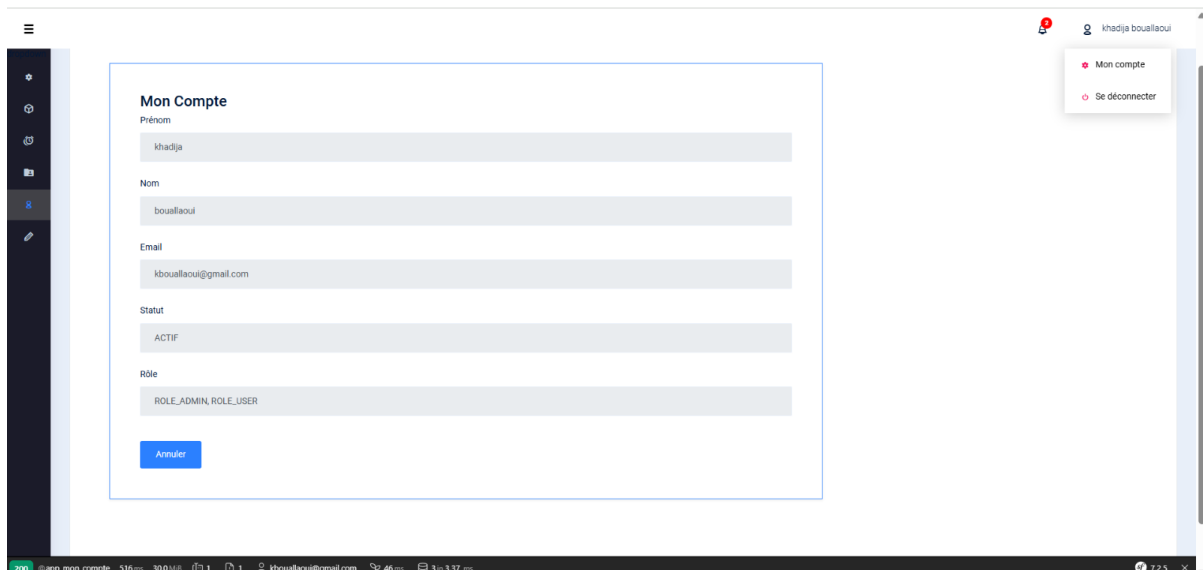


Figure 89. Page Mon compte.

g. Alertes & Notification :

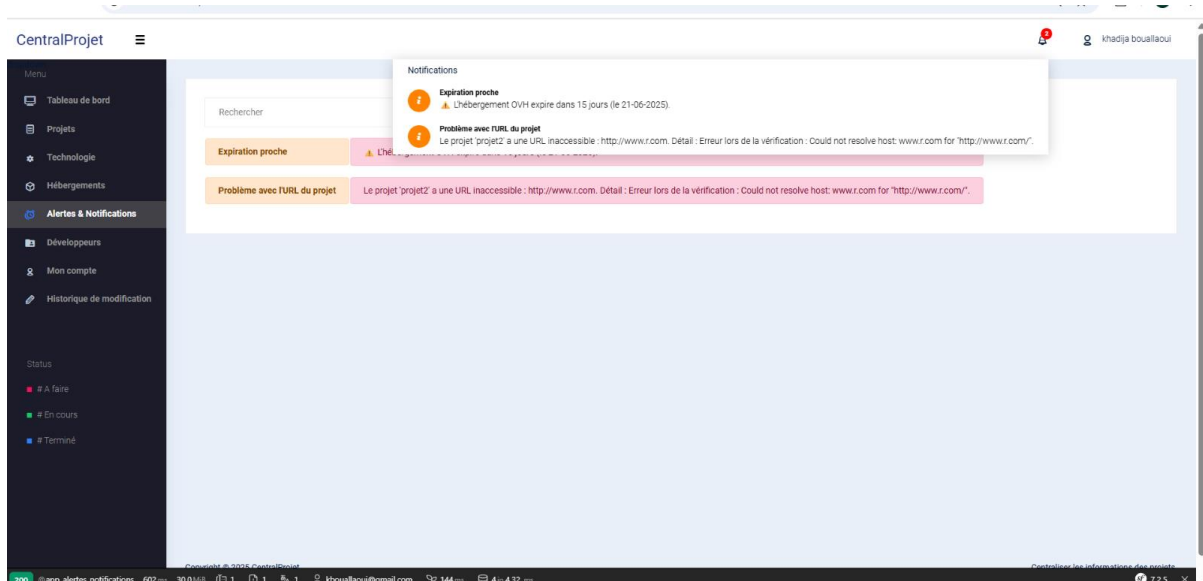


Figure 90. Page Alertes et notifications pleine.

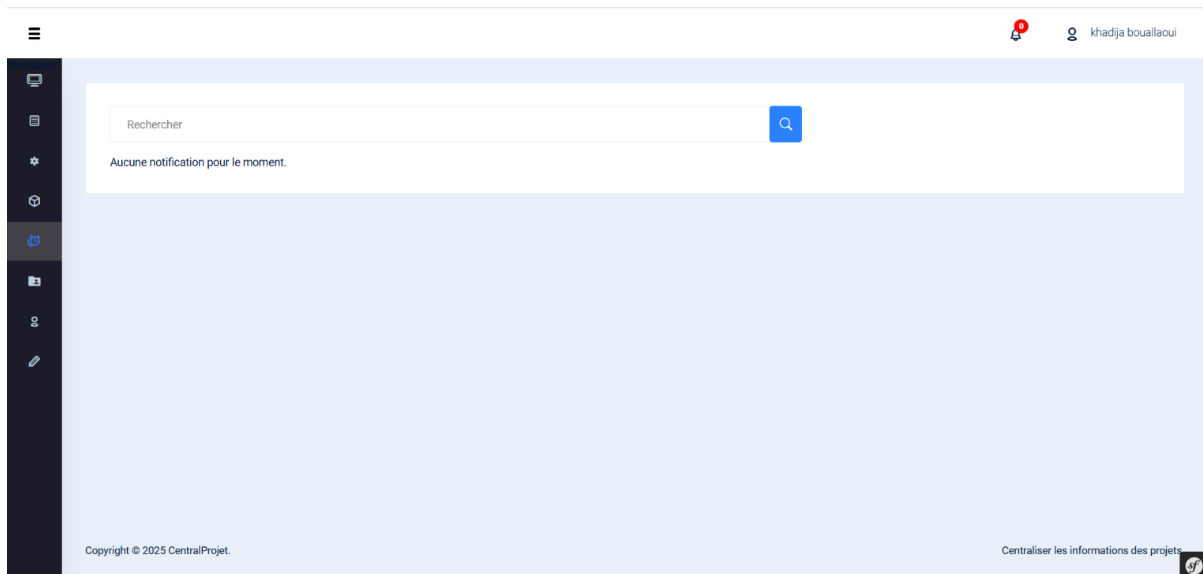


Figure 91. Page Alertes et notifications vide.

h. Suivre des modifications :

Entité modifiée	Utilisateur	Date de modification
Projet (ID: 24)	zharjan98@gmail.com	26-05-2025 15:23:39
Projet (ID: 25)	kbouallaoui@gmail.com	26-05-2025 15:30:23
Technologie (ID: 9)	kbouallaoui@gmail.com	26-05-2025 15:30:41
Projet (ID: 26)	divin@gmail.com	26-05-2025 15:35:53
Hébergement (ID: 2)	divin@gmail.com	26-05-2025 15:41:54
Développeur (ID: 95)	divin@gmail.com	26-05-2025 15:42:09
Projet (ID: 24)	zharjan98@gmail.com	26-05-2025 16:44:30
Projet (ID: 24)	zharjan98@gmail.com	26-05-2025 16:44:45
Projet (ID: 24)	zharjan98@gmail.com	26-05-2025 16:44:57
Projet (ID: 25)	zharjan98@gmail.com	26-05-2025 16:49:06
Projet (ID: 24)	kbouallaoui@gmail.com	27-05-2025 11:09:02
Projet (ID: 24)	kbouallaoui@gmail.com	27-05-2025 11:10:02

Figure 92. Page Historique.

6. Interface Développeur :

a. Page Dashboard user :

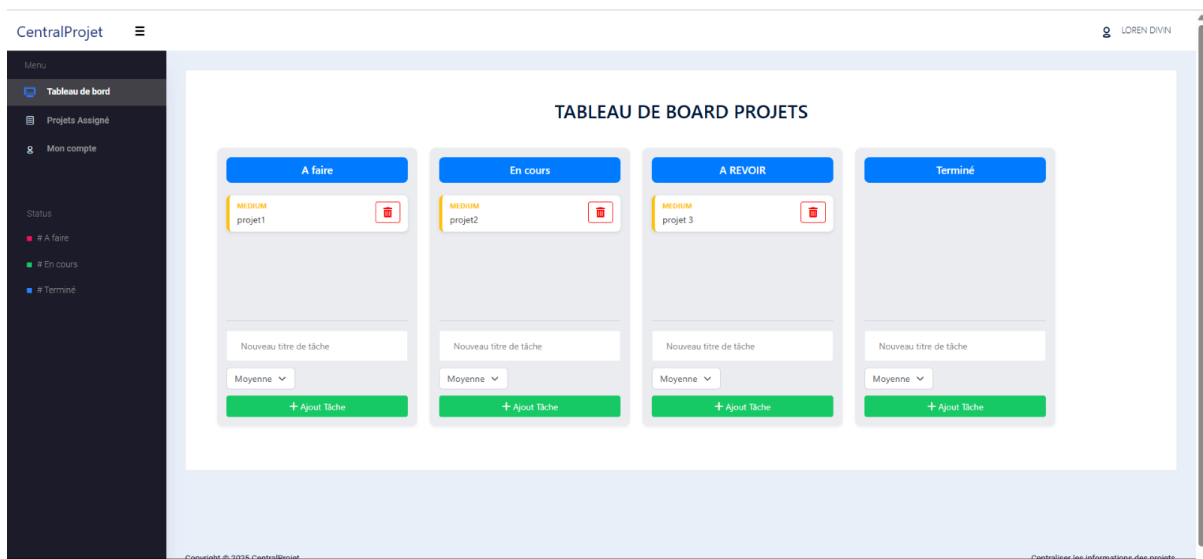


Figure 93. Page Dashboard user.

b. Pop-up voire détail du projet :

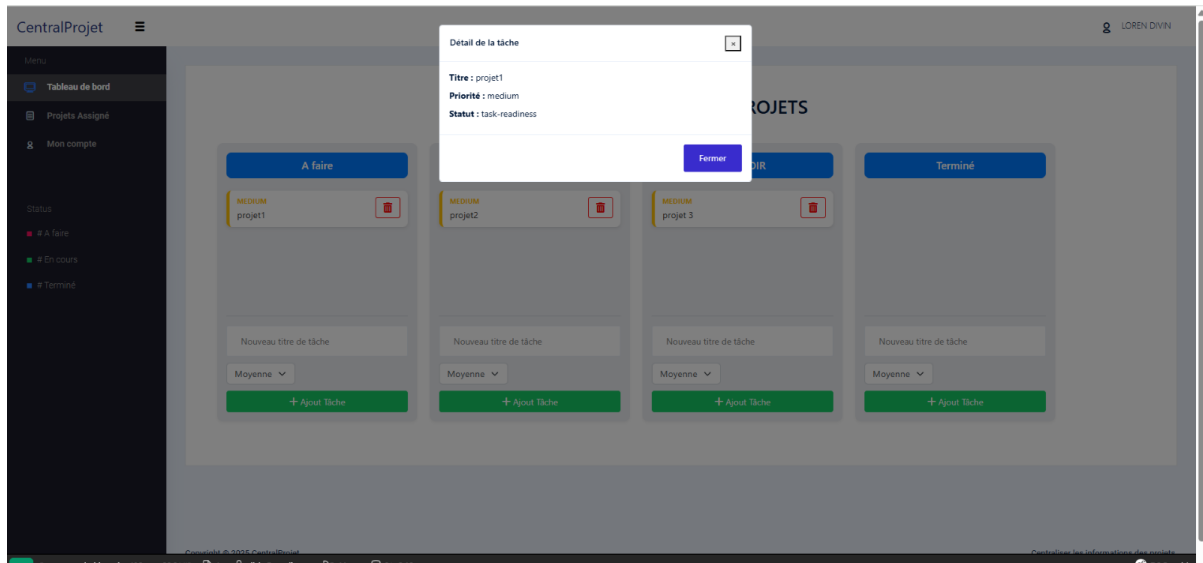


Figure 94. Pop-up plus détail carte projet.

c. Page projet assignées :

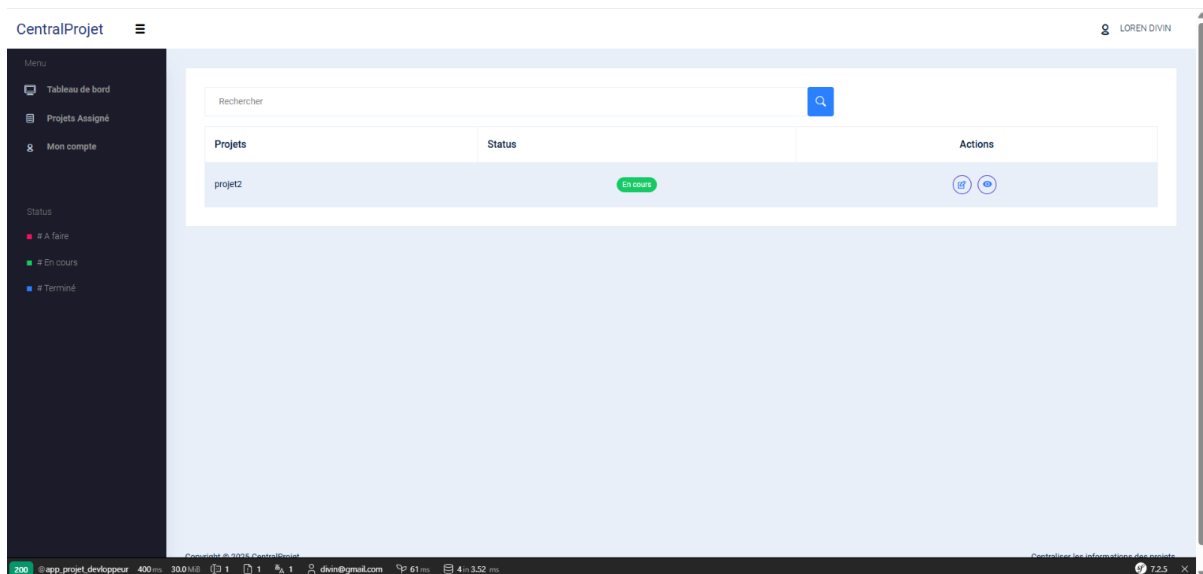


Figure 95 Page projet assignées pleine.

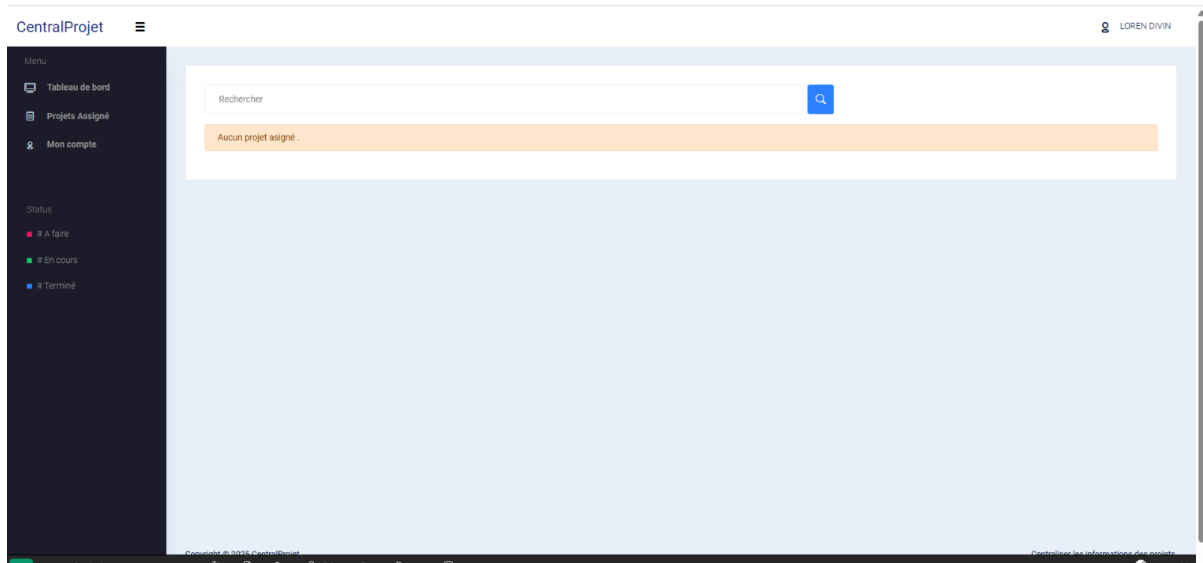


Figure 96. Page projet assignées vide.

d. Page rechercher :

e. Pop-up modification projet :

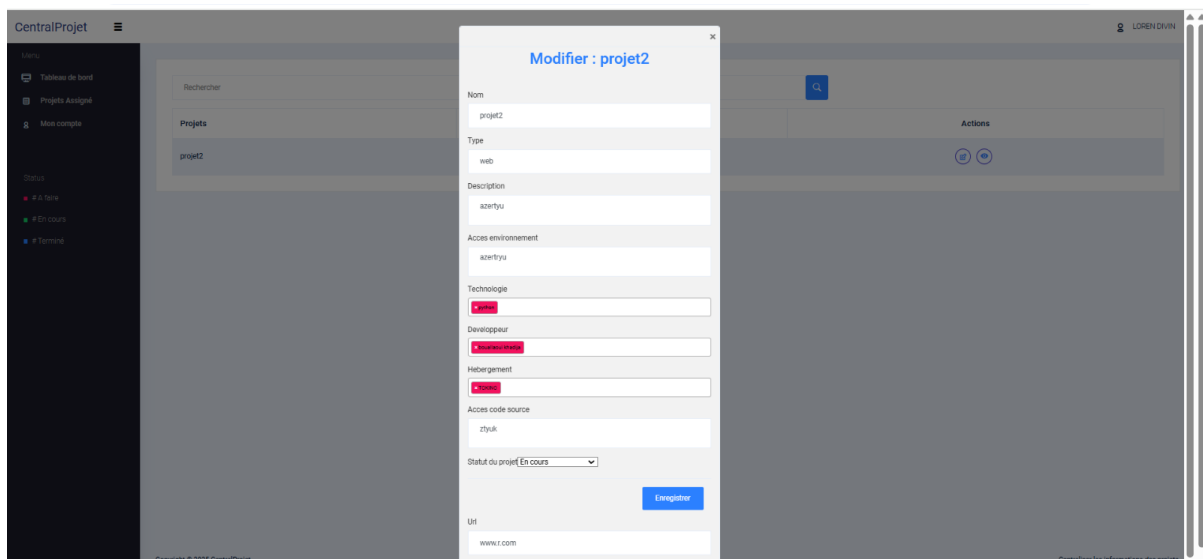


Figure 97. Pop-up modification projet assignées.

f. Pop-up voir plus détails :

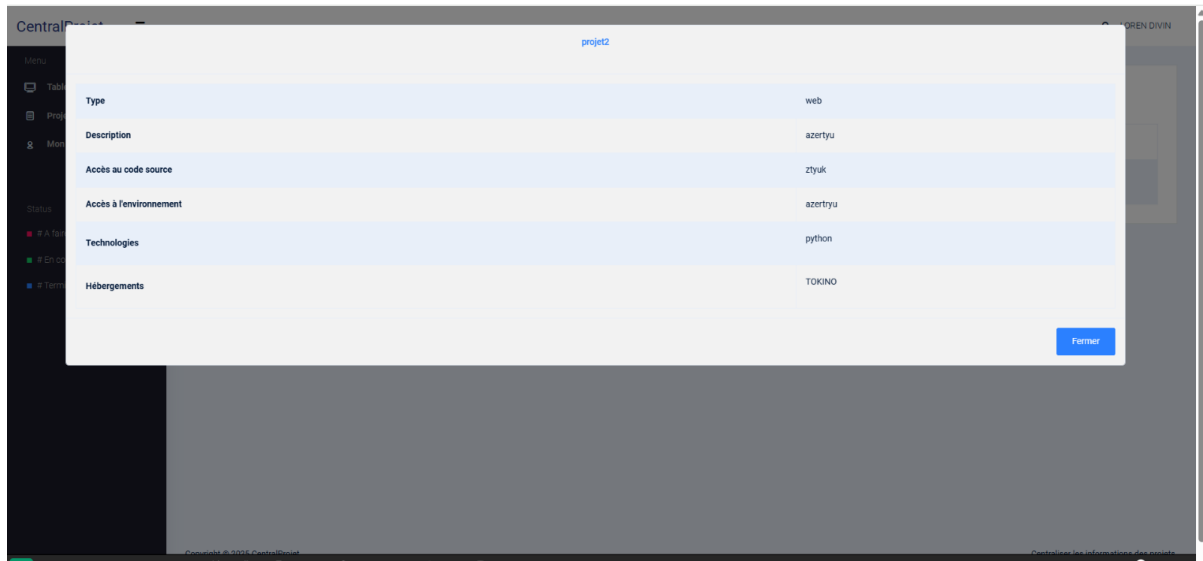


Figure 98. Pop-up plus détail projet assignées.

IV. Les tests réalisés

1. But des tests :

Dans le cadre du développement de notre application web **CentralProjet**, nous avons accordé une attention particulière à la phase de test, essentielle pour garantir la fiabilité, la robustesse et la maintenabilité de notre solution. En effet, les tests permettent de s'assurer que les fonctionnalités développées répondent aux besoins exprimés, qu'elles se comportent correctement dans divers scénarios, et qu'aucune régression n'est introduite lors des évolutions futures du code.

Nous avons principalement mis en œuvre des tests fonctionnels, à l'aide de l'outil PHPUnit, intégré au framework Symfony. Ces tests simulent des actions utilisateurs réelles, comme la création, l'affichage, la modification ou encore la suppression de projets. Chaque scénario testé reflète un cas d'usage concret, permettant ainsi de valider le bon fonctionnement des différents modules de l'application.

Cette approche de test contribue non seulement à améliorer la qualité du code, mais aussi à renforcer la confiance dans le bon comportement de l'application face à des entrées variées ou des conditions inattendues. De plus, elle facilite la détection rapide des erreurs ou anomalies, ce qui réduit considérablement les coûts de correction en phase avancée.

L'intégration des tests dans notre cycle de développement s'inscrit dans une démarche de développement professionnel et durable, visant à livrer une application stable, évolutive et fiable.

2. Types des tests :

a. Test unitaire :

Les tests unitaires constituent une étape fondamentale dans le processus de validation du développement logiciel. Ils ont pour objectif de vérifier, de manière isolée, le bon fonctionnement des plus petites unités de code telles que les fonctions, les méthodes ou les classes. Chaque test unitaire se concentre sur un composant spécifique du programme, en s'assurant que celui-ci réalise bien la logique métier prévue sans dépendre d'autres parties du système.

L'avantage principal des tests unitaires réside dans leur capacité à détecter rapidement les erreurs ou régressions dans le code dès la phase de développement. En testant chaque unité indépendamment, ils facilitent la maintenance et l'évolution du logiciel en garantissant que les modifications n'introduisent pas de dysfonctionnements inattendus.

Dans le cadre d'une application web développée avec Symfony, les tests unitaires permettent notamment de valider :

- Les méthodes des entités (getters/setters, calculs spécifiques) ;
- Les services métiers qui encapsulent la logique applicative ;
- Les EventSubscribers ou EventListeners qui réagissent aux événements du système.

b. Test fonctionnel :

Les tests fonctionnels jouent un rôle clé dans la validation globale de l'application en simulant une interaction utilisateur complète. Contrairement aux tests unitaires, qui vérifient des unités isolées, ces tests reproduisent des scénarios réels d'utilisation, garantissant que les différentes couches du système communiquent correctement pour produire un résultat attendu.

Ils permettent de s'assurer que les actions effectuées via l'interface web, telles que la navigation, la saisie de données dans un formulaire, la soumission ou encore la réception d'un message de confirmation, fonctionnent de manière fluide et cohérente. Ces tests sont souvent réalisés via des navigateurs simulés, ce qui permet de reproduire fidèlement le comportement de l'utilisateur.

Dans un projet Symfony, ils permettent par exemple de tester le parcours complet de création d'un projet, depuis l'affichage du formulaire jusqu'à la confirmation de son enregistrement en base de données, assurant ainsi que l'application répond bien aux exigences métier dans un contexte réaliste.

c. Test d'intégration :

Les tests d'intégration visent à vérifier la coopération entre plusieurs composants du système, en dépassant la vérification isolée des unités de code. Ils garantissent que les différentes

parties de l'application, comme les contrôleurs, les services et la couche de persistance, fonctionnent ensemble de manière harmonieuse.

Ces tests sont essentiels pour détecter les erreurs liées à l'intégration des modules, telles que des incompatibilités dans les données échangées ou des dysfonctionnements dans les interactions avec la base de données.

Dans une application Symfony, un test d'intégration typique peut consister à vérifier qu'un service métier utilise correctement un dépôt pour enregistrer une entité, ou qu'une action de contrôleur persiste bien les données reçues via un formulaire.

d. Test de validation du formulaire :

Les tests de validation de formulaire ont pour objectif de vérifier que les règles définies pour la saisie des données sont bien appliquées avant toute action métier. Ils assurent que les contraintes de validation — telles que les champs obligatoires, les formats spécifiques, ou encore les valeurs minimales et maximales — empêchent la soumission de données incorrectes ou incomplètes.

Ces tests permettent de garantir la robustesse des formulaires et la qualité des données qui seront traitées par l'application.

Dans Symfony, ils consistent notamment à soumettre des données invalides à un formulaire et à vérifier que les messages d'erreur adéquats sont générés, ce qui protège le système contre des données erronées et améliore l'expérience utilisateur.

e. Test de sécurité :

Les tests de sécurité, bien que non automatisés dans ce projet, sont indispensables pour garantir que l'accès aux différentes ressources de l'application est correctement contrôlé. Ils visent à vérifier que seules les personnes autorisées peuvent accéder aux fonctionnalités sensibles, protégeant ainsi les données et les opérations critiques.

Ces tests consistent notamment à s'assurer que les utilisateurs non authentifiés ne peuvent pas atteindre des pages réservées, et que les rôles et permissions sont bien respectés.

Dans ce projet, ces contrôles ont été effectués manuellement en vérifiant notamment la redirection des utilisateurs non connectés vers la page de connexion lorsqu'ils tentent d'accéder à des pages protégées, assurant ainsi la conformité avec les exigences de sécurité.

3. Test réalisé :

a. Test unitaire :

i) Service :

Les tests unitaires des services permettent de vérifier que la logique métier encapsulée dans ces composants fonctionne correctement de manière isolée. Ils garantissent que chaque méthode répond bien aux exigences attendues sans dépendre d'autres parties de l'application.

- NotificationService

```
khadija@LAPTOP-6FHN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$ ./vendor/bin/phpunit tests/EventSubscriber/NotificationSubscriberTest.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing App\Tests\EventSubscriber\NotificationSubscriberTest
...
3 / 3 (100%)

Time: 00:00.041, Memory: 8.00 MB

OK (3 tests, 4 assertions)
```

Figure 99. Résultat test NotificationService.

- UrlService

```
khadija@LAPTOP-6FHN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$ php bin/phpunit tests/Service/UrlServiceTests.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing App\Tests\Service\UrlServiceTests
...
3 / 3 (100%)

Time: 00:00.195, Memory: 8.00 MB

OK (3 tests, 10 assertions)
```

Figure 100. Résultat test UrlService.

- UrlCheckerService

```
$ php bin/phpunit tests/Service/UrlCheckerServiceTests.php
Class UrlCheckerServiceTests could not be found in C:\xampp\htdocs\central_projet\tests\Service\UrlCheckerServiceTests.php

khadija@LAPTOP-6FHN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$ php bin/phpunit tests/Service/UrlCheckerServiceTests.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing App\Tests\Service\UrlCheckerServiceTests
...
3 / 3 (100%)

Time: 00:02.072, Memory: 8.00 MB

OK (3 tests, 13 assertions)
```

Figure 101. Résultat test UrlCheckerService.

- HistoriqueService

```
khadija@LAPTOP-6FHN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$ php bin/phpunit tests/Service/HistoriqueServiceTests.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing App\Tests\Service\HistoriqueServiceTests
..                                                                2 / 2 (100%)

Time: 00:00.041, Memory: 8.00 MB

OK (2 tests, 2 assertions)

khadija@LAPTOP-6FHN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
```

Figure 102. Résultat test HistoriqueService

ii) **Entité :**

Les tests unitaires des entités valident le bon comportement des classes de données, notamment les getters, setters et les méthodes métier intégrées. Ils assurent que les règles de gestion propres à chaque entité sont respectées.

- **Developpeur**

```
khadija@LAPTOP-6FHN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$ ./vendor/bin/phpunit tests/Entity/DeveloppeurTest.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing App\Tests\Entity\DeveloppeurTest
..                                                                2 / 2 (100%)

Time: 00:00.028, Memory: 8.00 MB

OK (2 tests, 10 assertions)

khadija@LAPTOP-6FHN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
```

Figure 103. Résultat test Entity Developpeur.

- **Projet**

```
shadija@LAPTOP-6FVN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$ ./vendor/bin/phpunit tests/Entity/ProjetTest.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing App\Tests\Entity\ProjetTest
..
2 / 2 (100%)

Time: 00:00.045, Memory: 8.00 MB

OK (2 tests, 2 assertions)
shadija@LAPTOP-6FVN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$
```

Figure 104. Résultat test Entity Projet..

- **Hebergement**

```
shadija@LAPTOP-6FVN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$ ./vendor/bin/phpunit tests/Entity/HebergementTest.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing App\Tests\Entity\HebergementTest
...
3 / 3 (100%)

Time: 00:00.033, Memory: 8.00 MB

OK (3 tests, 10 assertions)
shadija@LAPTOP-6FVN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$
```

Figure 105. Résultat test Entity Hebergement..

- **Technologie**

```
shadija@LAPTOP-6FVN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$ php vendor/bin/phpunit tests/Entity/TechnologieTest.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing App\Tests\Entity\TechnologieTest
.
1 / 1 (100%)

Time: 00:00.041, Memory: 8.00 MB

OK (1 test, 3 assertions)
shadija@LAPTOP-6FVN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$
```

Figure 106. Résultat test Entity Technologie.

- **Notification**

```
shadija@LAPTOP-6FVN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$ D:/vendor/bin/phpunit tests/Entity/NotificationTest.php
bash: D:/vendor/bin/phpunit: No such file or directory

shadija@LAPTOP-6FVN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$ ./vendor/bin/phpunit tests/Entity/NotificationTest.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing App\Tests\Entity\NotificationTest
.
1 / 1 (100%)

Time: 00:00.043, Memory: 8.00 MB

OK (1 test, 7 assertions)
shadija@LAPTOP-6FVN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$
```

Figure 107. Résultat test Entity Notification.

- **Historique**

```
shadija@LAPTOP-6FVN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$ ./vendor/bin/phpunit tests/Entity/HistoriqueTest.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing App\Tests\Entity\HistoriqueTest
.
1 / 1 (100%)

Time: 00:00.040, Memory: 8.00 MB

OK (1 test, 4 assertions)
shadija@LAPTOP-6FVN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$
```

Figure 108. Résultat test Entity Historique.

- **User**

```
shadija@LAPTOP-6FVN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$ ./vendor/bin/phpunit tests/Entity/UserTest.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing App\Tests\Entity\UserTest
.
1 / 1 (100%)

Time: 00:00.033, Memory: 6.00 MB

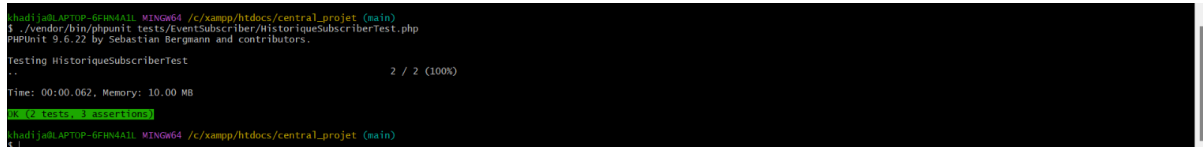
OK (1 test, 9 assertions)
shadija@LAPTOP-6FVN4A1L MINGW64 /c/xampp/htdocs/central_projet (main)
$
```

Figure 109 Résultat test Entity User.

iii) EventSubscriber :

Les tests unitaires des EventSubscribers permettent de s'assurer que les abonnements aux événements fonctionnent bien, que les bonnes actions sont déclenchées en réponse à chaque événement, et que la logique associée est correcte.

- **HistoriqueSubscriber**



```
shadi@LAPTOP-6FHW4A11 MINGW64 /c/xampp/htdocs/central_projet (main)
$ ./vendor/bin/phpunit tests/EventSubscriber/HistoriqueSubscriberTest.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

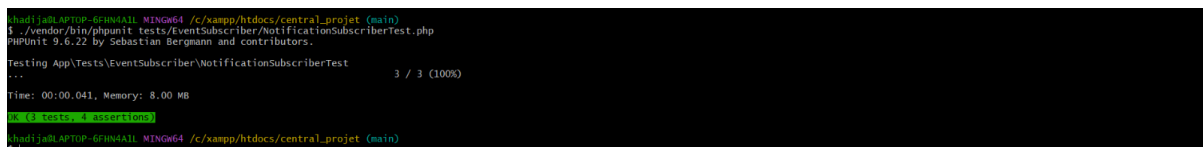
Testing HistoriqueSubscriberTest
...
2 / 2 (100%)

Time: 00:00.062, Memory: 10.00 MB

OK (2 tests, 2 assertions)
shadi@LAPTOP-6FHW4A11 MINGW64 /c/xampp/htdocs/central_projet (main)
```

Figure 110. Résultat test HistoriqueEventSubscribe

- **NotificationSubscriber**



```
shadi@LAPTOP-6FHW4A11 MINGW64 /c/xampp/htdocs/central_projet (main)
$ ./vendor/bin/phpunit tests/EventSubscriber/NotificationSubscriberTest.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing App\Tests\EventSubscriber\NotificationSubscriberTest
...
3 / 3 (100%)

Time: 00:00.041, Memory: 8.00 MB

OK (3 tests, 3 assertions)
shadi@LAPTOP-6FHW4A11 MINGW64 /c/xampp/htdocs/central_projet (main)
```

Figure 111. Résultat test NotificationSubscriber.

V. La qualité du code

Pour assurer la qualité du code dans le projet CentralProjet, j'ai choisi d'utiliser le framework Symfony, reconnu pour sa robustesse et ses bonnes pratiques intégrées. Symfony offre une architecture claire et modulable qui facilite la séparation des responsabilités entre les différentes couches de l'application, ce qui rend le code plus lisible, maintenable et évolutif.

De plus, j'ai mis en place des tests unitaires rigoureux avec PHPUnit, intégrés directement dans le workflow Symfony. Ces tests ont permis de valider efficacement le bon fonctionnement des fonctionnalités développées et d'identifier rapidement toute anomalie ou régression lors des modifications du code. La couverture des tests unitaires a été complète, et leur exécution a toujours abouti avec succès, garantissant ainsi la fiabilité de l'application.

Grâce à cette approche combinant Symfony et les tests unitaires, le projet CentralProjet bénéficie d'un code de haute qualité, facilitant les futures évolutions tout en minimisant les risques d'erreurs.

VI. Diagramme de Gantt réel

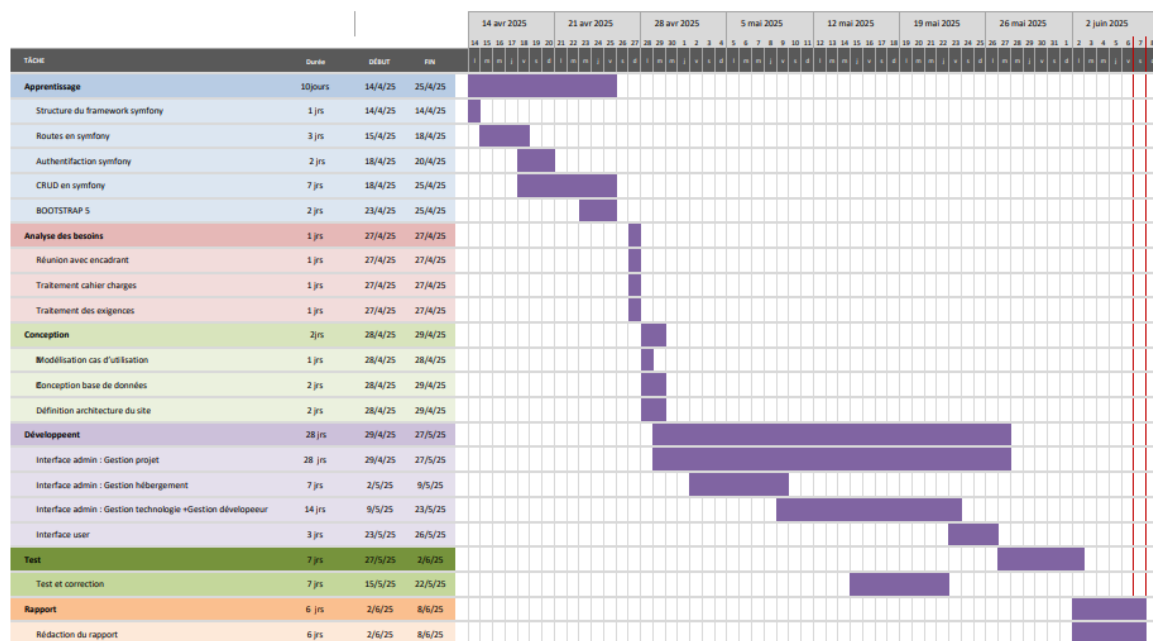


Figure 112. Diagramme de Gantt réel.

VII. Conclusion générale et perspective

1. Conclusion générale :

Le développement de l'application *CentralProjet* représente l'aboutissement d'un projet complet, répondant à un besoin réel de centralisation, de suivi et de gestion des projets au sein de l'entreprise. L'application permet de gérer efficacement les projets, d'en suivre l'évolution, de notifier les utilisateurs en cas d'échéances importantes, et d'archiver les actions effectuées.

Le projet a été réalisé avec le framework Symfony, en s'appuyant sur une architecture logicielle en trois tiers (présentation, logique métier, données), ce qui a permis une meilleure organisation du code, une évolutivité facilitée et une maintenance claire. L'envoi de notifications automatiques a été mis en œuvre à la fois par e-mail personnel et via des alertes directement dans l'application, assurant ainsi une réactivité des utilisateurs face aux événements importants.

Ce projet m'a permis de mettre en pratique les compétences acquises durant ma formation à la Faculté des Sciences et Techniques (FST), notamment en développement backend, en gestion de base de données, en création d'interfaces web ergonomiques et en validation logicielle grâce aux tests unitaires réalisés avec PHPUnit.

Mon intégration dans l'entreprise ISIUM durant ce projet m'a offert un encadrement professionnel précieux, me permettant de développer les compétences attendues dans le cadre de ma formation, tout en me familiarisant avec les pratiques réelles du monde professionnel et les exigences du marché de l'emploi dans le domaine du développement web.

2. Perspective :

Afin d'enrichir davantage *CentralProjet* et de répondre à des besoins plus avancés en entreprise, plusieurs pistes d'amélioration sont envisagées pour les futures évolutions du système :

- Ajout de tableaux de bord dynamiques :
L'intégration de tableaux de bord interactifs permettrait aux utilisateurs de visualiser en temps réel des indicateurs clés (KPI), des statistiques d'avancement, et des graphiques de suivi de performance. Cela offrirait une meilleure lisibilité sur l'état global des projets et permettrait une prise de décision plus rapide et plus éclairée.
- Mise en place d'un système de notifications avancé :
Le système actuel pourrait être amélioré avec :
 - Un centre de notifications accessible depuis l'interface, regroupant toutes les alertes importantes,
 - La possibilité de marquer les notifications comme lues ou non lues, pour mieux les gérer,
 - Des préférences personnalisables permettant à chaque utilisateur de choisir s'il souhaite recevoir les alertes par e-mail, par notification sur l'application, ou les deux, selon les types d'événements.
- Développement d'une application mobile (PWA ou native) :
Pour permettre une consultation rapide et en déplacement, une application mobile pourrait être développée. Celle-ci pourrait être une Progressive Web App (PWA) ou une application native Android/iOS, permettant un accès réactif aux projets, aux notifications et aux tableaux de bord.
- Renforcement de la sécurité :
Pour garantir la protection des données sensibles, plusieurs mécanismes peuvent être intégrés, notamment :
 - L'authentification à deux facteurs (2FA),
 - L'audit de sécurité pour suivre les connexions et les actions critiques,
 - La journalisation avancée des événements pour permettre une traçabilité complète.
- Automatisation des sauvegardes et amélioration de la maintenance :
La mise en place d'un système de sauvegarde automatique permettrait d'assurer la

continuité du service et la récupération des données en cas de panne. Par ailleurs, la rédaction d'une documentation technique claire et complète facilitera la prise en main de l'application par d'autres développeurs ou administrateurs.

VIII. Bibliographie/Webographie

Dans un contexte où la transformation numérique est au cœur des préoccupations des entreprises, la gestion centralisée des projets et des ressources devient essentielle pour améliorer l'organisation interne, optimiser la productivité et garantir la visibilité sur les délais et les responsabilités.

Dans le cadre de mon projet de fin d'études, j'ai conçu et développé une application web nommée CentralProjet, dont l'objectif principal est de centraliser les informations liées aux projets de l'entreprise, d'en assurer le suivi, et de générer des alertes intelligentes, aussi bien sous forme de notifications internes que par e-mail.

Pour réaliser ce projet, j'ai utilisé des technologies modernes et robustes, à savoir :

- Le framework Symfony 7, qui permet de construire des applications web sécurisées, bien structurées et évolutives, en suivant l'architecture MVC ;
- Le framework Bootstrap 5, qui facilite la création d'interfaces réactives et ergonomiques compatibles avec tous les types d'écrans (ordinateurs, tablettes, smartphones).

Ces outils m'ont permis de mettre en œuvre mes compétences en développement backend, en conception d'interfaces, et en gestion de données, tout en respectant les bonnes pratiques de développement et d'organisation logicielle.

- **Symfony 7** : <https://symfony.com>
- **Bootstrap 5** : <https://getbootstrap.com>

IX. Annexes

Annexe 1 : Captures écrans script des tâches crônes :

!

```
PS C:\WINDOWS\system32> # Chemin complet vers php.exe (modifie si besoin)
>> $phpPath = "C:\xampp\php\php.exe"
>>
>> # Chemin complet vers la console Symfony
>> $symfonyConsolePath = "C:\xampp\htdocs\central_projet\bin\console"
>>
>> # Commande Symfony à exécuter
>> $command = "app:check-urls"
>>
>> # Action : lancer php avec la commande Symfony
>> $action = New-ScheduledTaskAction -Execute $phpPath -Argument "$symfonyConsolePath" $command"
>>
>> # Déclencheur : tous les jours à 2h00 du matin
>> $trigger = New-ScheduledTaskTrigger -Daily -At 2:00AM
>>
>> # Principale : exécuter en tant que SYSTEM (ou change pour un utilisateur spécifique)
>> $principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -RunLevel Highest
>>
>> # Paramètres optionnels
>> $settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries
>>
>> # Création de la tâche planifiée
>> Register-ScheduledTask -TaskName "SymfonyCheckUrls" -Action $action -Trigger $trigger -Principal $principal -Settings $settings
>>
>> Write-Output "Tâche planifiée créée : SymfonyCheckUrls (exécution quotidienne à 2h00)"
>>
TaskPath                                     TaskName                                     State
-----
\
Task planifiée créée : SymfonyCheckUrls (exécution quotidienne à 2h00)
```

```
Copyright (C) Microsoft Corporation. Tous droits réservés.
Installez la dernière version de PowerShell pour de nouvelles fonctionnalités et améliorations ! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> # Chemin complet vers php.exe (à adapter si besoin)
>> $phpPath = "C:\xampp\php\php.exe"
>>
>> # Chemin complet vers la console Symfony
>> $symfonyConsolePath = "C:\xampp\htdocs\central_projet\bin\console"
>>
>> # Commande Symfony à exécuter
>> $command = "app:check-expirations"
>>
>> # Action : lancer php avec la commande symfony
>> $action = New-ScheduledTaskAction -Execute $phpPath -Argument "$symfonyConsolePath $command"
>>
>> # Déclencheur : tous les jours à 2h00 du matin (modifier si besoin)
>> $trigger = New-ScheduledTaskTrigger -Daily -At 2:00AM
>>
>> # Principale : exécuter en tant que SYSTEM (ou utilisateur connecté)
>> $principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -RunLevel Highest
>>
>> # Paramètres optionnels
>> $settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries
>>
>> # Création de la tâche planifiée
>> Register-ScheduledTask -TaskName "SymfonyCheckExpirations" -Action $action -Trigger $trigger -Principal $principal -Settings $settings
>>
>> Write-Output "Tâche planifiée créée : SymfonyCheckExpirations (exécution quotidienne à 2h00)"
>>
TaskPath                                     TaskName                                     State
-----
\
Task planifiée créée : SymfonyCheckExpirations (exécution quotidienne à 2h00)

PS C:\WINDOWS\system32>
```

Annexe 3 : Accès code source :

Le code source complet du projet **Central-Projet** est accessible sur GitHub à l'adresse suivante :

<https://github.com/khadija-png1/Central-Projet>

